

*Communauté Économique et Monétaire de
l'Afrique Centrale
(CEMAC)*



*Institut Sous-régional de Statistique
et d'Économie Appliquée*

Organisation Internationale

BP : 294 Yaoundé - Tél : 222 220 134

Cameroun AI Enthousiaste



*Direction du Développement et de la
Recherche*

ONG camerounaise

BP : 83000 Toulon, France

*Mémoire professionnel de fin d'étude présenté en vue de l'obtention du diplôme d'Ingénieur
Statisticien Économiste(ISE)*

OPTION : Data Science et Marketing

**CONCEPTION D'UN SYSTEME DE RECOMMANDATION HYBRIDE POUR LE
MATCHING EMPLOI-COMPETENCES AU CAMEROUN PAR INTEGRATION DES
LLMS ET DES GRAPHS DE CONNAISSANCES**

Rédigé par :

NGOULOU NGOUBILI Irch Defluviaire

Élève Ingénieur Statisticien Économiste cycle long – 5^e année

Sous l'encadrement de :

Dr. FOUTSE Yuehgoh

Présidente & Directrice exécutive adjointe, PhD Computer Science & Data Scientist

Année académique 2025-2026

Dédicace

À mes parents... (Max 1 page)

Remerciements

Sommaire

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	v
Liste des tableaux	vi
Liste des figures	viii
Avant-propos	ix
Résumé	x
Abstract	xi
Introduction générale	1
I Cadre théorique et conceptuel	6
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	7
1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG	7
1.2 Fondements économiques et problématique d'appariement	9
1.3 Systèmes de recommandation et person-job fit	10
1.4 LLMs, embeddings et graphes de connaissances	12
1.5 RAG, GraphRAG et orchestration agentique	14
2 Source de données et Approche méthodologique	19
2.1 Cadre méthodologique et sources de données	19
2.2 Modélisation sémantique et structuration des connaissances	25
2.3 Orchestration agentique et génération contrôlée	28

II	Présentation des résultats	31
3	Construction opérationnelle du système hybride emploi-compétences	32
3.1	Données exploitables et contraintes de qualité du système	32
3.2	Adaptation du modèle d’embeddings au domaine emploi-compétences	35
3.3	Mémoire vectorielle pgvector pour le retrieval	41
3.4	Neo4j : mémoire relationnelle du matching	46
3.5	Score hybride et reranking des recommandations	52
4	Évaluation de la performance du système	57
4.1	Protocole expérimental et métriques retenues	57
4.2	Retrieval sémantique et apport mesuré du graphe	58
4.3	Validation fonctionnelle du GraphRAG	60
4.4	Reranking hybride et calibration du score	62
4.5	Contrôle de génération par critic	65
4.6	Orchestration agentique avec LangGraph	67
	Conclusion générale	I
	Bibliographie	II
A	Annexes	VI

Liste des sigles et abréviations

ANN :	Approximate Nearest Neighbors
API :	Application Programming Interface
BERT :	Bidirectional Encoder Representations from Transformers
BI :	Business Intelligence
CEMAC :	Communauté Économique et Monétaire de l'Afrique Centrale
CV :	Curriculum Vitae
DCG :	Discounted Cumulative Gain
ESCO :	European Skills, Competences, Qualifications and Occupations
FAISS :	Facebook AI Similarity Search
FNE :	Fonds National de l'Emploi
FRED :	Federal Reserve Economic Data
GAT :	Graph Attention Network
GCN :	Graph Convolutional Network
GNN :	Graph Neural Network
GPT :	Generative Pre-trained Transformer
GPU :	Graphics Processing Unit
HNSW :	Hierarchical Navigable Small World
IA :	Intelligence Artificielle
IDCG :	Ideal Discounted Cumulative Gain
ISE :	Ingénieur Statisticien Économiste
ISSEA :	Institut Sous-régional de Statistique et d'Économie Appliquée
IVF :	Inverted File Index
KG :	Knowledge Graph
LLM :	Large Language Model
LoRA :	Low-Rank Adaptation
LSTM :	Long Short-Term Memory
MAP :	Mean Average Precision
MEPC :	Nomenclature Camerounaise des Métiers, Emplois et Professions
MRR :	Mean Reciprocal Rank
NCF :	Nomenclature Camerounaise des Formations
NCM :	Nomenclature Camerounaise des Métiers
NDCG :	Normalized Discounted Cumulative Gain
NLP :	Natural Language Processing
OIT :	Organisation Internationale du Travail
ONG :	Organisation Non Gouvernementale
QLoRA :	Quantized Low-Rank Adaptation
RAG :	Retrieval-Augmented Generation
RNN :	Recurrent Neural Network
RWKV :	Receptance Weighted Key Value
SEO :	Search Engine Optimization
SGPT :	Sentence Generative Pre-trained Transformer
SQL :	Structured Query Language
SR :	Système de Recommandation

Liste des tableaux

Tableau 2.1	Sources de données mobilisées et utilisation méthodologique	23
Tableau 3.1	Protocole de construction des paires de fine-tuning	36
Tableau 3.2	Hyperparamètres du fine-tuning SentenceTransformer	37
Tableau 3.3	Distribution des embeddings par type d'entité	43
Tableau 3.4	Résultats de calibration des poids du score hybride	54
Tableau 4.1	Benchmark des modèles d'embeddings sur 473 requêtes de test	58
Tableau 4.2	Comparaison retrieval : pgvector seul versus pgvector + graphe	59
Tableau 4.3	Pondérations retenues selon la fonction objectif du score hybride . . .	63
Tableau A.1	Labels de nœuds et relations structurantes du graphe	VI
Tableau A.2	Volume d'entités indexées dans la base vectorielle pgvector	VII
Tableau A.3	Pipeline général implémenté : étapes, entrées et sorties	VII
Tableau A.4	Correspondance entre objectifs spécifiques et choix méthodologiques	VIII
Tableau A.5	Volumes réellement exploités par les modules de recommandation . .	IX
Tableau A.6	Synthèse des traitements de nettoyage utiles au moteur de recomman- dation	X
Tableau A.7	Comparaison directe entre le baseline et le modèle fine-tuné	XI
Tableau A.8	Diagnostic statistique du chunking documentaire	XI
Tableau A.9	Couverture documentaire et duplication des chunks	XII
Tableau A.10	Cohérence sémantique intra-chunk	XII
Tableau A.11	Indicateurs techniques pgvector	XII
Tableau A.12	Index pgvector et index complémentaires	XIII
Tableau A.13	Hétérogénéité des degrés par famille de noeuds	XIV
Tableau A.14	Exemples de compétences centrales selon trois métriques	XV
Tableau A.15	Résumé du déplacement de rang après reranking Neo4j	XVI

Table des figures

Figure 1.1	Évolution simplifiée des modèles de langage et des représentations textuelles	12
Figure 1.2	Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG	15
Figure 2.1	Architecture méthodologique du workflow agentique de recommandation	21
Figure 3.1	Répartition des offres par source après nettoyage	33
Figure 3.2	Compétences fréquentes et disponibilité descriptive	34
Figure 3.3	Diagnostic du fine-tuning : perte, généralisation, métriques de validation et calendrier du learning rate	38
Figure 3.4	Comparaison des modèles d'embeddings : qualité de classement, rappel et latence	39
Figure 3.5	Projection UMAP des embeddings indexés	40
Figure 3.6	Schéma physique de la base pgvector : tables documentaires, embeddings et index de recherche	42
Figure 3.7	Pipeline de chunking structurel des référentiels documentaires	44
Figure 3.8	Distribution des tailles, sources et couverture du chunking documentaire	45
Figure 3.9	Distribution de la cohérence sémantique intra-chunk par source	45
Figure 3.10	Schéma logique du graphe de connaissances	46
Figure 3.11	Lecture conjointe des noeuds et relations Neo4j mobilisés par le matching	47
Figure 3.12	Tableau de bord statistique du graphe Neo4j	48
Figure 3.13	Centralités des compétences : degré, PageRank et intermédiarité approximée	50
Figure 3.14	Distribution comparée des scores pgvector et Neo4j utilisés dans le reranking	53
Figure 3.15	Effet du reranking Neo4j sur les métriques et les déplacements de rang	55
Figure 4.1	Comparaison des modèles d'embeddings sur le jeu de test	59
Figure 4.2	Effet de l'enrichissement graphe sur les métriques de retrieval	60
Figure 4.3	Évaluation fonctionnelle des requêtes Neo4j utiles au GraphRAG	61
Figure 4.4	Distribution des déplacements de rang après reranking par le graphe	63
Figure 4.5	Évolution de l'objectif NDCG@10 au cours des essais Optuna	64
Figure 4.6	Comparaison des pondérations retenues selon la fonction objectif	64
Figure 4.7	Distribution des scores du critic avec contexte réel et contexte mélangé	65
Figure 4.8	Effet du seuil du critic sur les décisions accept/revise	66
Figure 4.9	Workflow agentique orchestré dans LangGraph Studio	67
Figure A.1	Localisation nettoyée des offres	X
Figure A.2	Composition du graphe par types de noeuds et relations	XIII
Figure A.3	Distribution des degrés dans la projection matching	XIV
Figure A.4	Communautés Louvain, distances et composantes connexes	XIV
Figure A.5	Robustesse de la projection matching après retrait des hubs	XV

Figure A.6	Compétences par offre, candidat et métier dans Neo4j	XV
Figure A.7	Distribution échantillonnée du skill gap candidat-offre	XVI

Avant-propos

L'Institut Sous-régional de Statistique et d'Économie Appliquée (ISSEA), institution spécialisée de la CEMAC créée en 1984, a pour mission principale la formation de cadres supérieurs en statistique, économie et data science, capables de répondre aux enjeux contemporains de décision fondée sur les données. Dans ce cadre, l'ISSEA propose plusieurs filières de formation, parmi lesquelles figure celle des Ingénieurs Statisticiens Économistes (ISE), alliant rigueur mathématique, modélisation économétrique et outils modernes d'analyse de données (Data Science).

Arrivés à un niveau avancé de leur formation, les élèves Ingénieurs Statisticiens Économistes sont tenus d'effectuer un stage professionnel d'une durée minimale de trois (03) mois. Ce stage constitue une phase essentielle du cursus, permettant de confronter les connaissances théoriques acquises à des problématiques réelles, tout en favorisant l'acquisition de compétences opérationnelles en environnement professionnel. Il est sanctionné par la rédaction d'un mémoire professionnel, soutenu devant un jury.

C'est dans ce cadre que nous avons effectué un stage du 9 mars 2026 au 30 juin 2026 au sein de KmerAI, où nous avons été intégrés à la Direction du Développement et de la Recherche. Cette immersion nous a permis de travailler sur une problématique appliquée située au croisement de l'analyse de données, de l'intelligence artificielle générative, du traitement automatique du langage naturel et de l'aide à la décision : l'amélioration du matching entre offres d'emploi, profils de candidats et compétences recherchées sur le marché camerounais.

Le présent mémoire s'inscrit dans cette dynamique et porte sur le thème : « Conception d'un système de recommandation hybride pour le matching emploi-compétences au Cameroun par intégration des LLMs et des graphes de connaissances ». Il vise principalement à concevoir une architecture capable de structurer les données d'offres et de profils, d'exploiter les référentiels métiers et formations, puis de produire des recommandations plus explicables que les approches fondées uniquement sur des mots-clés. Plus spécifiquement, il s'agit de normaliser les données disponibles, d'aligner les métiers et compétences avec les référentiels ESCO, MEPC et NCF, d'adapter un modèle d'embeddings au domaine emploi-compétences, de construire un graphe de connaissances sous Neo4j et de poser les bases d'un moteur de recommandation hybride intégrant similarité sémantique, relations de graphe et analyse du *skill gap*.

Ce travail répond à un double enjeu. Sur le plan scientifique et méthodologique, il cherche à montrer comment les modèles de représentation sémantique, les bases vectorielles et les graphes de connaissances peuvent être combinés pour traiter un problème réel d'appariement sur le marché du travail. Sur le plan opérationnel, il propose une démarche reproductible susceptible d'aider KmerAI à développer des outils de recommandation, d'orientation et de montée en compétences adaptés aux besoins des candidats, des recruteurs et des acteurs de la formation.

Résumé

Mots-clés : Économétrie, Statistique, Développement...

Abstract

INTRODUCTION GÉNÉRALE

Contexte et justification

Le fonctionnement du marché de l'emploi constitue un déterminant central de la performance économique et sociale d'un pays. Au Cameroun, ce marché reste marqué par des déséquilibres structurels persistants, notamment l'inadéquation entre les compétences disponibles et les besoins des entreprises, la forte informalité et les difficultés d'insertion professionnelle des jeunes. Les données disponibles montrent que le chômage des jeunes demeure une réalité importante, avec un taux estimé à 6,6% en 2024 et 6,5% en 2025 selon les estimations modélisées de l'OIT relayées par la Banque Mondiale et la base FRED. Par ailleurs, plusieurs travaux sur le Cameroun soulignent que le problème ne tient pas uniquement au manque d'emplois, mais aussi à un mismatch entre formation, compétences et exigences du marché.

Dans ce contexte, la question du matching emploi-compétences devient centrale. En théorie, un marché du travail efficient devrait permettre d'associer rapidement les profils disposant des compétences requises aux postes vacants correspondants. En pratique, plusieurs frictions informationnelles persistent : asymétrie d'information entre recruteurs et candidats, faible structuration des profils de compétences, hétérogénéité des offres d'emploi et faiblesse des systèmes d'intermédiation comme le FNE et certains agences de recrutement. Ces dysfonctionnements conduisent à une situation paradoxale où coexistent des postes non pourvus et des chercheurs d'emploi, traduisant une inefficacité allocative du marché du travail.

Par ailleurs, l'essor des technologies numériques et de l'intelligence artificielle ouvre des perspectives nouvelles pour améliorer l'appariement entre offres et profils. Dans cette dynamique, KmerAI se positionne comme une plateforme camerounaise dédiée à la promotion de l'intelligence artificielle, à la vulgarisation de ses usages et à l'accompagnement des étudiants, chercheurs et entreprises dans l'adoption de solutions innovantes adaptées au contexte local. Ses missions consistent notamment à former aux bases de l'IA, à offrir un espace de partage d'expériences et à encourager la collaboration autour de cas d'usage appliqués à des secteurs stratégiques tels que la santé, l'agriculture, le transport, l'éducation et le marketing.

Cette orientation est particulièrement pertinente pour le marché de l'emploi camerounais, où les données liées aux offres, aux CV et aux compétences restent souvent dispersées et peu exploitées. Une approche fondée sur l'intelligence artificielle peut permettre d'analyser plus efficacement ces données hétérogènes afin de proposer des recommandations d'emploi plus

pertinentes, plus rapides et plus personnalisées.

Problématique

Le marché de l'emploi camerounais est marqué par une forte asymétrie d'information, une hétérogénéité des profils professionnels et des offres d'emploi, ainsi qu'une faible capacité des outils classiques à évaluer les écarts réels de compétences. Les systèmes fondés sur des mots-clés ou des filtres statiques ne parviennent pas à détecter les compétences implicites, les équivalences sémantiques et les trajectoires de montée en compétences.

Par ailleurs, si des référentiels structurants existent notamment la Nomenclature Camerounaise des Métiers (NCM 2013), le référentiel européen ESCO et la Nomenclature Camerounaise des Formations (NCF), leur intégration opérationnelle dans les plateformes et outils d'appariement reste encore limitée. Ils sont peu exploités pour aligner les intitulés locaux de métiers sur des standards internationaux, harmoniser les compétences attendues, qualifier les niveaux et domaines de formation, et rendre les correspondances profil-offre plus transparentes et comparables. Il en résulte un décalage entre la richesse potentielle de ces référentiels et leur usage effectif dans les systèmes d'information dédiés à l'emploi et à l'orientation.

Dans ce contexte, la question centrale n'est plus seulement d'apparier un profil à une offre, mais de concevoir un système capable (i) d'exploiter conjointement les référentiels NCM, ESCO et NCF pour structurer les métiers, les compétences et les formations, (ii) de mesurer finement le *skill gap* entre un candidat et un poste cible, et (iii) de proposer un parcours de montée en compétences compatible avec le niveau, le domaine de formation et les trajectoires de qualification du candidat. La question principale de recherche peut ainsi être formulée comme suit :

Comment concevoir un système de recommandation hybride, fondé sur les LLMs, les graphes de connaissances, les embeddings vectoriels et le filtrage collaboratif, capable de mesurer le skill gap entre un profil et une offre d'emploi, puis de recommander un parcours de montée en compétences cohérent avec la nomenclature des formations, le niveau du candidat et le métier visé pour les membres de la communauté KmerAi ?

Cette question générale se décline en plusieurs questions de recherche sous-jacentes :

- **Q1** : Comment identifier et formaliser, dans le contexte camerounais, les compétences, métiers, formations et relations pertinentes pour un matching emploi compétences fiable et opérationnel à partir des référentiels NCM, ESCO et NCF ?

- Q2 : Dans quelle mesure l'intégration d'un graphe de connaissances améliore-t-elle la qualité et la précision du matching par rapport aux approches purement vectorielles ?

Objectifs de l'étude

L'objectif général de cette étude est de concevoir et d'évaluer un système de recommandation hybride emploi-compétences capable d'aider à la décision dans le recrutement et la montée en compétence. Le système vise à rapprocher les profils de candidats et les offres d'emploi en combinant la recherche sémantique, les référentiels métiers et formations, le graphe de connaissances et une logique explicable de *skill gap*. Il ne s'agit donc pas seulement de retrouver les offres textuellement proches d'un profil, mais de produire un verdict opérationnel : identifier les offres immédiatement accessibles, distinguer celles qui nécessitent une montée en compétence, signaler les profils à conserver en vivier et expliciter les compétences à développer.

Plus spécifiquement, il sera question de :

- normaliser et aligner les offres d'emploi et profils candidats avec les référentiels ESCO, MEPC et NCF afin de produire des variables structurées intitulé de poste, compétences, niveau NCF, secteur, localisation exploitables par le système de matching ;
- adapter par *fine-tuning* contrastif un modèle *SentenceTransformer* au domaine emploi-compétences à partir de paires offre-profil, puis indexer les représentations produites dans une base vectorielle pgvector pour le retrieval sémantique ;
- construire un graphe de connaissances sous Neo4j modélisant les relations entre candidats, offres, compétences, métiers, secteurs et niveaux NCF, et concevoir un score hybride combinant similarité sémantique, couverture de compétences, compatibilité NCF et alignement sectoriel ;
- orchestrer, via un workflow *LangGraph* multi-étapes, l'ensemble du pipeline de recommandation retrieval sémantique, enrichissement graphe, analyse de *skill gap*, scoring hybride, critique et replanification afin de produire des verdicts interprétables (profil prêt, montée en compétence, vivier, hors cible) accompagnés d'une trajectoire de formation, et d'évaluer la qualité du retrieval par les métriques Recall@K, NDCG@K, MRR@K et MAP@K.

Intérêt de l'étude

L'intérêt d'une telle approche est renforcé par les limites des méthodes classiques de recrutement, souvent fondées sur des critères statiques ou sur des mots-clés, qui ne captent pas toujours la richesse des parcours et des compétences. En combinant différentes logiques de re-

commandation, un système hybride peut mieux prendre en compte les dimensions explicites et implicites des profils candidats, tout en améliorant la qualité des correspondances proposées. Cela rejoint la mission de KmerAI, qui vise à favoriser l'appropriation locale de l'IA pour résoudre des problèmes concrets du développement camerounais.

Hypothèses de recherche

Afin de répondre à la problématique, cette étude repose sur les hypothèses suivantes.

- **H1** : L'intégration conjointe de la NCM, d'ESCO et de la NCF dans un référentiel unifié métiers-compétences-formations améliore la structuration et les performances d'appariement profil-offre, par rapport à une approche sans ces référentiels ;
- **H2** : La modélisation du marché de l'emploi sous forme de graphe de connaissances (Neo4j), incluant explicitement les niveaux et domaines de formation de la NCF, améliore la qualité du matching par rapport à une approche purement vectorielle ;
- **H3** : Le *fine-tuning* d'un modèle de représentation sémantique sur le domaine emploi-compétences améliore la qualité des embeddings et la similarité sémantique entre profils et offres, par rapport au même modèle non adapté ;
- **H4** : La combinaison d'un RAG vectoriel, d'un GraphRAG et d'un score de recommandation hybride améliore la pertinence et l'explicabilité des recommandations de *skill gap* et de parcours de montée en compétences, notamment lorsque ces parcours sont contraints par la nomenclature des formations.

Méthodologie de recherche

La démarche adoptée s'inscrit dans un cadre d'ingénierie des données combinant structuration des connaissances et modélisation hybride. Elle repose sur la collecte et la normalisation des données issues d'offres d'emploi sur les différentes sites web et de profils, alignées sur la NCM 2013 afin d'assurer la cohérence sémantique. Un graphe de connaissances est ensuite construit sous Neo4j en intégrant les référentiels ESCO et NCM, à partir d'un processus d'extraction automatique de triplets à partir des données textuelles. Parallèlement, les descriptions sont vectorisées et indexées dans un espace sémantique permettant la mesure de similarité. Le moteur de recommandation combine ces différentes sources d'information à travers un score hybride intégrant similarité structurelle, similarité sémantique et signal collaboratif. Le système permet ainsi d'identifier les écarts de compétences entre profils et offres, et de générer des recommandations sous forme de trajectoires d'apprentissage. L'évaluation repose sur des métriques standard telles que la précision@K, le rappel et le NDCG, ainsi que sur l'analyse de

la cohérence des recommandations générées.

Plan du travail

Le mémoire est structuré en trois parties. La première présente le cadre théorique relatif aux systèmes de recommandation, aux modèles de langage et aux graphes de connaissances. La deuxième détaille l'approche méthodologique, incluant la modélisation du graphe et la conception du système hybride. La troisième analyse les résultats obtenus et discute les performances du modèle ainsi que ses implications dans le contexte du marché du travail camerounais.

Première partie

Cadre théorique et conceptuel

Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation

Ce chapitre situe le mémoire dans la littérature sur l'appariement emploi-compétences, les systèmes de recommandation, l'intelligence artificielle générative, les graphes de connaissances et la génération augmentée par récupération. Il ne vise pas à reprendre l'ensemble des raisonnements des ouvrages ou articles consultés. Il sélectionne les résultats qui éclairent directement le thème du stage : concevoir un système capable de recommander des offres, d'identifier des écarts de compétences et de produire des explications fondées sur des données structurées et non structurées.

La revue est guidée par une hypothèse centrale : le matching emploi-compétences n'est pas un simple problème de similarité textuelle. Il combine des frictions du marché du travail, une forte variabilité du vocabulaire, des contraintes de qualification, des relations métier-compétence-formation et un besoin d'explicabilité. Cette nature composite explique pourquoi la littérature pertinente relève à la fois de l'économie du travail, de la recommandation hybride, de la recherche sémantique, des graphes de connaissances et de la génération contrôlée.

1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG

Avant d'entrer dans la revue de littérature, il est nécessaire de stabiliser les concepts utilisés dans la suite du mémoire. Cette clarification évite une confusion fréquente : employer indifféremment IA générative, LLM, RAG, GraphRAG, moteur de recommandation et chatbot. Dans ce travail, ces termes désignent des fonctions complémentaires, mais non interchangeables.

Matching emploi-compétences.

Le matching emploi-compétences désigne le processus qui consiste à rapprocher un profil candidat d'une offre, d'un métier ou d'une trajectoire de formation à partir de caractéristiques observables : compétences, niveau de formation, expérience, secteur, localisation et objectif professionnel. Il est traité ici comme un problème d'appariement économique, mais aussi comme un problème de représentation des connaissances.

Système de recommandation.

Un système de recommandation est un dispositif algorithmique qui classe des éléments selon leur pertinence probable pour un utilisateur ou un cas d'usage [1]. Dans ce mémoire, les éléments recommandés peuvent être des offres d'emploi, des compétences, des métiers ou des pistes de formation. La pertinence ne renvoie donc pas seulement à une préférence ; elle renvoie aussi à une compatibilité professionnelle explicable.

Skill gap.

Le *skill gap*, ou écart de compétences, correspond à la différence entre les compétences déjà détenues par un candidat et celles attendues pour une offre ou un métier cible. Il permet de dépasser une logique binaire compatible/non compatible : un candidat peut être proche d'un poste tout en nécessitant une montée en compétences.

Embedding et recherche sémantique.

Un embedding est une représentation vectorielle d'un texte, d'un profil, d'une offre ou d'une compétence. La recherche sémantique utilise cette représentation pour retrouver des objets proches en sens, même lorsque les formulations diffèrent. Elle répond à un problème récurrent des données d'emploi : des expressions comme « data analyst », « analyste de données » et « reporting statistique » peuvent désigner des zones de compétences voisines.

Base vectorielle pgvector.

pgvector désigne l'extension PostgreSQL utilisée pour stocker et rechercher des embeddings [2]. Dans ce mémoire, elle sert de couche de recherche rapide pour retrouver les offres, métiers, compétences ou documents proches d'une requête. Son rôle est de présélectionner des candidats à la recommandation. Elle ne suffit cependant pas à décider seule, car un bon score vectoriel ne garantit pas que le niveau NCF, l'expérience ou les compétences obligatoires soient compatibles.

Graphe de connaissances Neo4j.

Un graphe de connaissances représente des entités et leurs relations : candidat, offre, compétence, métier, secteur, niveau, formation et référentiel [3]. Dans ce mémoire, Neo4j sert à représenter les relations explicites entre ces entités. Il permet de répondre à des questions que la seule similarité vectorielle ne traite pas correctement : quelles compétences une offre exige-t-elle ? quelles compétences le candidat possède-t-il ? quel niveau de formation est demandé ? quels chemins relient un métier à une formation ?

IA générative, LLM et prompting.

L'intelligence artificielle générative désigne les modèles capables de produire du contenu, notamment du texte. Un LLM est un grand modèle de langage entraîné sur de grands corpus textuels et capable de générer, reformuler ou synthétiser des réponses. Dans ce mémoire, le LLM n'est pas le moteur de vérité du système. Il intervient comme générateur contrôlé : il reçoit un contexte construit à partir de pgvector, Neo4j et des documents, puis rédige une réponse en français. Le prompting désigne l'ensemble des consignes données au modèle pour encadrer son comportement, limiter les inventions et structurer la réponse.

RAG, GraphRAG et Agentic RAG.

La RAG combine récupération d'information et génération : le système recherche d'abord des éléments pertinents, puis les transmet au LLM pour produire une réponse [4]. Le GraphRAG étend cette logique en ajoutant des faits et chemins issus d'un graphe de connaissances. L'Agentic RAG ajoute une couche d'orchestration : l'agent analyse l'intention, choisit les outils, récupère les données, construit le contexte, génère la réponse et peut demander une révision si le contexte est insuffisant. Dans ce mémoire, cette orchestration est assurée par LangGraph.

Critic et traçabilité.

Le critic est un mécanisme de contrôle qui examine si la réponse générée semble suffisamment couverte par le contexte récupéré. Il ne remplace pas une validation humaine, mais il réduit le risque d'une réponse fluide et insuffisamment fondée. La traçabilité désigne la capacité à afficher les outils mobilisés, les sources consultées et le chemin suivi par le workflow. Elle est indispensable dans un système d'aide à la décision, car une recommandation professionnelle doit être vérifiable.

1.2 Fondements économiques et problématique d'appariement

1.2.1 Marché du travail, recherche d'emploi et frictions

Le problème traité dans ce mémoire relève d'abord de l'économie du travail. Les modèles de recherche et d'appariement montrent que le chômage peut coexister avec des postes vacants lorsque la rencontre entre travailleurs et entreprises est coûteuse, lente ou imparfaitement informée [5, 6]. Cette lecture est importante : un système de recommandation emploi-compétences n'est pas seulement un outil informatique ; c'est un dispositif d'intermédiation qui cherche à réduire les coûts de recherche et à améliorer la qualité des mises en relation.

La littérature sur l'asymétrie d'information renforce ce diagnostic. Akerlof montre que l'in-

certitude sur la qualité peut dégrader les échanges lorsque les acteurs ne disposent pas de signaux fiables [7]. Sur le marché de l'emploi, cette asymétrie apparaît des deux côtés : le recruteur ne voit pas toujours les compétences réelles du candidat, et le candidat ne connaît pas toujours les exigences effectives du poste. La théorie du signalement de Spence explique alors le rôle des diplômes, certifications et expériences comme signaux observables [8]. Mais ces signaux restent incomplets lorsque les métiers évoluent rapidement, notamment dans les domaines numériques où les compétences techniques sont souvent décrites par des formulations instables.

1.2.2 Capital humain, tâches et inadéquation des compétences

La théorie du capital humain rappelle que la formation et l'expérience sont des investissements productifs [9]. Toutefois, la possession d'un capital humain ne garantit pas l'appariement. Encore faut-il que les compétences acquises soient visibles, comparables et alignées sur les compétences demandées. Autor montre que le progrès technique transforme surtout les tâches composant les métiers, et non uniquement les intitulés d'emploi [10]. Pour un système de recommandation, cela impose de descendre au niveau des compétences et des tâches plutôt que de s'arrêter aux titres de poste.

L'apport de cette littérature est de montrer que l'inadéquation emploi-compétences relève autant de la circulation imparfaite de l'information que d'un manque de formation. Sa limite, pour notre objet, est qu'elle ne fournit pas directement les outils de traitement des données nécessaires pour extraire, normaliser, comparer et expliquer les compétences. La section suivante examine donc les systèmes de recommandation comme réponse algorithmique à ce problème d'appariement.

1.3 Systèmes de recommandation et person-job fit

1.3.1 Approches par contenu, collaboratives et hybrides

Les systèmes de recommandation cherchent à classer des items selon leur pertinence probable pour un utilisateur. La littérature distingue classiquement les approches fondées sur le contenu, le filtrage collaboratif et les approches hybrides [11, 1]. Dans le contexte de ce mémoire, l'utilisateur peut être un candidat, un conseiller emploi ou une plateforme ; les items peuvent être des offres, compétences, métiers ou formations.

Les approches fondées sur le contenu utilisent les caractéristiques explicites des items et des profils. Elles sont adaptées au matching emploi-compétences parce que les offres et les

candidats possèdent des descriptions textuelles, des secteurs, des niveaux et des compétences. Pazzani et Billsus montrent que ces systèmes exploitent les attributs des contenus pour produire des recommandations personnalisées [12]. Leur avantage est l'interprétabilité ; leur limite est la dépendance au vocabulaire observé.

Le filtrage collaboratif exploite les interactions passées entre utilisateurs et items. Les approches de factorisation matricielle ont fortement structuré ce domaine en apprenant des facteurs latents à partir des matrices d'interactions [13]. Les modèles neuronaux ont ensuite généralisé cette logique, par exemple avec le Neural Collaborative Filtering [14]. Ces méthodes sont puissantes lorsque l'on dispose d'un historique dense : clics, candidatures, recrutements, évaluations ou parcours de formation. Dans le cas étudié ici, cette hypothèse est fragile, car les données d'interaction restent limitées et de nombreux profils se trouvent en situation de démarrage à froid.

Les approches hybrides combinent plusieurs sources de signal pour compenser les limites propres à chaque famille. Burke montre que l'hybridation peut améliorer la robustesse en associant contenu, collaboration, connaissances et règles métier [15]. Cette conclusion prépare directement le choix d'un système hybride : les signaux textuels, relationnels et métier doivent être séparés puis combinés, car chacun ne capture qu'une partie de la compatibilité.

1.3.2 Spécificité du person-job fit

La recommandation d'emploi se distingue de la recommandation de films ou de produits. Une offre n'est pas seulement un item susceptible de plaire ; elle impose des contraintes de qualification, d'expérience, de localisation, de disponibilité et de compétences. La littérature récente sur le *person-job fit* insiste sur cette double sélection : le candidat doit être intéressé par l'offre, mais l'employeur doit aussi pouvoir considérer le profil comme compatible [16]. La recommandation devient donc un problème bilatéral.

Cette spécificité change l'interprétation des scores. Dans le commerce électronique, une recommandation peut viser la probabilité de clic ou d'achat. Dans l'emploi, une recommandation doit distinguer au moins trois situations : le candidat est immédiatement compatible ; le candidat est proche mais nécessite une montée en compétences ; le candidat est trop éloigné du poste cible. La notion de *skill gap* devient alors centrale.

Les travaux récents mobilisant les graphes et les LLM dans les ressources humaines vont dans ce sens. HRGraph propose par exemple d'exploiter des graphes de connaissances enrichis par LLM pour la recommandation d'emploi [17]. D'autres travaux s'intéressent à la prédiction des compétences et à la construction de graphes de talents pour les ressources humaines

[18]. Ces contributions confirment l'intérêt des graphes pour représenter les relations entre compétences, métiers et profils, mais elles ne résolvent pas automatiquement la question de l'ancrage local : nomenclatures camerounaises, données réelles de plateforme, contraintes de formation et explication opérationnelle.

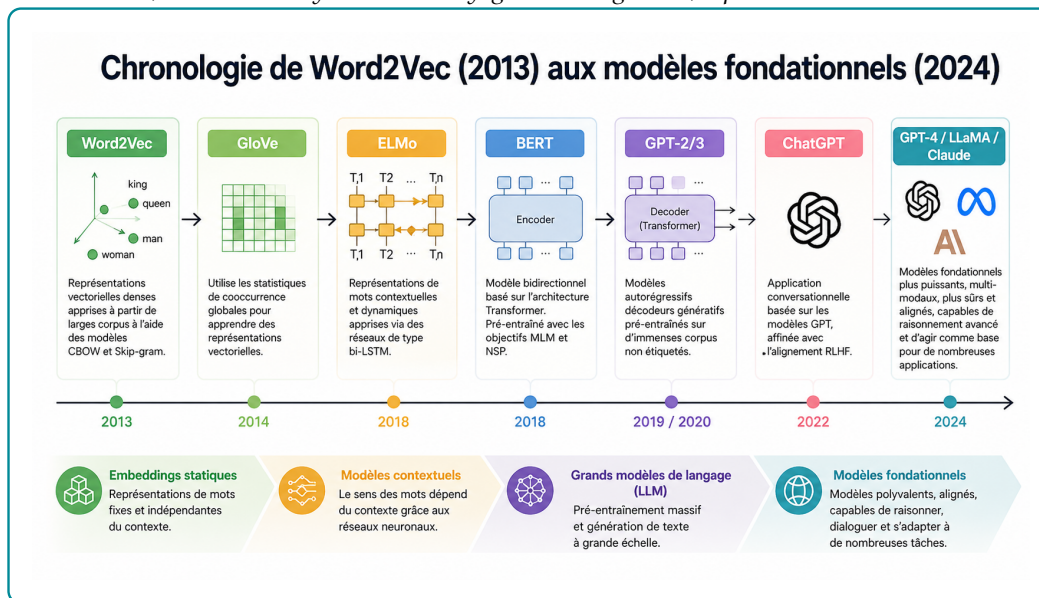
1.4 LLMs, embeddings et graphes de connaissances

1.4.1 Grands modèles de langage et génération contrôlée

Les grands modèles de langage ont transformé le traitement automatique du langage naturel en permettant des tâches de génération, synthèse, reformulation et raisonnement assisté. Leur rupture technique vient principalement de l'architecture Transformer proposée par Vaswani et al., fondée sur le mécanisme d'attention [19]. Avant cette rupture, les représentations textuelles étaient passées des sacs de mots aux embeddings denses comme Word2Vec, qui apprennent des vecteurs capturant des régularités sémantiques et syntaxiques [20]. Les mécanismes d'attention introduits en traduction neuronale ont ensuite permis aux modèles de pondérer dynamiquement les parties pertinentes d'une séquence [21].

Figure 1.1 – Évolution simplifiée des modèles de langage et des représentations textuelles

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



Deux grandes familles de modèles sont utiles pour ce mémoire. La première regroupe les modèles de représentation, comme BERT et Sentence-BERT. BERT apprend des représentations contextuelles bidirectionnelles adaptées à la compréhension du texte [22]. Sentence-BERT adapte cette logique pour produire des embeddings de phrases comparables efficacement par

similarité cosinus [23]. La seconde famille regroupe les modèles génératifs, comme GPT, T5 ou les modèles instruction-tuned. GPT-3 a montré qu'un modèle de grande taille pouvait effectuer des tâches variées à partir de quelques exemples dans le prompt [24]. T5 a proposé une formulation unifiée des tâches NLP comme transformation texte-vers-texte [25]. L'apprentissage par retour humain a ensuite amélioré la capacité des modèles à suivre des instructions [26].

L'acquis de cette littérature est considérable : les LLM savent reformuler, synthétiser et produire des explications lisibles. Sa limite, pour notre problématique, tient au statut de la vérité : un modèle génératif peut produire une réponse plausible sans être fondée sur les données locales. Dans un système d'emploi, cette limite impose une génération contrôlée par des sources récupérées.

1.4.2 Embeddings, recherche sémantique et bases vectorielles

Après la clarification conceptuelle, l'enjeu est de savoir dans quelles conditions les représentations vectorielles sont fiables pour la recherche d'information. Les benchmarks montrent que les modèles d'embeddings doivent être évalués sur des tâches variées, car un bon modèle généraliste peut être insuffisant dans un domaine spécialisé [27, 28]. Cette observation justifie l'adaptation d'un modèle SentenceTransformer au vocabulaire emploi-compétences.

La recherche sémantique résout une partie du problème de vocabulaire, mais elle reste une approximation. Deux textes proches peuvent cacher une incompatibilité : niveau insuffisant, compétence obligatoire absente, localisation incompatible ou métier trop éloigné. La similarité vectorielle doit donc être traitée comme un signal de présélection, non comme une décision finale.

La littérature sur la recherche approximative de plus proches voisins montre par ailleurs que le passage à l'échelle exige des structures d'indexation efficaces. HNSW fournit un cadre robuste pour la recherche vectorielle approximative [29], tandis que FAISS a popularisé la recherche vectorielle à grande échelle [30]. Dans ce mémoire, l'utilisation de PostgreSQL avec pgvector répond à une logique pragmatique : conserver les embeddings dans une base relationnelle exploitable, interrogeable et intégrable au reste du pipeline [2].

1.4.3 Graphes de connaissances et recommandation relationnelle

Hogan et al. montrent que les graphes de connaissances servent à structurer des informations hétérogènes, à rendre explicites des relations sémantiques et à soutenir des tâches de recherche ou de raisonnement [3]. Dans le domaine emploi-compétences, cette représenta-

tion complète la recherche vectorielle : elle rend visibles les relations entre candidats, offres, compétences, métiers, formations et niveaux.

La recommandation par graphe a également connu des avancées importantes. Les graph neural networks ont fourni des outils pour propager de l'information dans des graphes [31, 32]. LightGCN a montré que, dans la recommandation collaborative, une architecture simplifiée de propagation sur graphe pouvait être très efficace [33]. Ces travaux établissent l'intérêt de la structure relationnelle pour la recommandation.

Cependant, le présent mémoire ne cherche pas principalement à entraîner un GNN. Le choix méthodologique est différent : utiliser un graphe de connaissances comme support d'explication, de contraintes et de calcul du skill gap. Ce choix est plus adapté au contexte disponible. Les données d'interaction ne sont pas assez riches pour justifier un modèle collaboratif profond, tandis que les relations métier-compétence-formation sont directement interprétables.

1.5 RAG, GraphRAG et orchestration agentique

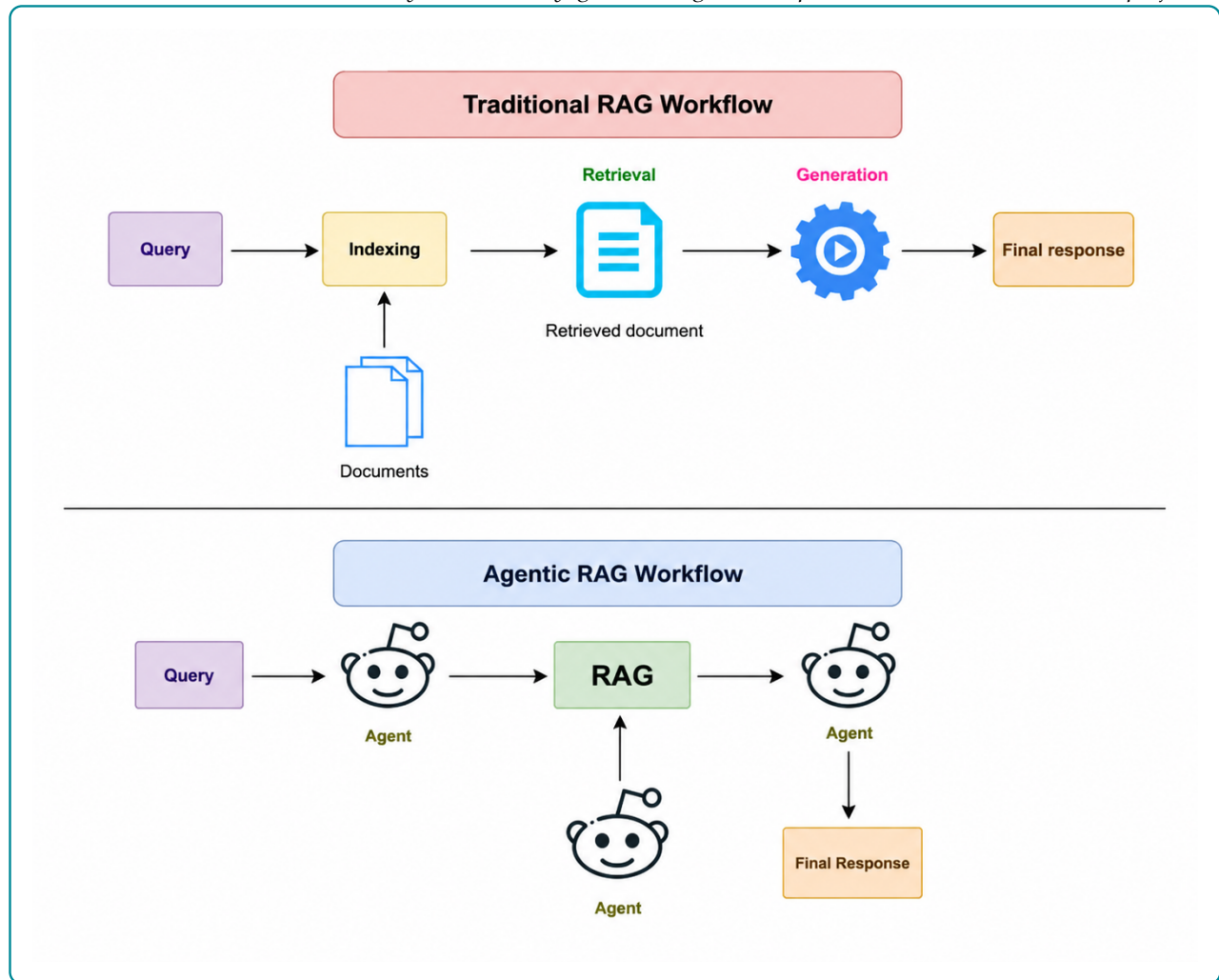
1.5.1 De la RAG au GraphRAG

La génération augmentée par récupération, ou RAG, combine une étape de recherche d'information et une étape de génération. Lewis et al. montrent que l'accès à une mémoire externe améliore les tâches intensives en connaissances, car le modèle ne dépend plus uniquement de ses paramètres internes [4]. Les synthèses récentes sur la RAG insistent sur les mêmes enjeux : qualité de la récupération, actualisation des connaissances, évaluation de la factualité et articulation entre retriever et générateur [34].

Dans une RAG classique, le système récupère des documents proches d'une requête, puis les transmet au LLM. Cette architecture est utile, mais elle reste centrée sur des passages textuels. Pour le matching emploi-compétences, le contexte doit aussi intégrer des relations : compétences possédées, compétences requises, niveau de formation, métier cible, secteur et formation possible. Le GraphRAG étend alors la récupération vers des informations structurées par graphe.

Figure 1.2 – Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.



La figure 1.2 illustre cette évolution. Une RAG simple suit une chaîne relativement fixe : requête, récupération, génération. Une architecture GraphRAG ajoute une couche relationnelle. Une architecture agentic introduit une capacité de planification et de contrôle : le système peut choisir un outil, observer le résultat, élargir le contexte et réviser la réponse.

1.5.2 Agentic RAG, critic et traçabilité

Les travaux ReAct montrent l'intérêt d'alterner raisonnement et action outillée dans les modèles de langage [35]. Self-RAG va plus loin en introduisant une logique d'auto-réflexion pour décider quand récupérer de l'information et comment critiquer la réponse [36]. Pour ce mémoire, ces travaux ne doivent pas être interprétés comme une autorisation à laisser un agent décider librement. Ils justifient plutôt une orchestration contrôlée : classifier l'intention, choisir les sources, récupérer les preuves, construire le contexte, générer, puis vérifier si la réponse est

suffisamment fondée.

Le manque de la littérature est ici opérationnel. Beaucoup de travaux décrivent des architectures générales, mais la fiabilité dépend de détails concrets : qualité des chunks, schéma du graphe, robustesse des requêtes, traçabilité des sources, gestion des cas sans résultat et critères de révision. C'est pourquoi l'architecture retenue ajoute un critic et des traces d'exécution. Le LLM produit la réponse finale, mais les données récupérées et les outils appelés doivent rester observables.

1.5.3 Évaluer la performance et la factualité

L'évaluation d'un système emploi-compétences ne peut pas se limiter à la fluidité de la réponse générée. Elle doit d'abord mesurer la qualité de la récupération d'information : le système retrouve-t-il les offres, compétences ou documents réellement pertinents ? La littérature en recherche d'information utilise pour cela des métriques de classement comme Precision@k, Recall@k, MRR et NDCG [27, 28]. Ces métriques supposent l'existence d'un ensemble de résultats pertinents, noté $Rel(q)$, pour une requête q , et d'une liste classée de résultats retournés par le système.

La **Precision@k** mesure la proportion de résultats pertinents parmi les k premiers résultats retournés. Si $R_k(q)$ désigne l'ensemble des k premiers éléments proposés pour la requête q , alors :

$$Precision@k(q) = \frac{|R_k(q) \cap Rel(q)|}{k}. \quad (1.1)$$

Cette métrique répond à la question suivante : parmi les premières recommandations affichées, combien sont réellement utiles ? Dans un système emploi-compétences, une Precision@5 élevée signifie que les cinq premières offres proposées contiennent peu de résultats non pertinents. Sa limite est qu'elle ne mesure pas si le système a retrouvé tous les bons résultats disponibles.

Le **Recall@k** mesure au contraire la proportion des éléments pertinents retrouvés dans les k premiers résultats :

$$Recall@k(q) = \frac{|R_k(q) \cap Rel(q)|}{|Rel(q)|}. \quad (1.2)$$

Cette métrique répond à la question : parmi toutes les offres ou compétences pertinentes, quelle part le système a-t-il réussi à faire remonter ? Elle est importante pour ne pas éliminer trop tôt des offres utiles. En recommandation emploi, un faible Recall@k peut signifier que le

moteur rate des opportunités pertinentes pour le candidat.

Le **MRR** (*Mean Reciprocal Rank*) évalue la position du premier résultat pertinent. Pour une requête q , si $rank_q$ est le rang du premier résultat pertinent, alors le score associé est :

$$RR(q) = \frac{1}{rank_q}. \quad (1.3)$$

Sur un ensemble de N requêtes, le MRR est :

$$MRR = \frac{1}{N} \sum_{q=1}^N \frac{1}{rank_q}. \quad (1.4)$$

Cette métrique est utile lorsque l'on veut savoir si le système place rapidement au moins une bonne réponse. Dans une interface de recommandation, cela correspond à une logique pratique : l'utilisateur regarde souvent les premiers résultats avant de parcourir toute la liste.

Le **NDCG@k** (*Normalized Discounted Cumulative Gain*) tient compte non seulement de la présence des résultats pertinents, mais aussi de leur ordre et éventuellement de leur degré de pertinence. Si rel_i désigne le niveau de pertinence du résultat placé au rang i , alors :

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}. \quad (1.5)$$

Le score est ensuite normalisé par le meilleur classement possible, noté $IDCG@k$:

$$NDCG@k = \frac{DCG@k}{IDCG@k}. \quad (1.6)$$

Cette métrique est particulièrement adaptée lorsque tous les résultats pertinents ne se valent pas. Par exemple, une offre parfaitement alignée avec les compétences du candidat devrait être mieux classée qu'une offre seulement partiellement compatible.

Dans une architecture RAG, l'évaluation ne s'arrête pas au classement des résultats. Elle doit aussi mesurer la qualité de la réponse générée à partir du contexte récupéré. RAGAS distingue notamment la fidélité au contexte, la pertinence de la réponse, la qualité de la récupération et l'usage des sources [37]. La **fidélité** vérifie si les affirmations de la réponse sont supportées par les éléments récupérés. La **pertinence de réponse** vérifie si la réponse traite effectivement la question posée. La **couverture des sources** vérifie si le contexte contient suffisamment d'informations pour répondre. Enfin, la **traçabilité** vérifie si les preuves ou citations sont identifiables.

Ces critères sont moins simples à formaliser que Precision@k ou Recall@k, car ils portent

sur du langage naturel. Des approches comme G-Eval utilisent des LLM comme juges pour approximer l'évaluation humaine [38]. Cette pratique doit rester prudente : un juge automatique peut aider à comparer des variantes de système, mais il ne remplace pas une validation métier. Dans ce mémoire, l'évaluation doit donc combiner des métriques de retrieval pour pgvector et Neo4j, des critères de factualité pour la réponse générée, et une interprétation professionnelle de l'utilité des recommandations.

Source de données et Approche méthodologique

2.1 Cadre méthodologique et sources de données

2.1.1 Démarche d'ingénierie appliquée retenue

La présente étude adopte une démarche d'ingénierie appliquée orientée vers la conception, l'intégration et l'évaluation d'un artefact numérique d'aide à la décision. L'objet étudié n'est donc pas uniquement un modèle statistique pris isolément, mais un système complet de recommandation emploi-compétences combinant données structurées, représentations vectorielles, graphe de connaissances, règles métier et RAG. Ce positionnement est cohérent avec la problématique du mémoire : améliorer l'appariement entre candidats, offres d'emploi, compétences, métiers et trajectoires de formation dans un contexte où l'information est hétérogène, incomplète et dispersée.

La logique méthodologique retenue repose sur quatre principes. Le premier est la *traçabilité* : chaque recommandation doit pouvoir être rattachée à des données observables, à un référentiel ou à une relation explicite du graphe. Le deuxième est l'*hybridation* : aucune source d'information ne suffit seule à résoudre le problème. La similarité sémantique capte les proximités de langage, le graphe encode les relations métier, les règles contrôlent les contraintes de niveau et le LLM sert à formuler une réponse intelligible. Le troisième principe est l'*explicabilité opérationnelle* : le système ne doit pas seulement retourner une liste d'offres, mais justifier le rapprochement, signaler les compétences communes, identifier les écarts et proposer une trajectoire de montée en compétences. Le quatrième principe est le *contrôle génératif* : les réponses produites par le LLM doivent être contraintes par le contexte récupéré afin de réduire le risque d'hallucination, conformément à la logique du RAG [4, 37].

Sur le plan de l'orchestration, cette étude retient explicitement *LangGraph* comme cadre méthodologique de pilotage du système agentique [39]. Ce choix signifie que le raisonnement applicatif n'est pas laissé à une boucle conversationnelle informelle. *LangGraph* est donc utilisé pour organiser le routage entre les briques du système, conserver les traces des outils mobilisés, permettre une révision lorsque le contexte est insuffisant et rendre le workflow

auditable.

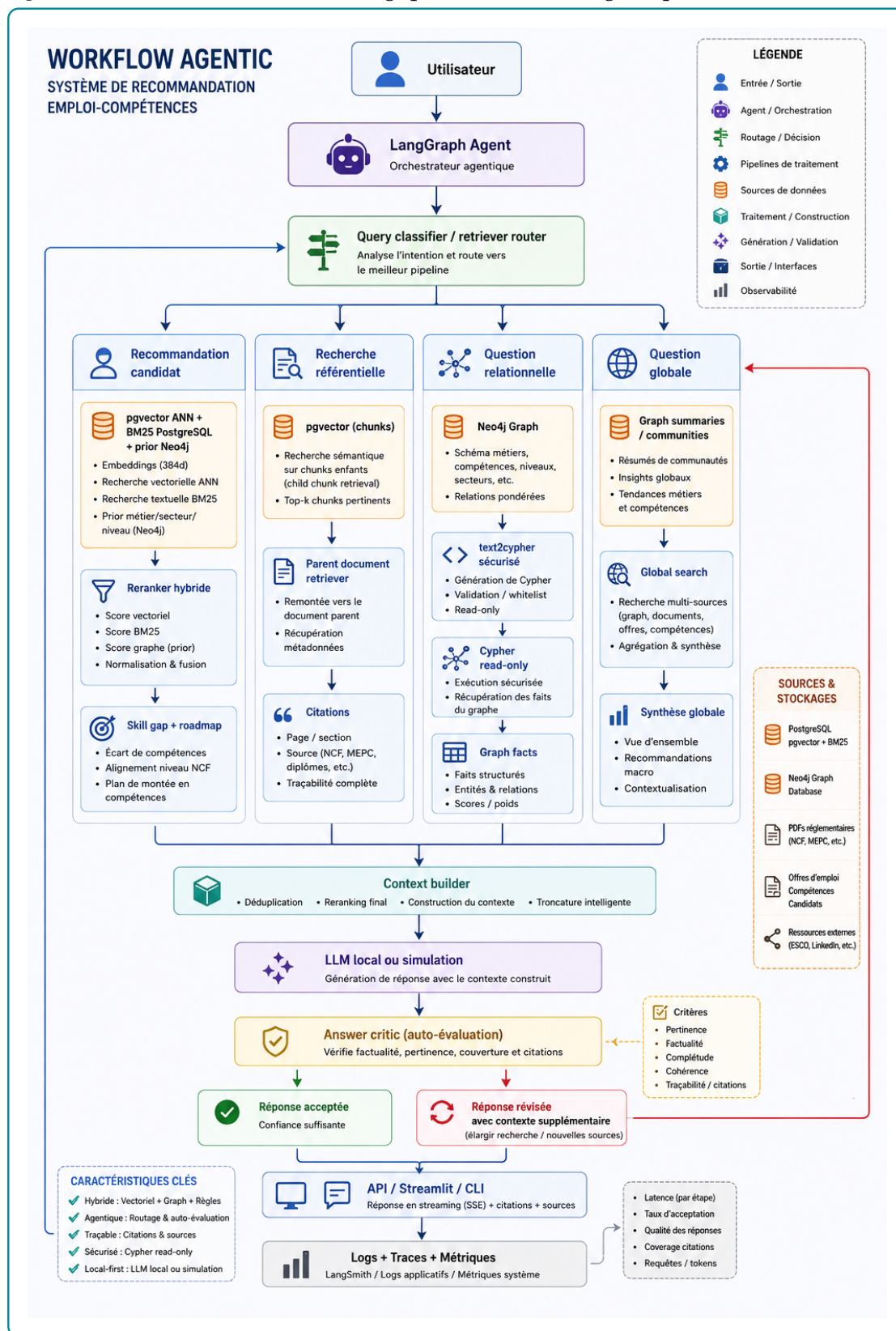
La figure 2.1 présente l'architecture générale retenue pour la solution. Elle doit être lue comme une carte méthodologique du système plutôt que comme un simple schéma technique. Le point d'entrée est la requête de l'utilisateur. Cette requête n'est pas transmise directement au modèle génératif ; elle est d'abord interprétée par une couche d'orchestration qui détermine le type de besoin exprimé. Cette étape est centrale, car une demande de recommandation candidat-offre, une question sur un référentiel, une interrogation relationnelle sur les compétences ou une question globale ne mobilisent pas les mêmes sources ni les mêmes traitements.

Le workflow distingue ainsi plusieurs familles de traitements. La recommandation candidat s'appuie sur une combinaison de recherche vectorielle, de recherche textuelle et de signaux issus du graphe afin de produire un classement explicable. La recherche référentielle privilégie les fragments documentaires et les métadonnées associées, car l'enjeu est alors de retrouver une information sourcée. Les questions relationnelles exploitent davantage Neo4j, où les liens entre métiers, compétences, niveaux, secteurs et formations permettent de répondre à des questions que la similarité textuelle seule ne suffit pas à résoudre. Les questions globales mobilisent une synthèse plus large, fondée sur des résumés, des communautés ou des agrégations issues de plusieurs sources.

L'intérêt méthodologique de cette architecture est donc l'hybridation contrôlée. PostgreSQL avec pgvector joue le rôle de mémoire sémantique pour retrouver des objets proches dans l'espace des embeddings. Neo4j joue le rôle de mémoire relationnelle pour vérifier, expliquer et pondérer les liens métier-compétence-formation. Les documents référentiels apportent une base de citation et de traçabilité. Le constructeur de contexte rassemble ensuite ces résultats dans un format exploitable par le LLM, en évitant de lui confier seul la décision de pertinence. La génération intervient donc en aval, après récupération et structuration du contexte.

Enfin, la partie inférieure du schéma rappelle que le système ne s'arrête pas à la production d'une réponse. Un module de critique évalue si la réponse est suffisamment pertinente, factuelle, complète et traçable. Si le contexte est insuffisant, la boucle peut relancer une recherche élargie avant de produire une réponse révisée. Ce choix méthodologique est important pour le mémoire : l'architecture ne cherche pas seulement à générer un texte convaincant, mais à produire une recommandation fondée sur des sources identifiables, des relations vérifiables et des critères métier explicites. Les sections suivantes détaillent chacune de ces briques : préparation des données, construction des embeddings, stockage vectoriel, graphe de connaissances,

Figure 2.1 – Architecture méthodologique du workflow agentique de recommandation



Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

moteur hybride, orchestration agentique et protocole d'évaluation.

Le tableau A.4, placé en annexe, synthétise la correspondance entre les objectifs spécifiques du mémoire et les choix méthodologiques retenus.

2.1.2 Sources de données et leur rôle dans le système

Le système mobilise deux catégories de données. La première correspond aux données opérationnelles du marché du travail : offres d'emploi et profils de candidats. Ces données décrivent respectivement la demande de travail et l'offre de travail disponible. La seconde catégorie correspond aux référentiels de structuration : ESCO, MEPC et NCF. Ces référentiels permettent d'éviter une approche purement textuelle et de rattacher les intitulés, compétences et niveaux de formation à des nomenclatures interprétables.

Les offres d'emploi apportent les informations relatives aux postes disponibles : intitulé, employeur, secteur, localisation, type de contrat, expérience attendue, niveau d'étude, compétences mentionnées et description détaillée. Les profils candidats décrivent les caractéristiques individuelles : diplôme, niveau d'étude, spécialité, expérience, compétences, mobilité et objectif professionnel. Ces deux sources sont naturellement bruitées : les intitulés ne sont pas normalisés, les compétences peuvent être formulées librement et certaines variables sont incomplètes. La méthodologie doit donc transformer ces données en objets comparables.

ESCO fournit une nomenclature internationale des métiers et compétences. Son intérêt est de disposer d'un vocabulaire structuré et de relations métier-compétence déjà établies. La MEPC permet d'ancrer le système dans la nomenclature camerounaise des métiers. La NCF fournit une structure nationale des niveaux et domaines de formation. L'intégration conjointe de ces sources permet de relier le langage des annonces, les profils des candidats et les classifications institutionnelles.

Table 2.1 – Sources de données mobilisées et utilisation méthodologique

Source	Nature	Utilisation dans la méthode
Offres d'emploi	Données opérationnelles collectées par KmerAI	Normalisation, extraction de compétences, génération de <code>text_to_embed</code> , indexation vectorielle, création des noeuds d'offres et calcul des scores de matching.
Profils candidats	Base locale de demandeurs	Normalisation du niveau, du diplôme, de la mobilité et du métier visé ; création des embeddings candidats ; analyse de compatibilité avec les offres.
ESCO	Référentiel métiers-compétences	Source de métiers, compétences, groupes ISCO et relations métier-compétence, utilisée pour structurer le graphe et enrichir les correspondances.
MEPC	Nomenclature canadienne des métiers	Ancrage national des métiers, structuration hiérarchique et alignement partiel avec les groupes internationaux.
NCF	Nomenclature canadienne des formations	Codification des niveaux et domaines de formation, utilisée pour la compatibilité candidat-offre et l'analyse de montée en compétences.
PDF réglementaires	Documents officiels fragmentés	Constitution de <code>DocChunk</code> pour la recherche référentielle et la citation des sources.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

2.1.3 Préparation et normalisation des données

La préparation des données constitue la première couche méthodologique du système. Son objectif est de produire des tables propres, stables et exploitables par les deux moteurs de recherche : le moteur vectoriel et le moteur graphe. Cette étape ne se limite pas au nettoyage cosmétique des textes. Elle détermine la qualité des relations construites, la pertinence des embeddings et la fiabilité des recommandations.

Pour les offres d'emploi, la normalisation consiste d'abord à réduire les doublons et les variations de forme. Une annonce peut apparaître plusieurs fois avec des différences mineures de titre, d'employeur ou de localisation. Le traitement doit donc stabiliser les identifiants et conserver les informations utiles sans multiplier artificiellement les observations. Les champs textuels sont ensuite harmonisés : suppression des caractères parasites, standardisation des espaces, séparation des champs multi-valeurs et conservation des informations de contexte. Les

compétences mentionnées sont transformées en listes exploitables, tandis que les localisations et secteurs sont séparés en valeurs principales et valeurs secondaires.

Pour les candidats, la normalisation porte sur le diplôme, le niveau d'étude, la spécialité, l'expérience, les compétences, le secteur visé et la mobilité géographique. Une attention particulière est portée au niveau de formation. Lorsque l'information disponible le permet, le niveau est rapproché de la NCF. Ce rapprochement est indispensable parce qu'une recommandation emploi-compétences ne peut pas se fonder uniquement sur la proximité textuelle : une offre peut être sémantiquement proche d'un profil tout en exigeant un niveau de qualification supérieur.

Chaque offre et chaque candidat reçoit enfin un champ `text_to_embed`. Ce champ synthétise les éléments utiles pour la recherche sémantique : intitulé, compétences, secteur, niveau, description et objectif. Le choix d'un texte synthétique répond à une contrainte pratique : les modèles d'embeddings travaillent sur des séquences textuelles, mais les données métier sont partiellement structurées. Le champ `text_to_embed` sert donc d'interface entre les variables métiers et l'espace vectoriel.

2.1.4 Alignement des référentiels métiers et formations

L'alignement des référentiels répond à un besoin central : rendre comparables des données issues de sources différentes. Les annonces utilisent des formulations libres, ESCO propose une taxonomie internationale, la MEPC suit une logique nationale de classification des métiers et la NCF structure les formations. La méthode adoptée est prudente : elle vise un alignement exploitable pour la recommandation, sans prétendre établir une équivalence parfaite entre toutes les nomenclatures.

La MEPC est structurée selon ses niveaux hiérarchiques : grands groupes, sous-groupes et groupes de base. La NCF est organisée autour des niveaux de formation, grands domaines, domaines spécialisés et domaines détaillés. Cette structuration permet de conserver l'information institutionnelle au lieu de la réduire à de simples libellés. Les tables obtenues alimentent ensuite le graphe Neo4j et la base vectorielle.

L'alignement avec ESCO s'appuie sur les codes et les proximités de libellés lorsque ces informations sont disponibles. Cette stratégie est méthodologiquement raisonnable, mais elle doit être interprétée avec prudence. Un même groupe de classification peut regrouper plusieurs réalités professionnelles. L'alignement sert donc à améliorer la couverture et l'interprétabilité, non à produire une vérité définitive sur l'équivalence entre métiers.

2.2 Modélisation sémantique et structuration des connaissances

2.2.1 Choix du modèle d’embeddings et fine-tuning

Le système utilise un modèle SentenceTransformer pour représenter les offres, candidats, métiers, compétences et fragments documentaires dans un espace vectoriel. Ce choix est adapté au problème, car le matching emploi-compétences repose largement sur la similarité sémantique entre textes courts ou semi-structurés : intitulés de postes, compétences, descriptions d’annonces, objectifs professionnels et libellés de référentiels. Les SentenceTransformers ont été conçus pour produire des embeddings directement comparables par similarité cosinus, ce qui les rend adaptés aux tâches de recherche sémantique [23].

Le fine-tuning retenu n’est pas un fine-tuning génératif. Il ne vise pas à apprendre au modèle à rédiger des réponses, mais à adapter l’espace vectoriel au vocabulaire local du domaine emploi-compétences. La logique est contrastive : rapprocher les paires pertinentes et éloigner les paires moins pertinentes. Par exemple, une offre de « data analyst » doit être rapprochée d’un profil mentionnant analyse de données, statistique, Python, SQL ou visualisation, même si les formulations exactes diffèrent. A l’inverse, un profil orienté développement web ne doit pas être considéré comme équivalent à une offre de comptabilité uniquement parce que certaines expressions génériques se ressemblent.

Les paires d’apprentissage sont construites à partir des données préparées. Elles peuvent mobiliser les relations offre-compétence, métier-compétence, candidat-métier et offre-candidat lorsque les signaux disponibles sont suffisamment fiables. Cette étape est critique : un fine-tuning apprend les régularités de ses exemples. Des paires mal construites peuvent dégrader l’espace vectoriel au lieu de l’améliorer. La méthodologie retient donc une construction contrôlée des exemples, puis une évaluation séparée du modèle de base et du modèle adapté.

2.2.2 Indexation vectorielle dans PostgreSQL et pgvector

Les embeddings produits sont stockés dans PostgreSQL avec l’extension pgvector. Ce choix permet de conserver dans un même environnement des représentations vectorielles et des métadonnées relationnelles. pgvector permet l’indexation et la recherche de voisins proches dans PostgreSQL, ce qui facilite l’intégration avec les autres tables du système [2]. Pour accélérer les recherches top-K, la méthode s’appuie sur une logique d’indexation approximative des voisins proches, cohérente avec les travaux sur HNSW [29].

La base vectorielle joue deux rôles. Le premier est le retrieval sémantique : retrouver les offres, métiers, compétences ou documents proches d’une requête. Le second est le préfiltrage

pour la recommandation : produire un ensemble de candidats plausibles avant d'appliquer des contraintes plus structurées. Ce préfiltrage est nécessaire, car le graphe de connaissances apporte de l'explicabilité, mais il n'est pas toujours le meilleur premier filtre lorsque la requête est formulée librement.

Chaque résultat vectoriel doit conserver ses métadonnées : identifiant, type d'entité, libellé, source, score de similarité et texte associé. Sans ces métadonnées, la génération finale serait incapable de citer ses sources et le système perdrait sa traçabilité.

2.2.3 Construction du graphe de connaissances sous Neo4j

Le graphe de connaissances représente la couche relationnelle du système. Son rôle est de modéliser explicitement les relations entre les candidats, les offres, les compétences, les métiers, les secteurs, les localisations et les référentiels. Cette représentation est justifiée par la nature même du problème : le matching emploi-compétences n'est pas seulement une question de proximité textuelle, mais une question de relations. Une offre requiert des compétences, un candidat possède des compétences, un métier est classé dans une nomenclature, un niveau de formation conditionne l'accès à certains postes, et une trajectoire de montée en compétences dépend des écarts identifiés.

La méthodologie retient des noeuds correspondant aux entités du système. Les relations décrivent les liens fonctionnels : une offre **REQUIERT** une compétence, un candidat **POSSEDE** une compétence, une offre est rattachée à un secteur ou à une localisation, un métier est classé dans un groupe, et un fragment documentaire provient d'un document référentiel.

Ce graphe sert à trois opérations méthodologiques. Premièrement, il vérifie les correspondances proposées par la recherche vectorielle. Deuxièmement, il explique les recommandations en identifiant les compétences communes et manquantes. Troisièmement, il permet des requêtes relationnelles que la recherche vectorielle gère mal, par exemple : quelles compétences sont associées à un métier donné ? quelles offres exigent une compétence précise ? quelles compétences manquent à un candidat pour viser une famille de métiers ?

2.2.4 Score hybride et logique de skill gap

Le score de recommandation combine plusieurs signaux complémentaires. La similarité sémantique mesure la proximité entre le texte du profil et celui de l'offre. La couverture de compétences mesure la proportion des compétences requises par l'offre qui sont déjà présentes chez le candidat. La compatibilité de niveau vérifie si le niveau de formation du candidat est cohérent avec le niveau attendu par l'offre. L'alignement sectoriel mesure la cohérence entre

le secteur ou le domaine du candidat et celui de l'offre.

La forme générale du score est la suivante :

$$S_{hyb}(c, o) = \alpha S_{sem}(c, o) + \beta S_{graph}(c, o) + \gamma C_{ncf}(c, o) + \delta A_{sect}(c, o),$$

où c désigne un candidat, o une offre, S_{sem} la similarité sémantique, S_{graph} la couverture de compétences issue du graphe, C_{ncf} la compatibilité de niveau de formation et A_{sect} l'alignement sectoriel. Cette formulation traduit un choix méthodologique important : le système ne délègue pas toute la décision au LLM. Le LLM intervient pour synthétiser et expliquer, mais le classement repose sur des signaux calculables et auditaibles.

Les pondérations ne doivent pas être fixées uniquement par intuition. La méthodologie retient donc une calibration statistique par optimisation bayésienne avec Optuna, en utilisant le *Tree-structured Parzen Estimator* comme stratégie de recherche [40]. Dans l'expérience logicielle retenue, le score calibré est volontairement ramené à deux signaux observables dans les sorties d'évaluation : le score sémantique issu de pgvector et le score Neo4j issu de l'enrichissement graphe. Le problème optimisé est donc :

$$S_{cal}(c, o) = w_{sem} S_{sem}(c, o) + w_{Neo4j} S_{Neo4j}(c, o), \quad w_{sem} + w_{Neo4j} = 1.$$

Le principe d'Optuna est le suivant. A chaque essai, l'algorithme propose une valeur candidate de w_{sem} , en déduit $w_{Neo4j} = 1 - w_{sem}$, recalcule le score des offres candidates, rerange les offres pour chaque candidat, puis évalue ce nouveau classement avec une fonction objectif. Les essais précédents alimentent le modèle probabiliste du *TPESampler* : les zones de poids ayant produit de bons scores sont explorées plus intensément que les zones peu prometteuses. La méthode ne cherche donc pas un coefficient « vrai » au sens économique ; elle cherche la pondération qui maximise empiriquement une métrique de ranking sur l'échantillon d'évaluation.

Trois fonctions objectif sont testées afin de ne pas confondre un bon classement global avec une bonne première recommandation : le NDCG@10 seul, la moyenne NDCG@10–Recall@10–Precision@10 et la moyenne MRR@10–Precision@10. Le NDCG@10 valorise l'ordre complet du top 10 ; le rappel et la précision évaluent la couverture binaire des offres pertinentes ; le MRR@10 met davantage de pression sur le rang de la première offre pertinente. Cette calibration est volontairement séparée de la génération LLM. Elle concerne seulement le moteur de classement. Elle inclut également un garde-fou : les poids proposés par Optuna sont com-

parés à des solutions de frontière, par exemple un score purement sémantique ou un score purement fondé sur Neo4j. La calibration est donc utilisée comme un test empirique des pondérations, non comme une justification automatique du caractère hybride du système. Si les labels de pertinence sont des proxys et non des jugements humains, les coefficients obtenus doivent être interprétés comme optimaux dans le protocole expérimental courant, pas comme des paramètres définitifs du marché du travail.

L'analyse de *skill gap* complète le score. Elle distingue les compétences acquises, les compétences requises et les compétences manquantes. Cette distinction permet de produire un verdict opérationnel : candidat immédiatement compatible, candidat compatible après montée en compétence, candidat à conserver en vivier ou profil hors cible. Ce verdict est plus utile qu'un simple score numérique, car il transforme la recommandation en aide à la décision.

2.3 Orchestration agentique et génération contrôlée

2.3.1 Architecture Agentic RAG et orchestration LangGraph

L'orchestration du système repose sur un workflow agentique implémentable avec *LangGraph*. Méthodologiquement, LangGraph est retenu parce qu'il permet de représenter l'agent comme un graphe d'états plutôt que comme une simple succession de prompts [39]. La requête utilisateur est d'abord analysée afin d'identifier son intention : diagnostic, recommandation candidat, *skill gap*, recherche référentielle, question relationnelle sur le graphe, orientation métier ou recherche générale. Le système sélectionne ensuite les outils adaptés. Cette étape de routage est essentielle : interroger pgvector, Neo4j ou les documents réglementaires ne répond pas au même besoin.

Le workflow suit une chaîne contrôlée : `analyse_request`, `plan_tools`, `execute_tools`, `build_context`, `generate_final_answer`, puis critique éventuelle de la réponse. Le noeud d'analyse identifie le use-case. Le noeud de planification sélectionne les outils. Le noeud d'exécution appelle les bases et services nécessaires. Le noeud de construction de contexte organise les résultats récupérés. Le noeud de génération produit la réponse finale avec le LLM. Le critic vérifie ensuite si la réponse paraît suffisamment fidèle au contexte ; en cas de faiblesse, le workflow peut élargir le contexte avant de régénérer la réponse.

Ce choix s'inscrit dans la logique de l'Agentic RAG : le système n'est pas un simple prompt envoyé à un modèle, mais une orchestration d'outils spécialisés autour d'un état partagé [35, 36]. Dans le présent travail, l'agent n'est pas conçu comme une entité autonome non contrôlée. Il est plutôt un routeur raisonné, limité à des outils définis, dont les sorties sont tracées.

2.3.2 Text2Cypher et rôle du LLM

Deux usages du LLM sont distingués. Le premier concerne l'interrogation du graphe en langage naturel. Pour les questions de type `graph_query` ou les recherches générales nécessitant un raisonnement relationnel, la question peut être transformée en requête Cypher par un module Text2Cypher. Le modèle retenu pour cette tâche est `gemma-2-9b-finetuned-2024v1`. La génération de Cypher est encadrée par le schéma Neo4j : types de noeuds, propriétés et relations autorisées. Cette contrainte est indispensable, car un modèle génératif qui ne connaît pas le schéma réel peut produire des requêtes syntaxiquement plausibles mais inexécutables ou non pertinentes.

Le second usage du LLM concerne la génération de la réponse finale. Le système utilise l'API OpenRouter avec le modèle `openai/gpt-oss-20b:free`. Ce modèle ne doit pas inventer la réponse à partir de ses connaissances générales. Il reçoit un contexte construit par le workflow : résultats pgvector, lignes Neo4j, fragments documentaires et métadonnées de source. Le prompt lui demande de répondre en français, de structurer la réponse et de citer les informations utilisées.

Cette séparation des rôles est méthodologiquement importante. Text2Cypher sert à produire une requête structurée vers le graphe. Le LLM final sert à formuler une synthèse lisible. Dans les deux cas, le modèle est subordonné aux données disponibles et non l'inverse.

2.3.3 Modes d'accès et traçabilité des décisions

Le système prévoit trois modes d'accès : une interface Streamlit pour l'usage conversationnel, une API FastAPI pour l'intégration applicative et une CLI pour les tests de développement. Ces interfaces ne changent pas la logique de recommandation ; elles exposent le même moteur sous des formes différentes. La CLI facilite la vérification rapide des use-cases, Streamlit permet une démonstration interactive et FastAPI prépare l'intégration dans un environnement applicatif comme la plateforme Kmer-AI, les agences de recrutement (FNE et l'ACPE).

La traçabilité est intégrée à plusieurs niveaux. Les traces du workflow indiquent les noeuds traversés et les outils appelés. Les résultats conservent les identifiants des entités, les sources documentaires, les scores et les chemins du graphe. La réponse finale doit citer les éléments utilisés. Cette exigence est décisive dans un système d'aide à l'orientation ou au recrutement : une recommandation sans preuve peut être séduisante, mais elle n'est pas défendable.

2.3.4 Protocole d'évaluation prévu du système

L'évaluation est conçue comme une évaluation en plusieurs étages. Le premier étage porte sur le retrieval vectoriel : le système retrouve-t-il les offres, compétences, métiers ou documents attendus dans les premiers résultats ? Les métriques retenues sont notamment Precision@K, Recall@K, MRR@K, MAP@K et NDCG@K, couramment utilisées pour les tâches de recherche d'information [27, 28].

Le deuxième étage porte sur le graphe : les relations retournées sont-elles correctes, utiles et interprétables ? Les tests doivent couvrir les requêtes métier-compétence, candidat-compétence, offre-compétence, compatibilité de niveau et recherche de chemins explicatifs. Le troisième étage porte sur le score hybride : il s'agit de comparer les classements produits par la recherche vectorielle seule, le graphe seul et la combinaison hybride. Le quatrième étage porte sur la réponse générée : fidélité au contexte, pertinence par rapport à la question, couverture des preuves, citations et utilité opérationnelle.

La méthode distingue donc clairement performance de récupération et qualité de génération. Une réponse bien rédigée ne prouve pas que le système a retrouvé les bonnes informations. Inversement, un bon retrieval ne garantit pas une bonne synthèse. Cette séparation protège l'analyse contre une erreur fréquente dans les systèmes RAG : juger le système uniquement à l'apparence de la réponse finale.

Deuxième partie

Présentation des résultats

Construction opérationnelle du système hybride emploi-compétences

Ce chapitre présente la matérialisation opérationnelle de l'architecture méthodologique décrite précédemment. Son objectif est de montrer ce qui a été effectivement construit, alimenté, indexé, relié et rendu exploitable dans le système de recommandation emploi-compétences.

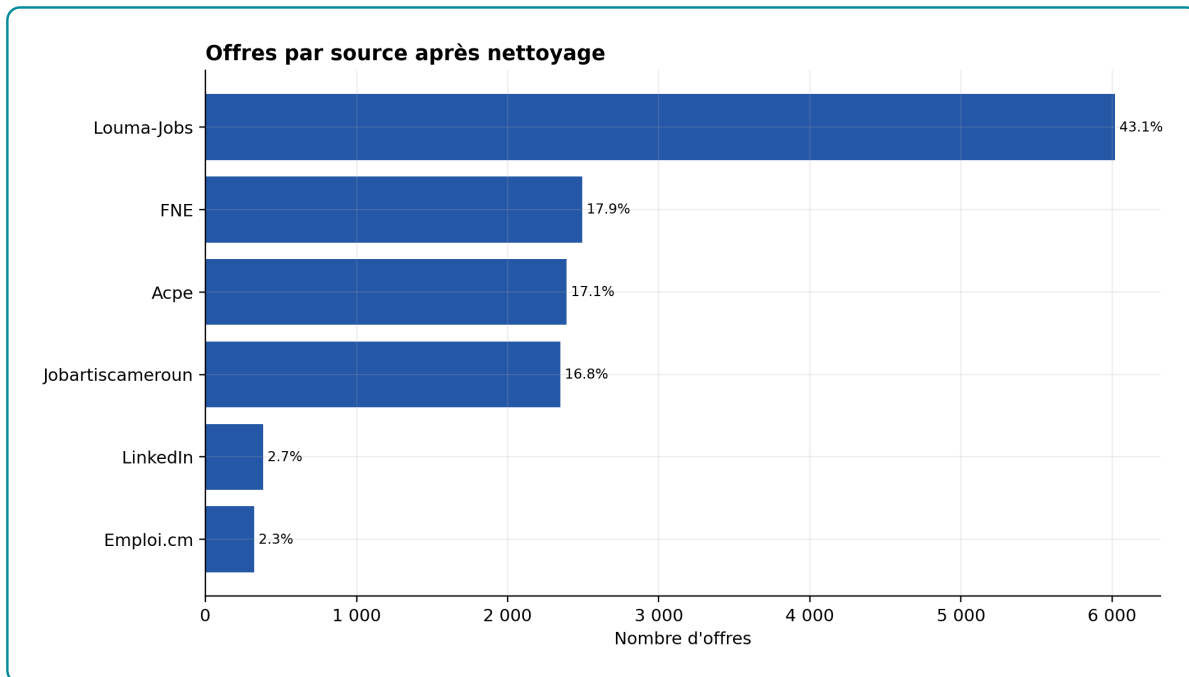
La présentation suit l'ordre réel du workflow : diagnostic des données exploitables, adaptation du modèle d'embeddings, matérialisation de la mémoire vectorielle, construction du graphe Neo4j, calibration du score hybride, puis orchestration agentique et exposition du système. Les résultats sont donc introduits au moment où ils justifient une décision d'implémentation, et non comme une simple succession de graphiques.

3.1 Données exploitables et contraintes de qualité du système

3.1.1 Volumes disponibles après nettoyage

Dans le workflow, cette étape correspond à l'entrée ETL : elle vérifie que les données nettoyées peuvent alimenter l'encodeur, pgvector et Neo4j. Les volumes détaillés sont reportés en annexe au tableau A.5. Le point utile ici est simple : le corpus contient assez d'offres, de candidats, de compétences, de métiers, de secteurs, de localisations et de référentiels pour construire un moteur hybride, mais ces objets ne sont pas équilibrés.

L'asymétrie entre 13 957 offres et 41 298 candidats impose donc de contrôler les effets de popularité. Un secteur ou une compétence très représenté peut dominer la similarité sans être le meilleur signal métier. Cette limite justifie le workflow retenu : nettoyer les données, encoder les textes, rappeler un vivier par pgvector, puis laisser Neo4j contrôler les relations utiles au matching.

Figure 3.1 – Répartition des offres par source après nettoyage

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Les 13 957 offres proviennent de sources hétérogènes, dont plusieurs ont été scrapées : Emploi.cm, Jobartis Cameroun, LinkedIn, Louma-Jobs, ainsi que les sources institutionnelles FNE et ACPE. Louma-Jobs domine le corpus avec 6 021 offres, soit 43,14 %. Le corpus est donc exploitable pour un prototype opérationnel, mais il ne doit pas être présenté comme une photographie exhaustive du marché camerounais de l'emploi.

3.1.2 Qualité des variables utiles au matching emploi-compétences

La qualité utile au matching dépend surtout de six variables : description, compétences, localisation, secteur, niveau et source. Elles alimentent directement le champ `text_to_embed`, les filtres pgvector et les relations Neo4j. Le diagnostic reste donc ciblé : que peut-on utiliser sans inventer d'information ?

3.1.2.1 Localisation exploitable pour filtrer sans exclure

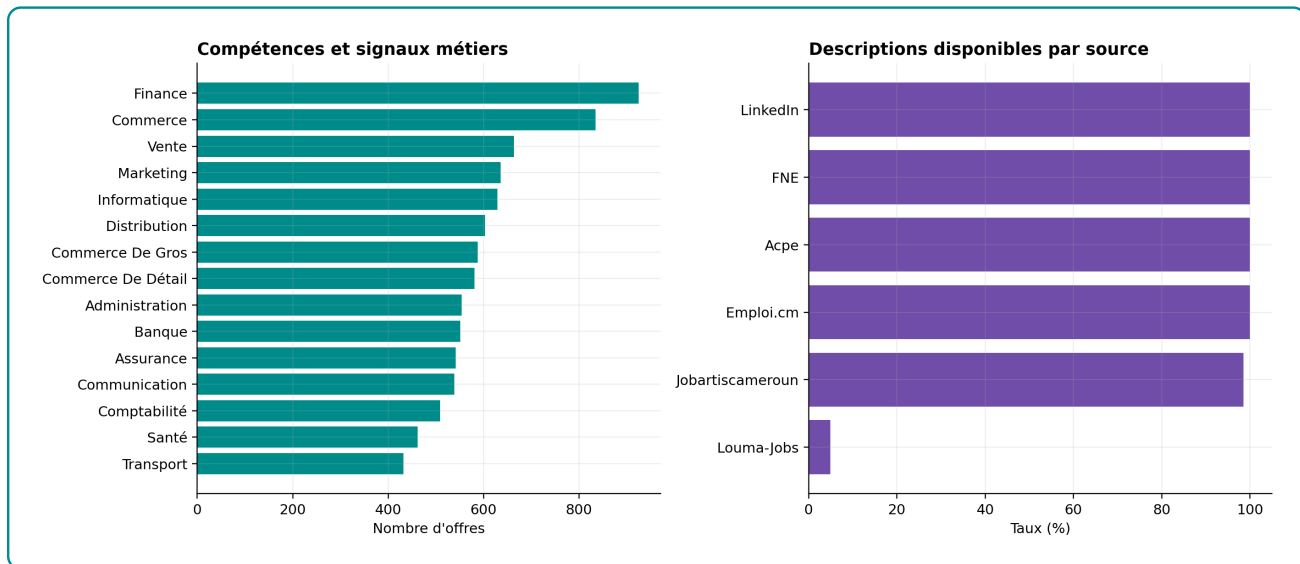
Le diagnostic détaillé de localisation est renvoyé en annexe (figure A.1). Dans le corps du chapitre, on retient seulement que la localisation est suffisamment renseignée pour filtrer et expliquer les recommandations, mais pas assez homogène pour être utilisée comme contrainte dure.

La localisation est exploitable, mais seulement comme filtre souple : 79,88 % des offres sont rattachées au Cameroun et 20,12 % à l'international. Lorsque la ville manque mais que le pays

existe, le nettoyage conserve le pays sans inventer une ville. Le système peut donc expliquer une recommandation par zone géographique, mais il ne doit pas exclure brutalement une offre parce que la ville est absente.

3.1.2.2 Compétences et descriptions : signal utile mais hétérogène

Figure 3.2 – Compétences fréquentes et disponibilité descriptive



Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Les signaux les plus fréquents sont Finance, Commerce, Vente, Marketing, Informatique, Distribution, Banque, Assurance, Communication, Comptabilité, Santé et Transport. Cette structure est cohérente avec un système emploi-compétences, mais elle révèle un risque méthodologique : plusieurs libellés sont des domaines d'activité plutôt que des compétences observables. Dans l'implémentation, ces variables sont donc utilisées comme signaux d'appariement et de contexte, non comme preuve définitive d'une compétence maîtrisée par un candidat.

La disponibilité descriptive est le principal point faible du corpus. Les sources FNE, ACPE, LinkedIn et Emploi.cm disposent d'une description pour toutes leurs offres dans le fichier nettoyé ; Jobartis Cameroun atteint 98,55 %. La limite vient de Louma-Jobs : seulement 295 descriptions disponibles sur 6 021 offres, soit 4,90 %. Cette hétérogénéité explique un choix central de l'ETL : la représentation envoyée au modèle d'embeddings ne peut pas être uniquement le détail textuel de l'offre ; elle doit combiner métadonnées, compétences, secteur, localisation et description lorsque celle-ci existe.

3.1.3 Nettoyage réalisé et limites conservées

Le nettoyage suit une règle conservatrice : corriger les formats sans fabriquer une information absente. Les libellés sont harmonisés, les localisations sont normalisées par priorité ville puis pays, et les doublons ne sont supprimés que lorsque la ligne normalisée est strictement identique. Ce choix protège le corpus multi-sources : deux offres peuvent avoir le même intitulé tout en différant par la ville, la source, le secteur, le niveau ou la description.

Le tableau détaillé des règles de nettoyage est placé en annexe (tableau A.6). Le chapitre conserve uniquement le principe d'ingénierie : corriger les formats sans inventer une information absente des données brutes.

Les graphiques secondaires détaillant la structure sectorielle, les villes détaillées, les niveaux NCF, l'expérience, la structure des candidats, les longueurs de textes et les référentiels MEPC-NCF sont renvoyés en annexes. Dans ce chapitre, seuls les résultats qui changent une décision d'architecture sont conservés : enrichir les embeddings, utiliser pgvector pour le rappel rapide, exploiter Neo4j pour les contraintes métier, et garder un critic pour contrôler les réponses générées.

3.2 Adaptation du modèle d'embeddings au domaine emploi-compétences

Dans le workflow méthodologique, cette section correspond à l'étape située après l'ETL et avant l'indexation pgvector. Les données nettoyées sont transformées en paires d'apprentissage, puis l'encodeur SentenceTransformer est adapté au vocabulaire emploi-compétences. Le résultat attendu n'est pas une réponse générée, mais un encodeur capable de produire des vecteurs plus utiles pour le retrieval.

3.2.1 Encodeur retenu et paires contrastives

Le modèle de base retenu est sentence-transformers/all-MiniLM-L6-v2. Ce choix est défendable pour une raison d'ingénierie : il produit des embeddings de 384 dimensions, donc moins coûteux à stocker et à rechercher que des modèles à 768 dimensions, tout en restant compatible avec une indexation HNSW dans pgvector. Dans ce projet, la contrainte est concrète : le même encodeur sert ensuite à vectoriser les candidats, les offres, les compétences, les métiers et les référentiels. Un modèle plus lourd peut améliorer certains cas sémantiques, mais il augmente la taille de stockage, la latence d'encodage et le coût de réindexation.

Ce modèle est acceptable comme point de départ parce qu'il fournit déjà une représentation

dense générale des phrases. Il n'est toutefois pas suffisant comme modèle final. Un encodeur généraliste rapproche des textes proches en langage courant, mais il ne connaît pas nécessairement les proximités utiles au domaine emploi-compétences camerounais : intitulés locaux, secteurs, niveaux de formation, familles métiers, descriptions incomplètes et compétences parfois formulées comme domaines. Le fine-tuning vise donc à spécialiser l'espace vectoriel sans changer l'architecture de production.

Table 3.1 – Protocole de construction des paires de fine-tuning

Élément	Implémentation observée
Modèle de départ	sentence-transformers/all-MiniLM-L6-v2, embeddings denses de 384 dimensions.
Rôle de <code>sentence1</code>	Représentation structurée de l'offre : titre, secteur, contrat, niveau d'études, expérience, ville et compétences.
Rôle de <code>sentence2</code>	Détails nettoyés de l'annonce uniquement, c'est-à-dire le texte descriptif que le modèle doit rapprocher de la représentation structurée.
Filtrage informatif	Conservation des paires avec compétences disponibles, <code>sentence1</code> d'au moins 40 caractères et <code>sentence2</code> d'au moins 120 caractères ; le champ <code>ft_eligible</code> est appliqué lorsqu'il existe.
Nombre total de paires	6 476 paires offre-description.
Split	4 542 paires en train, 981 en validation et 953 en test, avec stratification par secteur principal.

Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.

La structure `sentence1/sentence2` est volontairement asymétrique. `sentence1` représente la requête métier structurée que le système manipulera souvent en production : métadonnées, compétences et contexte de l'offre. `sentence2` représente le texte descriptif à retrouver. Ce choix entraîne le modèle à rapprocher une description naturelle d'une formulation structurée, ce qui correspond mieux au besoin du système qu'un entraînement sur deux phrases descriptives ordinaires.

Le filtrage des offres informatives est nécessaire. Entraîner le modèle sur des paires dont la description est vide, trop courte ou sans compétences introduirait du bruit : le modèle apprendrait à rapprocher des textes pauvres au lieu de structurer le domaine. Le seuil de

120 caractères pour sentence2 et de 40 caractères pour sentence1 est donc un garde-fou d'apprentissage. Il réduit le volume utilisé pour le fine-tuning, mais améliore la qualité du signal.

3.2.2 Fine-tuning : contraintes de calcul et preuve de gain

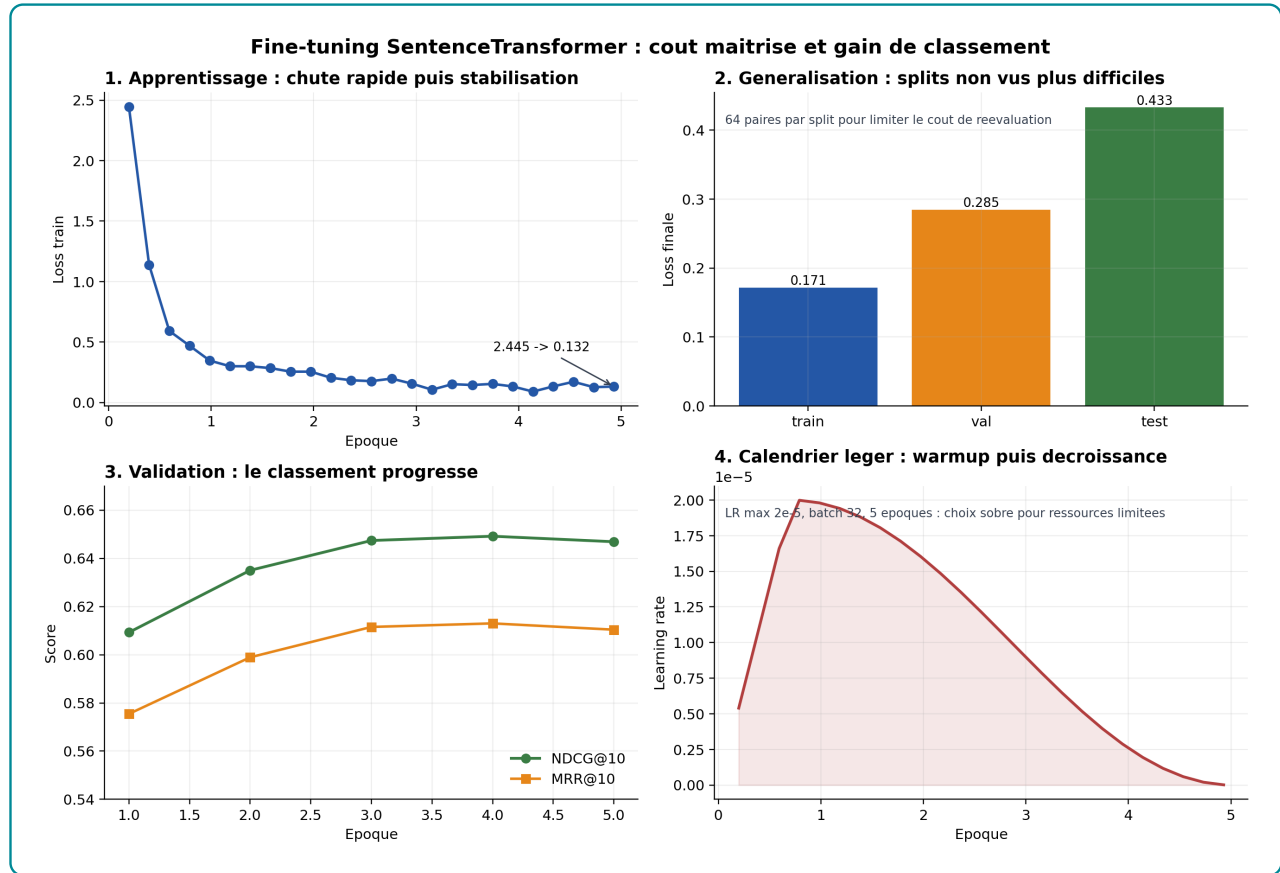
3.2.2.1 Paramétrage contraint et dynamique d'apprentissage

Table 3.2 – Hyperparamètres du fine-tuning SentenceTransformer

Hyperparamètre	Valeur
Nombre d'époques	5
Batch size	32
Négatifs implicites par exemple	31
Learning rate maximal	2×10^{-5}
Scheduler	Cosine avec warmup
Warmup	100 étapes configurées, limitées par les étapes disponibles de l'époque
Perte	<i>MultipleNegativesRankingLoss</i> avec similarité cosinus
Temps d'entraînement observé	15 160 s
Checkpoint retenu	Meilleur checkpoint sur validation autour de l'époque 4

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Ces hyperparamètres ne sont pas des valeurs arbitraires. Ils traduisent un compromis imposé par les ressources disponibles : le fine-tuning devait rester exécutable localement, sans infrastructure GPU lourde ni entraînement long. Le modèle all-MiniLM-L6-v2 a donc été conservé parce qu'il produit des vecteurs de 384 dimensions, moins coûteux à encoder, stocker et rechercher dans pgvector. Le batch de 32 limite la consommation mémoire tout en conservant assez d'exemples négatifs implicites pour la *MultipleNegativesRankingLoss*. Le nombre de cinq époques évite un entraînement excessif sur un corpus de paires encore imparfait. Le *learning rate* de 2×10^{-5} , avec *warmup* puis décroissance, permet une adaptation progressive du modèle sans déstabiliser les représentations déjà apprises.

Figure 3.3 – Diagnostic du fine-tuning : perte, généralisation, métriques de validation et calendrier du learning rate

Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.

La figure 3.3 se lit en quatre blocs. Le premier montre que la perte d'entraînement chute fortement au début, de 2,445 à 0,132 en fin d'entraînement : le modèle apprend rapidement à rapprocher les paires positives du domaine. Le deuxième bloc compare la perte finale sur un échantillon fixe de 64 paires par split : 0,171 sur train, 0,285 sur validation et 0,433 sur test. Cette hausse est normale : les jeux non vus sont plus difficiles que les exemples utilisés pendant l'apprentissage. Le troisième bloc montre que les métriques de validation progressent jusqu'aux dernières époques, avec un léger tassement final ; il n'y a donc pas de signal évident d'effondrement. Le quatrième bloc rappelle le choix de ressources : un *learning rate* maximal de 2×10^{-5} , 100 étapes de *warmup*, un batch de 32 et seulement cinq époques. Le protocole cherche donc un gain utile de classement, pas une optimisation coûteuse et difficile à reproduire.

Ces valeurs ne remplacent pas les métriques de ranking, car la *MultipleNegativesRankingLoss* dépend de la composition des batches. Elles servent surtout de contrôle de cohérence : si la

perte train baissait mais que la validation stagnait ou chutait, le fine-tuning serait suspect. Ici, la baisse de perte et la progression du NDCG@10/MRR@10 indiquent que l'adaptation du modèle améliore bien le retrieval dans le domaine emploi-compétences.

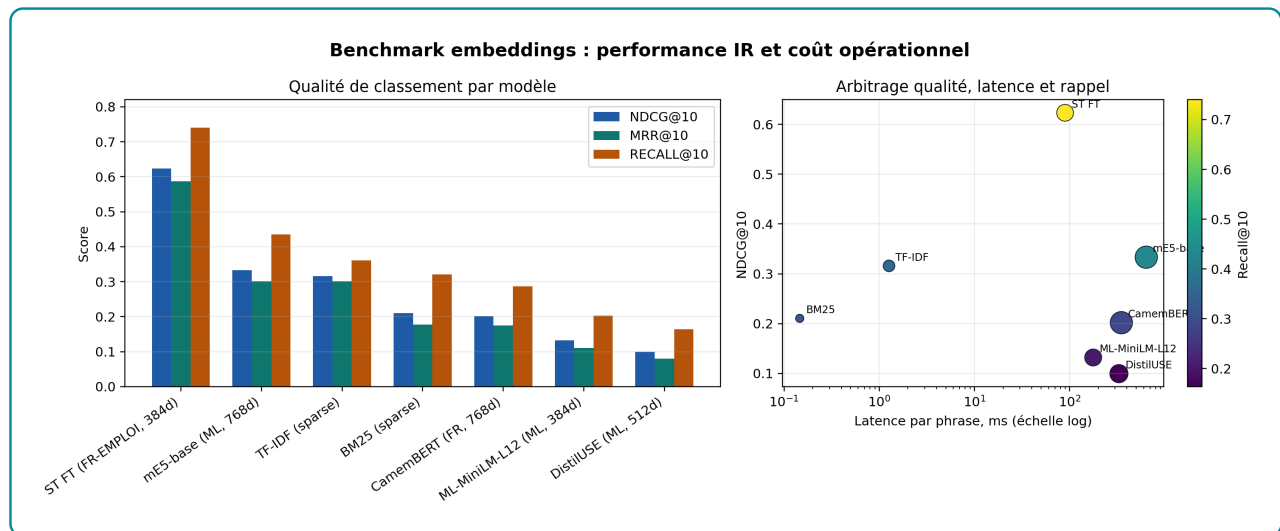
La validation atteint un NDCG@10 maximal de 0,6492 à l'époque 4, avant un léger recul à 0,6469 à l'époque 5. Ce profil est cohérent avec un apprentissage rapide suivi d'un plateau. Il serait donc fragile de prolonger l'entraînement sans contrôle : au-delà d'un certain point, le gain marginal devient faible et le risque de sur-adaptation augmente.

3.2.2.2 Preuve de gain sur le ranking sémantique

La comparaison numérique directe entre baseline et modèle fine-tuné est reportée en annexe (tableau A.7), tandis que l'évaluation complète du retrieval est reprise au chapitre 4.

Cette comparaison a été régénérée sur 953 paires de test après la mise à jour du jeu de paires. Elle est la preuve la plus directe de l'intérêt du fine-tuning : le modèle de base ne structure pas correctement le domaine dans ce protocole, alors que le modèle adapté récupère beaucoup plus souvent la description pertinente dans les premiers rangs. Le NDCG@10 est multiplié par 3,7 et le MRR@10 par 4,2. Le gain de recall@10 est particulièrement important, car l'application ne dépend pas d'une seule réponse immédiate ; elle récupère d'abord un ensemble court de candidats pertinents avant reranking hybride.

Figure 3.4 – Comparaison des modèles d'embeddings : qualité de classement, rappel et latence



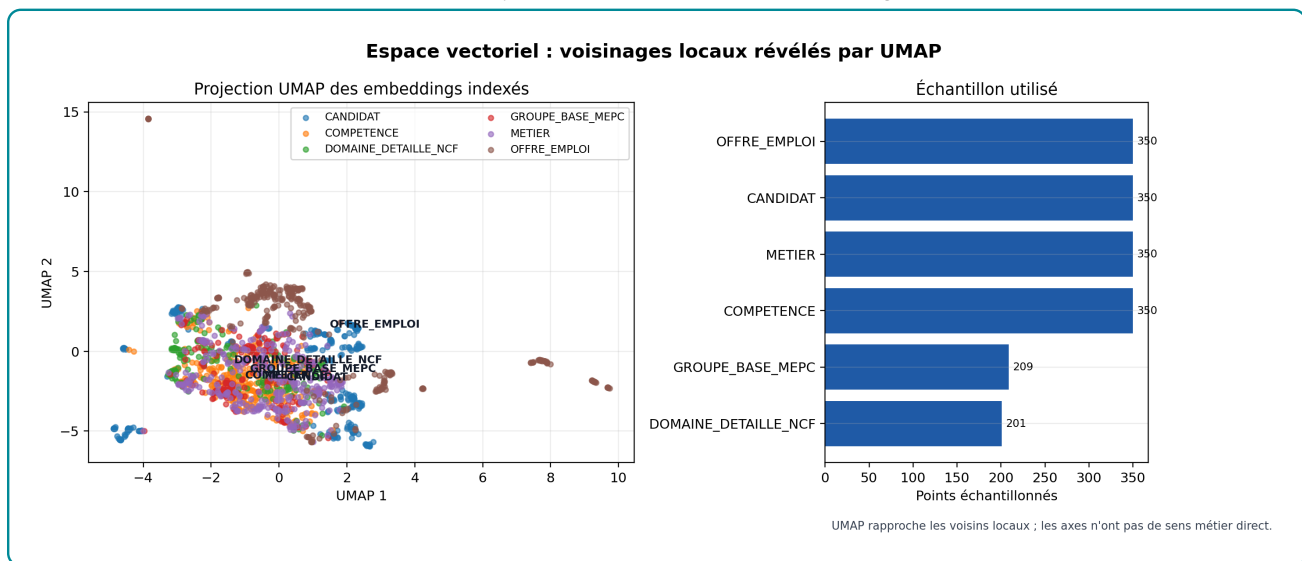
Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.

Le benchmark multi-modèles confirme le résultat. Le modèle fine-tuné atteint un NDCG@10 de 0,6232, un MRR@10 de 0,5874 et un recall@10 de 0,7398. Le meilleur modèle généraliste

dense testé, mE5-base, atteint seulement 0,3333 en NDCG@10 et 0,4355 en recall@10, avec une latence nettement plus élevée. TF-IDF reste très rapide et compétitif sur certains signaux lexicaux, mais son recall@10 reste inférieur à celui du modèle fine-tuné. L'intérêt du fine-tuning est donc empirique, pas seulement théorique. La bonne architecture n'oppose pas « deep learning » et lexical ; elle utilise le modèle fine-tuné comme signal sémantique principal et conserve les signaux lexicaux comme complément.

3.2.3 Structure de l'espace vectoriel après fine-tuning

Figure 3.5 – Projection UMAP des embeddings indexés



Source : Nos travaux, traitements Python et SentenceTransformer, à partir des paires emploi-compétences et des entités nettoyées du projet.

La figure 3.5 utilise UMAP pour projeter en deux dimensions un échantillon de 1 810 embeddings indexés : 350 candidats, 350 offres, 350 compétences, 350 métiers, 209 groupes de base MEPC et 201 domaines détaillés NCF. UMAP est ici un outil de lecture visuelle. La première lecture porte sur la séparation partielle des familles d'objets. Les offres d'emploi apparaissent plus excentrées, avec un centroïde situé dans une zone différente de celle des autres types d'entités. C'est cohérent avec leur contenu : une offre combine titre, secteur, contrat, localisation, niveau et description, donc un vocabulaire plus contextuel. Les candidats, compétences, métiers, groupes MEPC et domaines NCF sont davantage rapprochés dans la partie centrale du nuage. Cette proximité n'est pas un défaut. Le système a précisément besoin que ces objets partagent un espace commun : un candidat doit pouvoir être rapproché de compétences, de métiers et d'offres, sinon pgvector ne pourrait pas servir de premier étage de

retrieval.

La deuxième lecture concerne le recouvrement entre catégories. Le nuage ne montre pas des blocs parfaitement séparés, et il ne faut pas chercher cette séparation. Dans un système emploi-compétences, une compétence, un métier et un profil candidat peuvent partager les mêmes termes : « comptabilité », « vente », « marketing », « informatique ». Le recouvrement visuel signifie donc que le modèle place dans une même région des objets qui parlent du même domaine, même s'ils n'ont pas le même type technique dans la base. C'est utile pour récupérer des voisins sémantiques ; c'est aussi la raison pour laquelle Neo4j reste nécessaire ensuite, car pgvector rapproche les textes mais ne vérifie pas à lui seul les relations POSSEDE, REQUIERT ou NECESSITE.

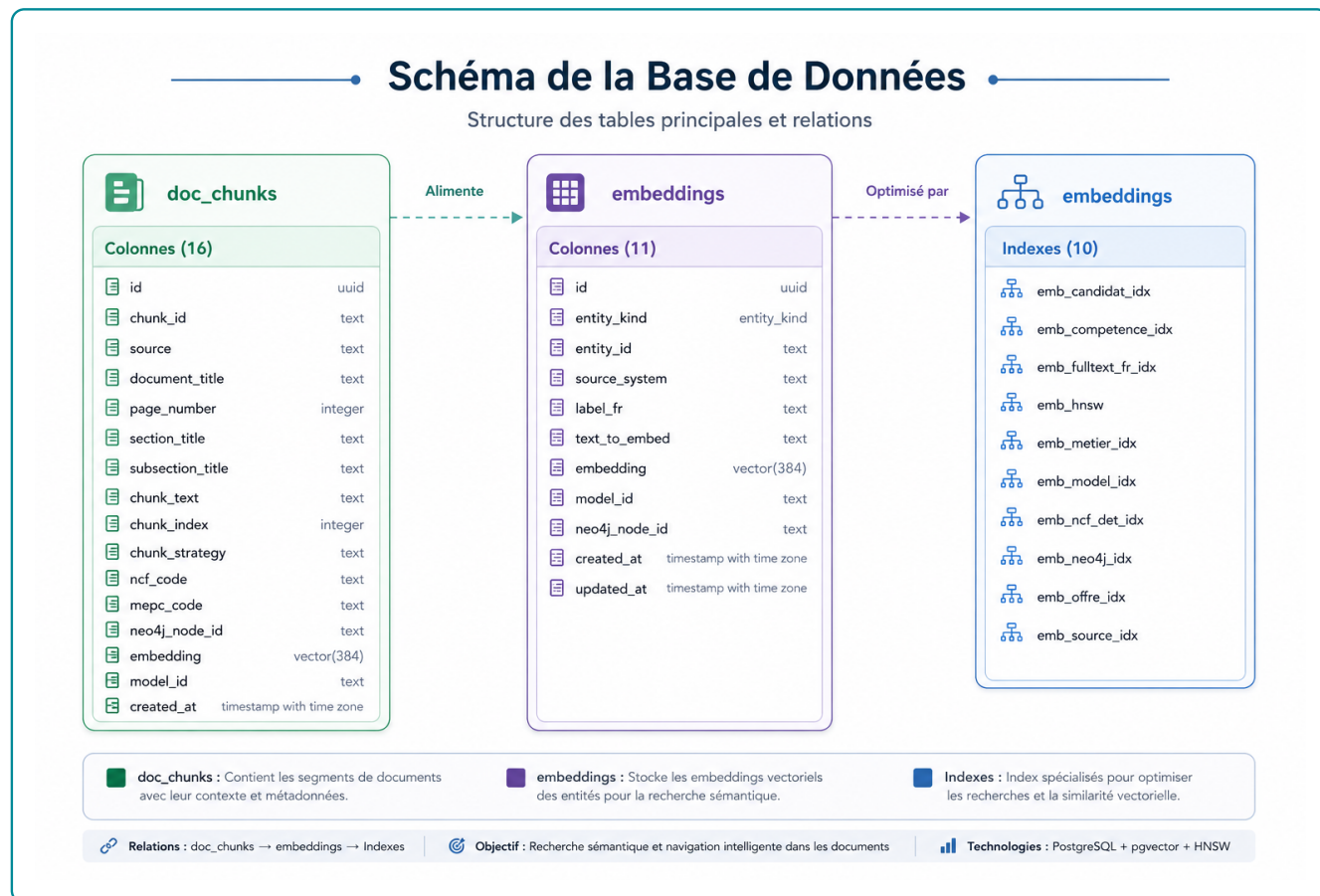
La visualisation doit enfin être lue avec discipline. UMAP ne prouve pas que les recommandations sont bonnes et ne remplace pas le benchmark. Il sert à vérifier que l'espace vectoriel n'est pas incohérent : les offres ne sont pas mélangées au hasard, les objets référentiels restent raccordés aux profils et aux compétences, et les zones mixtes rendent possible la recherche sémantique. La preuve quantitative reste le ranking présenté en section précédente, notamment le NDCG@10, le MRR@10 et le recall@10.

3.3 Mémoire vectorielle pgvector pour le retrieval

Dans le workflow, pgvector intervient juste après l'encodage des entités par le modèle fine-tuné. Il constitue le premier étage du retrieval de l'Agentic RAG : transformer une requête, un profil ou un fragment documentaire en vecteur, récupérer rapidement un top- K d'objets proches, puis transmettre ce vivier au moteur hybride et au graphe Neo4j. pgvector ne décide donc pas seul de la recommandation finale ; il fournit la liste de départ que le graphe va contrôler.

3.3.1 Schéma physique, tables et volumes indexés

Figure 3.6 – Schéma physique de la base pgvector : tables documentaires, embeddings et index de recherche



Source : Nos travaux, diagnostics PostgreSQL-pgvector

La figure 3.6 donne la lecture la plus simple de la brique pgvector. La table embeddings stocke les vecteurs des entités structurées : candidats, offres, compétences, métiers, groupes MEPC et domaines NCF. La table doc_chunks stocke les fragments documentaires issus des référentiels. Les index spécialisés servent ensuite à interroger ces deux familles d'objets par similarité vectorielle, par recherche textuelle, par type d'entité, par modèle d'embedding ou par identifiant Neo4j.

Cette séparation évite une confusion fréquente : pgvector n'est pas seulement une colonne vector. C'est une couche de retrieval qui associe trois informations dans la même base : le vecteur, les métadonnées métier et le lien vers Neo4j. Le schéma suffit donc à comprendre les deux tables utiles : embeddings pour les entités structurées et doc_chunks pour les fragments documentaires. Le détail technique des champs n'est pas répété ici, car l'image présente déjà

la structure physique de la base.

Le modèle d'encodage utilisé est all-MiniLM-L6-v2-ft-offres-cm. Les vecteurs ont une dimension de 384, ce qui limite le coût de stockage et de recherche tout en restant adapté au retrieval sémantique. L'instance courante contient 72 643 embeddings d'entités structurées et 142 embeddings documentaires, soit 72 785 vecteurs interrogeables.

Table 3.3 – Distribution des embeddings par type d'entité

Type d'entité	Vecteurs	Part	Couverture Neo4j
Candidats	41 298	56,85 %	100 %
Offres d'emploi	13 957	19,22 %	100 %
Compétences	13 939	19,19 %	100 %
Métiers	3 039	4,18 %	100 %
Groupes de base MEPC	209	0,29 %	100 %
Domaines détaillés NCF	201	0,28 %	100 %
Total entités structurées	72 643	100 %	100 %
Chunks documentaires	142	–	100 %

Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings, des métadonnées et des fragments documentaires indexés.

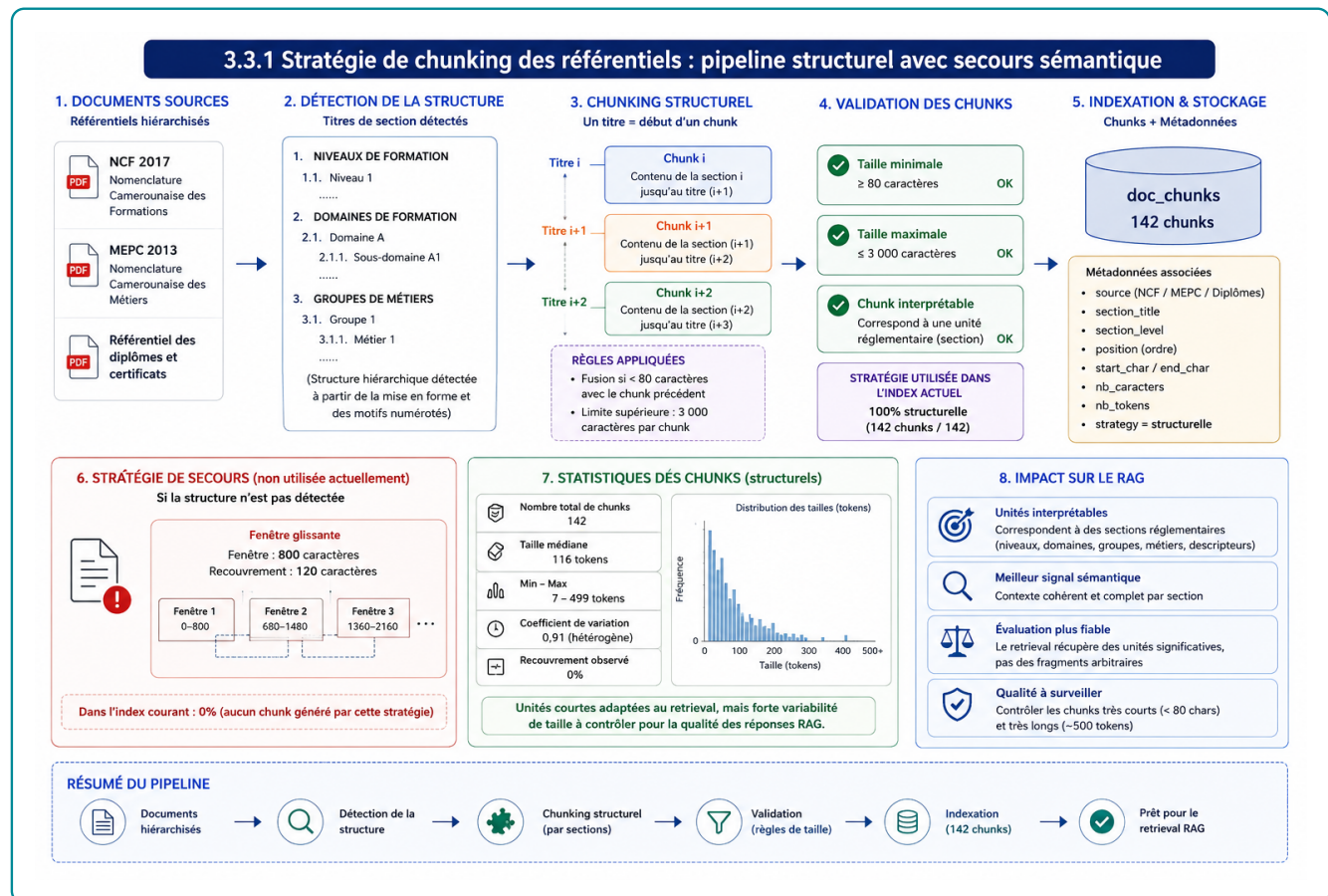
Cette distribution montre que pgvector couvre les deux côtés du matching. Les candidats représentent 56,85 % des embeddings et les offres 19,22 %. Les compétences et métiers forment le vocabulaire pivot qui permet de relier une similarité textuelle à une logique emploi-compétences. La couverture Neo4j de 100 % est le point décisif : chaque résultat vectoriel peut être replacé dans le graphe pour récupérer secteur, niveau, localisation, compétences reliées ou chemin explicatif.

3.3.2 Référentiels découpés en chunks interrogeables

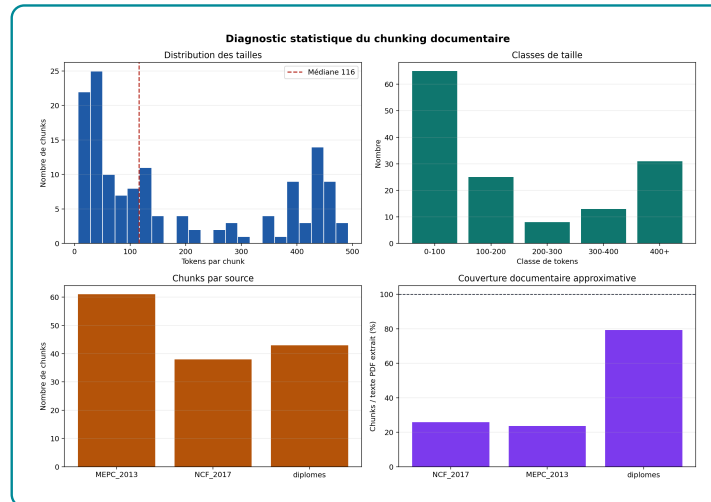
Les référentiels ne sont pas indexés comme de simples PDF. Ils sont d'abord découpés en fragments documentaires, puis chaque fragment reçoit un embedding et un identifiant Neo4j. Les documents référentiels ajoutent 142 chunks : 43 pour les diplômes, 61 pour le MEPC 2013 et 38 pour la NCF 2017. Cette double indexation est nécessaire au GraphRAG : pgvector retrouve le passage pertinent, puis Neo4j le rattache à une formation, un métier, un niveau ou un domaine.

La figure 3.7 situe précisément cette étape dans le workflow : documents sources, détection des titres, découpage structurel, validation des tailles, puis indexation dans doc_chunks. La stratégie d'ingestion est structurelle par défaut. Elle détecte les titres de section à partir de la mise en forme et des motifs numérotés, puis chaque titre ouvre un chunk dont le contenu s'étend jusqu'au titre suivant. Les fragments de moins de 80 caractères sont fusionnés avec le fragment précédent et une limite supérieure de 3 000 caractères contrôle la taille maximale. La stratégie sémantique de secours par fenêtres glissantes existe dans le pipeline, mais elle n'a pas été utilisée dans l'index courant : les 142 chunks proviennent tous du découpage structurel.

Figure 3.7 – Pipeline de chunking structurel des référentiels documentaires

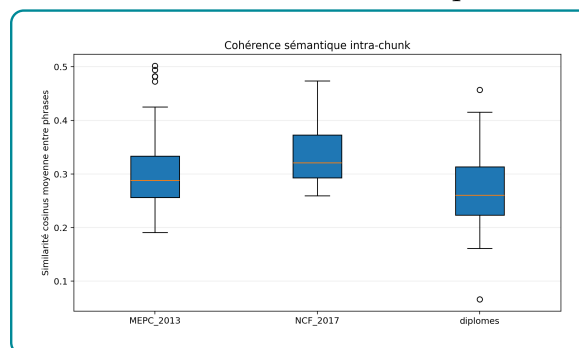


Source : Nos travaux, schématisation du pipeline de chunking documentaire et d'indexation pgvector.

Figure 3.8 – Distribution des tailles, sources et couverture du chunking documentaire

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

La figure 3.8 vérifie que le pipeline produit des unités interrogeables. La médiane de 116 tokens est adaptée au retrieval : les passages sont assez courts pour être précis, mais assez longs pour conserver un contexte de section. La forte variabilité, avec un coefficient de variation de 0,91, vient de la structure des référentiels : certaines sections sont de simples intitulés, d'autres contiennent des listes ou descriptions plus longues. Ce n'est pas une erreur de traitement ; c'est un signal de surveillance. Les très petits chunks risquent d'être peu informatifs, tandis que les plus longs peuvent mélanger plusieurs idées.

Figure 3.9 – Distribution de la cohérence sémantique intra-chunk par source

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

La figure 3.9 complète cette lecture par source. La cohérence sémantique moyenne vaut 0,303 : les chunks restent exploitables, mais ils ne sont pas toujours très homogènes. Cette valeur est cohérente avec des référentiels administratifs qui combinent codes, titres, listes et descriptions courtes. L'interprétation pratique est claire : le chunking structurel respecte bien la logique réglementaire, mais les réponses RAG doivent citer les passages récupérés et éviter

de surinterpréter un chunk très court ou trop composite. Les mesures détaillées sont reportées en annexe aux tableaux A.9, A.8 et A.10.

3.3.3 Index et modes de recherche mobilisés

Les index pgvector ne sont pas seulement un détail de base de données ; ils déterminent la manière dont le workflow récupère les preuves. Trois usages sont distingués. Le premier est le rappel sémantique top-K par HNSW, utilisé pour obtenir rapidement les offres, candidats, métiers ou compétences proches d'une requête. Le deuxième est le filtrage par type d'entité, indispensable pour éviter de comparer une offre avec un chunk documentaire lorsque l'outil attend une offre. Le troisième est la recherche textuelle française, utile lorsque la requête contient un terme exact que l'embedding peut diluer.

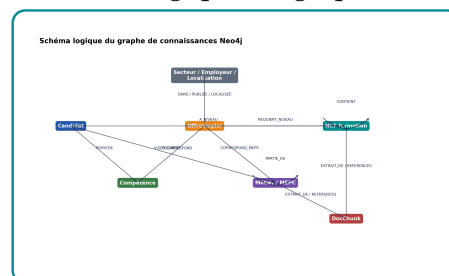
La latence observée reste compatible avec un usage interactif : la médiane vaut 30,89 ms pour un top-10 HNSW. Le maximum de 160,40 ms rappelle cependant que l'Agentic RAG doit raisonner en chaîne complète : retrieval, enrichissement Neo4j, construction du contexte et génération. Les indicateurs techniques détaillés sont donc placés en annexe aux tableaux A.11 et A.12. Dans le corps du chapitre, la conclusion est limitée mais suffisante : pgvector produit un vivier rapide, typé et raccordable à Neo4j.

3.4 Neo4j : mémoire relationnelle du matching

Dans le workflow, Neo4j intervient après le rappel pgvector. pgvector propose des voisins sémantiques ; Neo4j vérifie ensuite si ces voisins sont reliés par des chemins métier défendables : compétences possédées, compétences requises, métier de référence, niveau NCF, secteur et localisation. Cette section montre donc comment le graphe transforme une proximité textuelle en explication contrôlable.

3.4.1 Schéma, noeuds et relations mobilisés par le matching

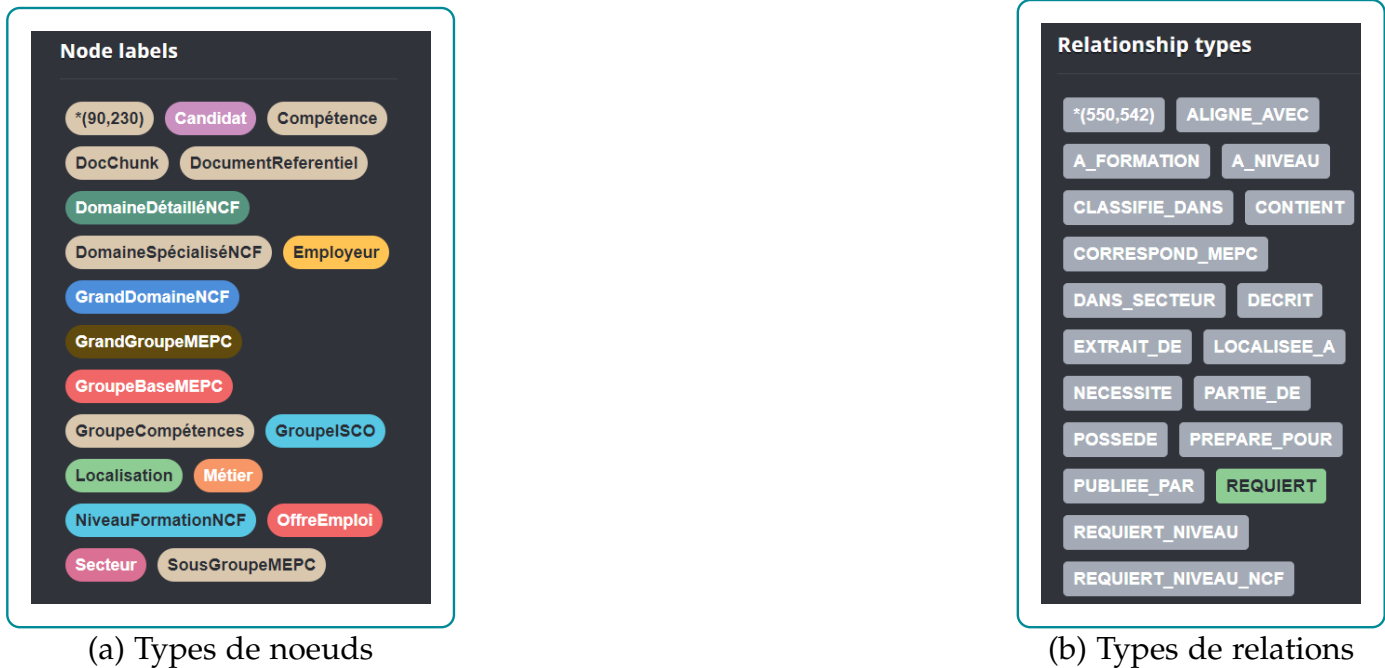
Figure 3.10 – Schéma logique du graphe de connaissances



Source : Nos travaux, diagnostics Neo4j

La figure 3.10 donne la structure logique du graphe : les candidats et les offres sont reliés aux compétences, métiers, secteurs, localisations, niveaux NCF, référentiels MEPC et chunks documentaires. L'idée centrale est simple : pgvector mesure une proximité textuelle, tandis que Neo4j vérifie si cette proximité repose sur des relations métier lisibles.

Figure 3.11 – Lecture conjointe des noeuds et relations Neo4j mobilisés par le matching



Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

La figure 3.11 doit être lue en deux temps. Le panneau des noeuds indique les objets que le système manipule : Candidat, OffreEmploi, Compétence, Métier, référentiels MEPC, niveaux et domaines NCF, secteurs, employeurs, localisations et fragments documentaires. Le panneau des relations indique ce que le système sait vérifier entre ces objets : un candidat POSSEDE des compétences, une offre REQUIERT des compétences, un métier NECESSITE des compétences de référence, une offre est rattachée à un secteur, à un niveau, à un employeur et à une localisation. L'information utile n'est donc pas le noeud isolé, mais le couple « objet + relation ».

La figure 3.11 précise les objets manipulés et les relations effectivement mobilisées. Une recommandation devient défendable lorsque le chemin est explicite : candidat → compétences possédées → compétences requises par l'offre, puis éventuellement métier, niveau NCF, secteur ou formation de référence. Si ce chemin est absent ou porté seulement par une compétence trop générale, le graphe doit rétrograder l'offre plutôt que la justifier artificiellement.

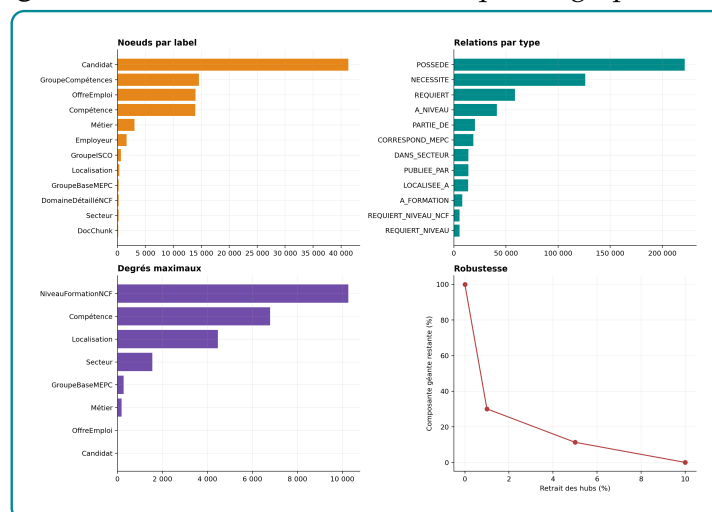
Ces relations restent des signaux opérationnels, pas des vérités absolues. Les liens REQUIERT et POSSEDE dépendent de l'extraction et du rattachement lexical des compétences. Une com-

pétence mal extraite ou trop générique peut donc améliorer artificiellement le score. C'est pourquoi les indicateurs de structure, de hubs et de centralité sont nécessaires avant d'utiliser Neo4j comme brique de reranking.

3.4.2 Structure du graphe et risques de hubs

3.4.2.1 Taille réelle du graphe et couverture de la projection matching

Figure 3.12 – Tableau de bord statistique du graphe Neo4j



Source : Nos travaux, diagnostics Neo4j

La figure 3.12 synthétise les volumes et la structure globale du graphe avant l'analyse détaillée. Elle sert ici de point d'entrée : le graphe doit d'abord être compris par ses ordres de grandeur, puis seulement par ses hubs, ses communautés et ses chemins de matching.

Le graphe complet contient 90 230 noeuds et 550 542 relations, répartis en 18 labels et 18 types de relations. Sa densité dirigée globale est très faible, environ $6,76 \times 10^{-5}$, ce qui est normal pour un graphe hétérogène de connaissances : toutes les entités n'ont pas vocation à être reliées entre elles. Cette faible densité ne signifie donc pas que le graphe est pauvre ; elle signifie qu'il encode des relations spécialisées plutôt qu'un réseau où tout serait connecté à tout.

Pour analyser la partie réellement mobilisée par le matching, une projection non orientée a été construite à partir des relations POSSEDE, REQUIERT, NECESSITE, CORRESPOND_MEPC, A_NIVEAU, REQUIERT_NIVEAU, DANS_SECTEUR et LOCALISEE_A. Cette projection contient 72 516 noeuds et 499 080 arêtes ; elle forme une seule composante connexe. Ce résultat est important : les entités utiles au matching ne sont pas dispersées en sous-graphes indépendants, elles appartiennent à un même réseau de propagation candidat-offre-compétence. En pratique, cela permet au

GraphRAG de construire des chemins courts entre un profil, une offre, une compétence, un métier ou un niveau de formation. La composition détaillée des noeuds et relations est reportée en annexe (figure A.2).

Les principaux labels sont Candidat avec 41 298 noeuds, GroupeCompétences avec 14 579 noeuds, OffreEmploi avec 13 957 noeuds, Compétence avec 13 939 noeuds et Métier avec 3 039 noeuds. Les cinq premiers labels concentrent 96,21 % des noeuds. Cette structure est cohérente avec l'objectif métier : le graphe est d'abord un graphe d'appariement, non un graphe encyclopédique. Le risque statistique est clair : si le score de recommandation n'est pas normalisé, les familles très représentées peuvent dominer la recherche au détriment de signaux plus spécifiques comme le métier, le niveau ou le secteur.

Cette structure impose toutefois une lecture prudente. Comme les relations de compétences dominant fortement, le score final dépend beaucoup de leur qualité. Si une offre n'a pas de relation REQUIERT, Neo4j ne peut pas calculer correctement la couverture de compétences. Si une compétence générique devient trop connectée, elle peut créer un hub peu discriminant. L'apport du graphe existe donc à condition de contrôler la complétude et la précision des relations.

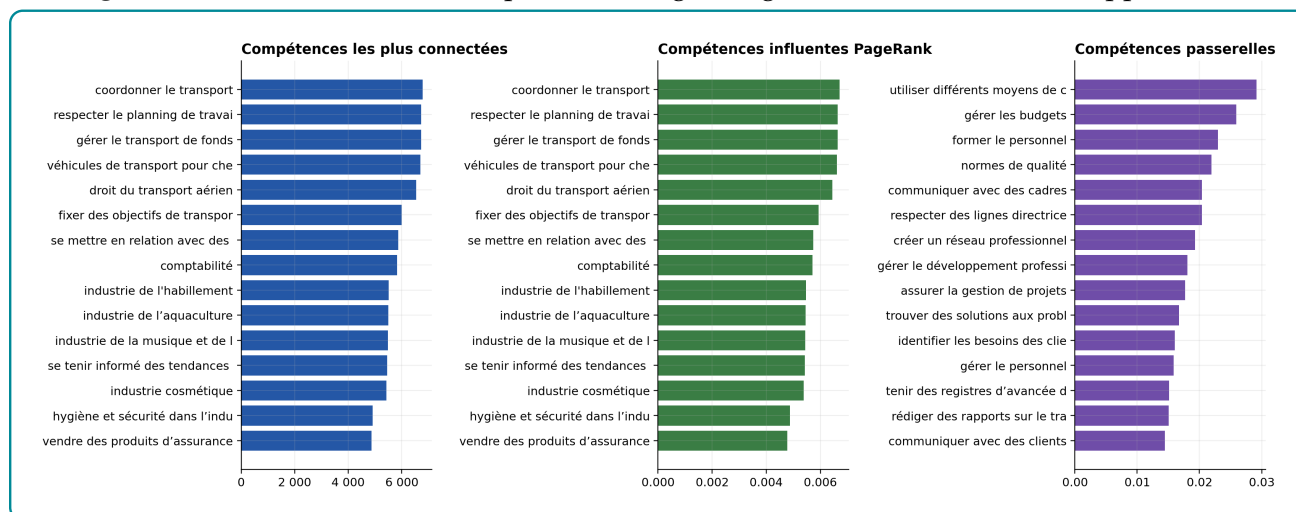
3.4.2.2 Noeuds très connectés : signal utile ou bruit de popularité

La distribution complète des degrés est placée en annexe (figure A.3). La distribution des degrés est fortement asymétrique. L'estimation exploratoire d'une loi de puissance sur la queue de distribution donne un exposant compris entre 1,40 et 2,51 selon le seuil $x_{\min}$ retenu. Ce n'est pas une preuve formelle de loi de puissance, mais un diagnostic utile : le graphe comporte bien des hubs. Dans un système de recommandation, ces hubs doivent être traités avec prudence. Une compétence très connectée n'est pas automatiquement discriminante ; elle peut simplement être générique, très fréquente ou issue d'un référentiel large.

Le tableau d'hétérogénéité des degrés par famille de noeuds est reporté en annexe (tableau A.13). Les degrés moyens montrent une forte hétérogénéité. Les noeuds NiveauFormationNCF ont un degré moyen de 5 812,56, parce que neuf niveaux seulement absorbent une grande partie des relations candidat–niveau et offre–niveau. Les GroupeBaseMEPC, les Secteur, les Métier, les Localisation et les Compétence jouent aussi un rôle de concentration. Cette hétérogénéité est une force pour l'explication, mais une faiblesse pour le scoring si elle n'est pas normalisée : le graphe peut favoriser des noeuds populaires au lieu de noeuds réellement informatifs.

3.4.2.3 Compétences pivots selon degré, PageRank et intermédiarité

Figure 3.13 – Centralités des compétences : degré, PageRank et intermédiarité approximée



Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

Les centralités donnent trois lectures complémentaires. La centralité de degré et le PageRank identifient surtout les compétences les plus connectées, par exemple « coordonner le transport », « respecter le planning de travail dans les transports », « gérer le transport de fonds » et « comptabilité ». L'intermédiarité approximée, calculée sur le sous-graphe des noeuds de plus fort degré, fait ressortir d'autres compétences : « utiliser différents moyens de communication », « gérer les budgets », « former le personnel », « normes de qualité » et « assurer la gestion de projets ». Ces dernières sont plus intéressantes pour la recommandation, car elles jouent un rôle de passerelle entre plusieurs familles de métiers.

Les exemples de compétences centrales selon les métriques de graphe sont placés en annexe (tableau A.14).

La distinction entre degré, PageRank et intermédiarité est importante. Un noeud à fort degré est fréquent ; il peut donc être utile, mais peu discriminant. Un noeud avec PageRank élevé structure le réseau parce qu'il est relié à des noeuds eux-mêmes importants. Un noeud avec intermédiarité élevée sert de passerelle entre plusieurs zones du graphe. Pour le *skill gap*, ces compétences passerelles sont particulièrement utiles : elles indiquent des compétences transversales qui peuvent rapprocher plusieurs trajectoires professionnelles. Dans le moteur hybride, cela justifie de ne pas traiter toutes les compétences manquantes de la même manière.

3.4.2.4 Communautés Louvain et robustesse aux hubs

Les communautés Louvain et distances sont détaillées en annexe (figure A.4).

La détection de communautés a été réalisée avec Louvain sur un échantillon pondéré de la composante géante, enrichi par les noeuds de plus fort degré afin de conserver l'ossature du réseau. Elle identifie 100 communautés dans la projection analysée. Ce résultat ne doit pas être vendu comme une typologie définitive des métiers ; il sert à vérifier que le graphe contient des regroupements relationnels exploitables. La distance moyenne échantillonnée dans la composante géante vaut 4,10 sauts, avec une médiane de 4. En pratique, cela signifie qu'un candidat, une offre, une compétence ou un métier peut être relié au reste du réseau par quelques relations seulement, ce qui rend le GraphRAG opérationnel pour produire des chemins explicatifs courts.

Le test de robustesse après retrait des hubs est reporté en annexe (figure A.5).

Le test de robustesse retire successivement les 1 %, 5 % et 10 % de noeuds les plus centraux selon le degré, puis mesure la taille de la composante géante restante. Ce test est sévère, car il cible les hubs au lieu de supprimer des noeuds au hasard. Son intérêt est de mesurer la dépendance du graphe à quelques entités très connectées. Si la composante géante s'effondre fortement après retrait des hubs, cela signifie que le système dépend excessivement de compétences ou niveaux génériques. Dans ce cas, le reranking doit pénaliser les hubs trop peu discriminants et valoriser les compétences spécifiques.

3.4.2.5 Skill gap observé entre profils et offres

Les compétences moyennes par objet métier sont visualisées en annexe (figure A.6).

L'analyse métier confirme le rôle pivot des compétences. Une offre exige en moyenne 4,19 compétences, avec une médiane de 6. Un candidat possède en moyenne 5,36 compétences, avec une médiane de 6. Les métiers sont beaucoup plus riches : 41,48 compétences en moyenne et 37 en médiane, parce qu'ils agrègent des exigences de référentiel. Cette différence est saine : l'offre et le candidat portent un signal opérationnel court ; le métier porte un contexte plus large qui sert à l'enrichissement, à l'explication et à la montée en compétences.

La distribution détaillée du skill gap candidat-offre est renvoyée en annexe (figure A.7).

Le *skill gap* a été mesuré sur un échantillon de couples candidat-offre en comparant les compétences REQUIERT de l'offre aux compétences POSSEDE du candidat. Le déficit moyen est de 5,32 compétences, avec une médiane de 6 ; seulement 12,33 % des couples échantillonnés ont un déficit inférieur ou égal à trois compétences. Ce résultat est sévère, mais utile : le graphe ne dit pas que la majorité des candidats sont immédiatement compatibles avec une offre prise au hasard. Il montre au contraire que le matching doit sélectionner, filtrer et reranker. C'est précisément la raison d'être du moteur hybride : utiliser pgvector pour présélectionner des

couples plausibles, puis Neo4j pour mesurer finement les compétences acquises, manquantes et passerelles.

L'interprétation opérationnelle est directe. Lorsque le déficit est faible, l'offre peut être considérée comme accessible ou proche. Lorsque le déficit est moyen, la recommandation doit être accompagnée d'un plan de montée en compétences. Lorsque le déficit est élevé, le système doit éviter de présenter l'offre comme immédiatement réaliste. Le graphe apporte donc une information que la similarité vectorielle seule ne fournit pas : il distingue la ressemblance textuelle d'une compatibilité professionnelle argumentée.

La conclusion à retenir est donc précise. Le graphe est riche parce qu'il relie les objets utiles au recrutement ; il est déséquilibré parce que certaines compétences et certains métiers concentrent beaucoup de relations ; il est décisionnel parce qu'il modifie le classement produit par pgvector ; il reste imparfait parce que les liens de compétences dépendent de la qualité de l'extraction et du rattachement lexical. Cette nuance est importante : Neo4j améliore l'expliquabilité et le reranking, mais il ne doit pas être présenté comme une garantie automatique de vérité métier.

3.5 Score hybride et reranking des recommandations

Le moteur hybride constitue le cœur décisionnel du système. Sans lui, l'application ne serait qu'un chatbot branché sur une recherche vectorielle. L'implémentation suit une logique en trois temps : pgvector présélectionne un ensemble d'offres candidates, Neo4j enrichit chaque couple candidat-offre avec des signaux relationnels, puis un score hybride réordonne les offres avant la génération. Le classement final n'est donc pas directement le score de similarité textuelle ; c'est un arbitrage pondéré entre le score pgvector et le score Neo4j, calibré ensuite par Optuna.

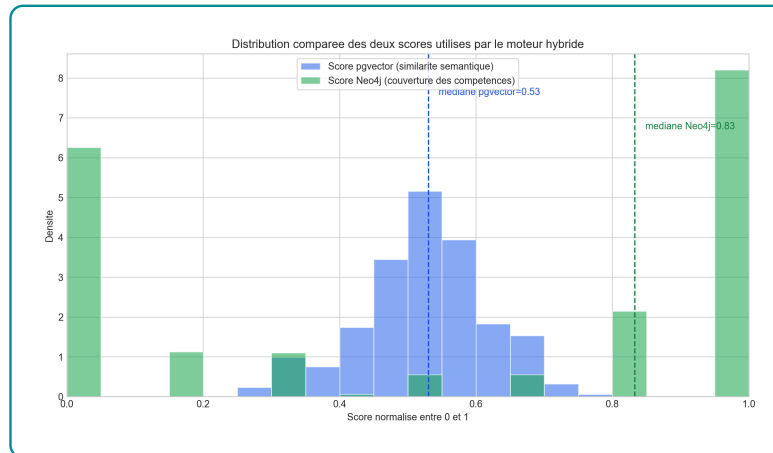
3.5.1 Formalisation du score hybride

Le score hybride utilisé pour le reranking ne doit pas être présenté comme une addition de nombreux critères indépendants. Dans la version évaluée ici, il combine deux signaux seulement. Le premier vient de pgvector : il mesure la proximité sémantique entre le profil ou la requête et l'offre candidate. Le second vient de Neo4j : il mesure la compatibilité relationnelle, principalement à travers la couverture des compétences requises par l'offre et possédées par le candidat. La formulation retenue est donc :

$$S_{\text{hybride}} = \alpha S_{\text{pgvector}} + \beta S_{\text{Neo4j}}, \quad \alpha + \beta = 1$$

où S_{pgvector} correspond à la colonne `score_sem` et S_{Neo4j} à la colonne `taux_match`. Cette écriture est volontairement simple : pgvector répond à la question « les textes se ressemblent-ils ? », alors que Neo4j répond à la question « les compétences exigées sont-elles couvertes ? ». Les autres informations du graphe, comme le niveau NCF, le secteur ou la localisation, restent utiles pour l'explication et les verdicts, mais elles ne sont pas ajoutées comme poids autonomes dans cette calibration. Sinon, le score deviendrait difficile à défendre statistiquement et risquerait de compter plusieurs fois le même signal métier.

Figure 3.14 – Distribution comparée des scores pgvector et Neo4j utilisés dans le reranking



Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings

La figure 3.14 montre que les deux scores n'ont pas le même comportement. Le score pgvector est continu et relativement concentré : sa moyenne vaut 0,5225 et sa médiane 0,5304 sur les 690 couples candidat–offre évalués. Le score Neo4j est plus discontinu : sa moyenne vaut 0,5606, sa médiane 0,8330, mais 31,30 % des couples ont un score nul. Cette différence est normale. pgvector renvoie presque toujours une proximité textuelle, même moyenne ; Neo4j est plus sévère, car une offre peut être textuellement proche tout en n'ayant aucune compétence requise couverte par le candidat.

Table 3.4 – Résultats de calibration des poids du score hybride

Objectif optimisé	Outil	α Sem	β Neo4j	NDCG@10
NDCG@10	Neo4j seul	0,0000	1,0000	0,9697
(NDCG@10 + Recall@10 + Precision@10) / 3	Neo4j seul	0,0000	1,0000	0,9697
(MRR@10 + Precision@10) / 2	Mixte Optuna	0,6615	0,3385	0,9674

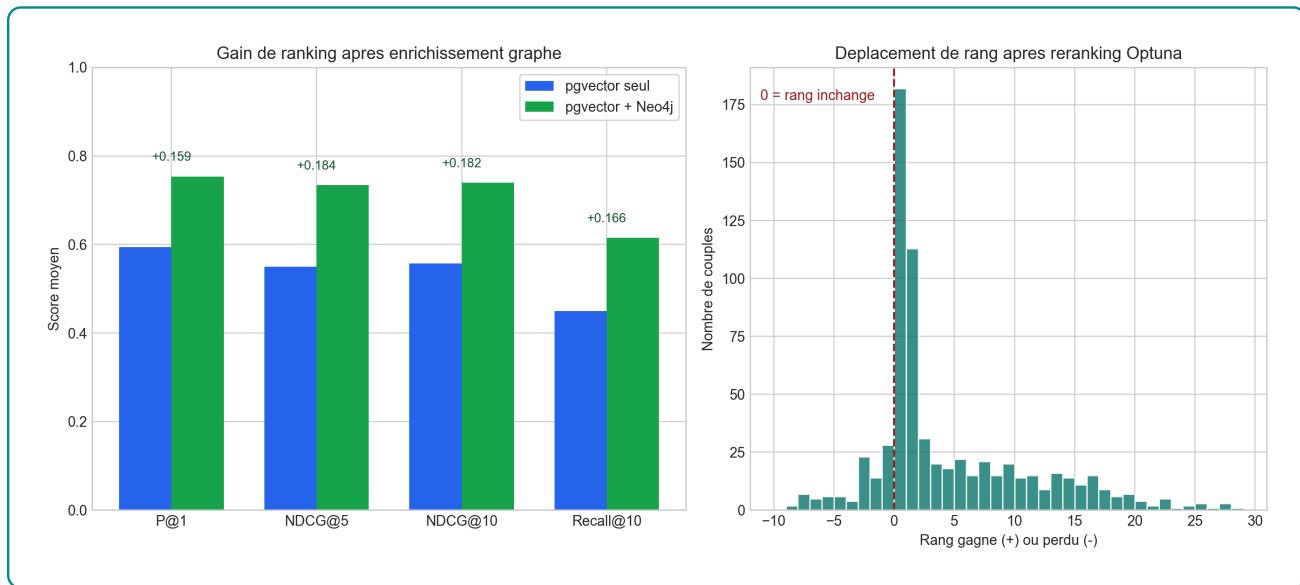
Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

La calibration Optuna a été réalisée avec un TPESampler, 500 essais et une graine fixée à 42. Lorsque l'objectif est de maximiser le NDCG@10, le meilleur jeu retenu donne tout le poids au score Neo4j : $\alpha = 0$ et $\beta = 1$. Ce résultat ne signifie pas que pgvector est inutile. Il signifie que, sur le sous-ensemble déjà récupéré par pgvector, le reclassement est mieux ordonné lorsque la compatibilité graphe domine. La bonne lecture est donc séquentielle : pgvector sert à constituer le vivier de candidats plausibles ; Neo4j sert ensuite à départager ces candidats par couverture de compétences. Pour un objectif davantage centré sur le rang exact de la première bonne offre, le critère (MRR@10 + Precision@10) / 2 retient au contraire un mélange, avec 0,6615 pour pgvector et 0,3385 pour Neo4j. Le choix final dépend donc du besoin applicatif : maximiser la qualité du top 10 ou privilégier la première recommandation.

3.5.2 Effet du reranking hybride

L'effet du reranking se mesure à deux niveaux. Le premier niveau regarde les rangs des couples candidat-offre déjà présents dans le vivier récupéré par pgvector. Le second niveau regarde les métriques de ranking lorsque l'on compare le système pgvector_only au système pgvector_plus_graph. Cette distinction est importante : Neo4j ne remplace pas le retrieval vectoriel, il intervient après lui pour réordonner les offres candidates.

La figure 3.15 montre un effet net du graphe sur l'ordre des résultats. Sur les 690 couples évalués, 59,86 % gagnent des positions après reranking, 13,77 % perdent des positions et 26,38 % restent au même rang. Le gain moyen est de 4,01 rangs, avec une médiane de 1 rang et un maximum de 28 rangs. Cette moyenne ne doit pas être interprétée comme une amélioration automatique de chaque offre : elle indique que le score Neo4j change réellement l'ordre produit par pgvector, en remontant surtout les offres dont les compétences requises sont mieux couvertes.

Figure 3.15 – Effet du reranking Neo4j sur les métriques et les déplacements de rang

Source : Nos travaux, diagnostics PostgreSQL-pgvector, à partir des embeddings

Le résumé détaillé des déplacements de rang après reranking est placé en annexe (tableau A.15).

L'ablation confirme ensuite que ces déplacements ne sont pas seulement cosmétiques. Sur 69 requêtes évaluées, l'ajout du graphe augmente la précision@1 de 0,5942 à 0,7536, le NDCG@5 de 0,5495 à 0,7339, le NDCG@10 de 0,5575 à 0,7392 et le recall@10 de 0,4491 à 0,6147. Le hit rate@10 reste identique à 0,9420 : cela signifie que les éléments pertinents sont souvent déjà présents dans le vivier pgvector, mais que Neo4j les ordonne mieux. C'est exactement le rôle attendu du reranking : ne pas refaire la recherche depuis zéro, mais transformer une liste plausible en liste mieux priorisée.

La conclusion doit rester stricte. Le reranking hybride améliore le classement dans ce protocole, mais la pertinence utilisée est un proxy construit à partir de métadonnées normalisées et non une annotation humaine indépendante. Le résultat est donc une preuve opérationnelle de cohérence interne, pas encore une preuve définitive de satisfaction recruteur ou candidat. Pour défendre le système en production, il faudra compléter cette évaluation par des jugements humains ou des retours d'usage.

Conclusion

Le chapitre a suivi le workflow réel du système. Les données nettoyées alimentent d'abord l'encodeur ; l'encodeur produit les vecteurs ; pgvector construit le vivier de retrieval ; Neo4j vérifie les relations métier ; le score hybride rerange les offres candidates. Cette chaîne évite de

confier toute la décision au LLM : la génération intervient après la récupération, l'enrichissement relationnel et le scoring. La conclusion d'implémentation est donc précise. Le système est opérationnel parce que ses briques sont raccordées : données normalisées, modèle d'embeddings spécialisé, mémoire vectorielle, graphe de connaissances et reranking hybride. Il reste cependant dépendant de la qualité des données scrapées, de la complétude des compétences et de la validité des relations Neo4j. Le chapitre suivant évalue justement ce que ces choix apportent au retrieval, au classement et à la génération de la recommandation.

Évaluation de la performance du système

Ce chapitre évalue le système implémenté au chapitre précédent. Il ne répète pas la description technique des briques ; il vérifie ce que ces briques apportent effectivement au fonctionnement du système. L'évaluation suit donc une logique en cinq niveaux : qualité du retrieval sémantique, effet du graphe sur le classement, validité fonctionnelle des requêtes GraphRAG, calibration du score hybride et contrôle de la génération finale par le critic.

Deux précautions guident l'interprétation. Premièrement, l'évaluation du classement est conduite sans génération LLM : elle mesure la qualité du retrieval et du reranking, non la qualité rédactionnelle de la réponse. Deuxièmement, la pertinence utilisée dans l'ablation est un proxy construit à partir des métadonnées normalisées ; elle permet de comparer deux variantes du même système, mais ne remplace pas une annotation humaine par des recruteurs ou des conseillers métier.

4.1 Protocole expérimental et métriques retenues

L'évaluation mobilise trois familles d'artefacts. Le benchmark du modèle d'embeddings utilise le jeu de test issu des paires contrastives offre–profil, soit 473 requêtes. L'ablation pgvector contre pgvector enrichi par Neo4j utilise un tirage de 80 candidats avec graine fixée à 42 ; 69 candidats sont effectivement évalués après contrôles d'exécution. Enfin, la calibration du critic utilise un jeu contrastif de 102 évaluations construit à partir de 51 cas réels et de leurs contextes mélangés.

Les métriques de retrieval retenues sont classiques en recherche d'information : Recall@K, Precision@K, NDCG@K, MRR@K et HitRate@K. Le Recall@K mesure si les éléments pertinents sont récupérés dans les premiers résultats. Le NDCG@K évalue aussi leur ordre. Le MRR@K met l'accent sur le rang de la première bonne recommandation. Ces métriques sont complémentaires : un système peut retrouver une bonne offre dans le top 10 tout en la plaçant trop bas pour être utile à l'utilisateur.

Pour l'ablation, les deux configurations reçoivent le même pool initial de 30 offres produit par pgvector. La configuration `pgvector` seul conserve l'ordre vectoriel. La configuration `pgvector + graphe` enrichit ce même pool avec Neo4j, puis le rerange à partir des relations candidat–offre–compétence. Cette construction isole l'effet du graphe : la différence observée

ne vient pas d'un corpus différent, mais de l'information relationnelle ajoutée au classement.

4.2 Retrieval sémantique et apport mesuré du graphe

4.2.1 Modèle d'embeddings spécialisé

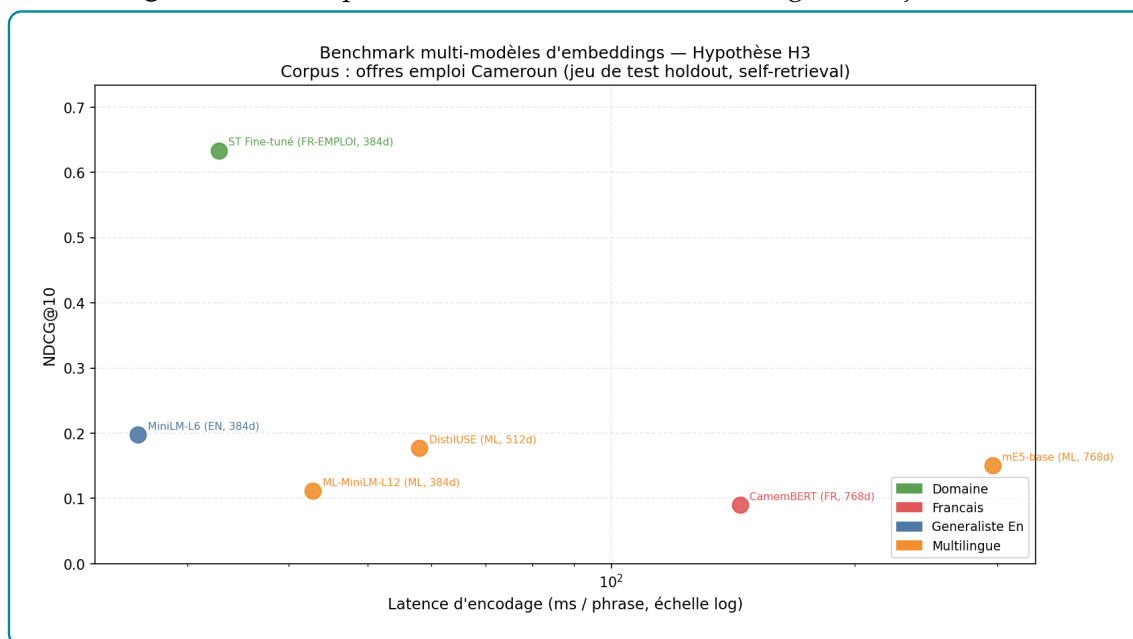
Le premier test vérifie si le fine-tuning du SentenceTransformer améliore réellement le retrieval. Le tableau 4.1 compare le modèle spécialisé à plusieurs modèles généralistes ou multilingues sur les mêmes 473 requêtes de test.

Table 4.1 – Benchmark des modèles d'embeddings sur 473 requêtes de test

Modèle	NDCG@10	MRR@10	Recall@10	Latence ms/phrased
ST fine-tuné emploi-compétences	0.6336	0.5653	0.8520	32.77
MiniLM-L6 généraliste	0.1980	0.1603	0.3192	26.03
DistilUSE multilingue	0.1773	0.1434	0.2896	57.96
mE5-base multilingue	0.1505	0.1274	0.2262	295.76
ML-MiniLM-L12	0.1115	0.0922	0.1734	42.82
CamemBERT sentence	0.0904	0.0771	0.1332	144.15

Source : Nos travaux, évaluation SentenceTransformer sur le jeu de test offre-profil.

Le modèle fine-tuné atteint un Recall@10 de 0.8520, contre 0.3192 pour MiniLM-L6 généraliste. L'écart est important : le modèle spécialisé retrouve beaucoup plus souvent l'élément attendu dans les dix premiers résultats. Le NDCG@10 passe de 0.1980 à 0.6336, ce qui montre aussi un meilleur ordre des résultats pertinents. La latence moyenne reste contenue à 32.77 ms par phrase, ce qui reste compatible avec une recherche interactive locale.

Figure 4.1 – Comparaison des modèles d’embeddings sur le jeu de test

Source : Nos travaux, évaluation SentenceTransformer sur le jeu de test offre–profil.

La figure 4.1 confirme que le gain ne vient pas seulement d’une métrique isolée. Le modèle fine-tuné domine sur la qualité de classement, le rappel et le compromis performance–latence. Ce résultat justifie son utilisation comme encodeur principal de pgvector.

4.2.2 Ablation pgvector seul contre pgvector enrichi par Neo4j

Le deuxième test mesure l’apport du graphe. Le même pool initial de 30 offres est utilisé dans les deux configurations ; seul le reranking par les relations Neo4j change. Le tableau 4.2 présente les résultats obtenus sur les 69 candidats effectivement évalués.

Table 4.2 – Comparaison retrieval : pgvector seul versus pgvector + graphe

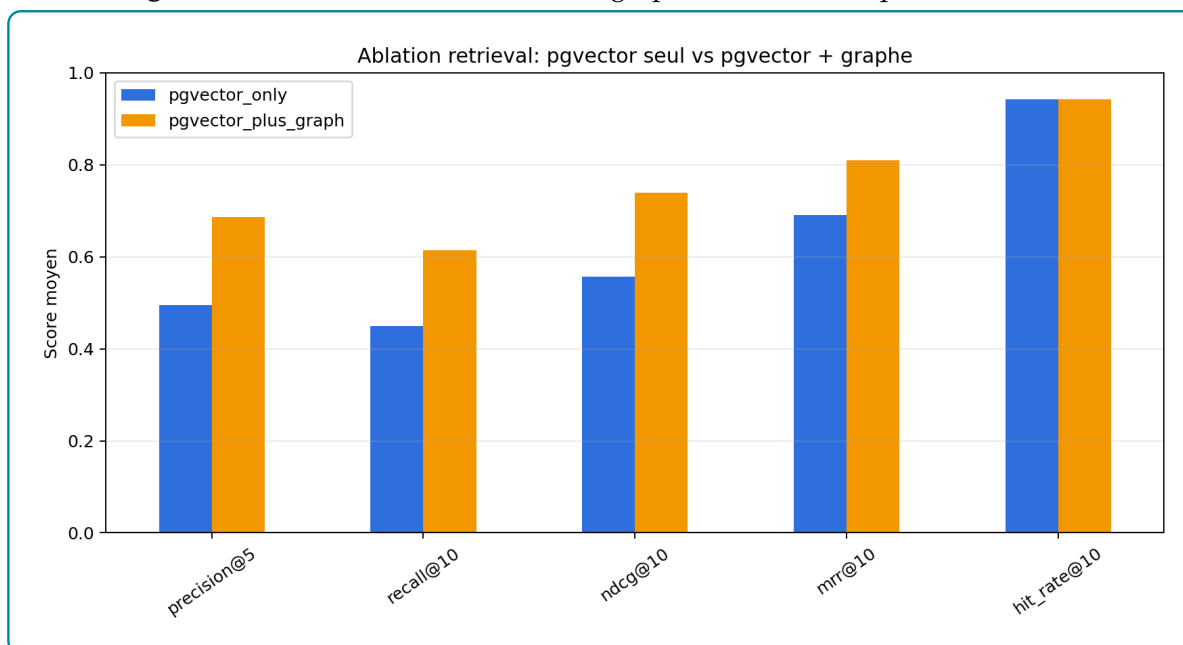
Configuration	Precision@5	Recall@10	NDCG@10	MRR@10	HitRate@10
pgvector seul	0.4957	0.4491	0.5575	0.6908	0.9420
pgvector + graphe	0.6870	0.6147	0.7392	0.8093	0.9420
Différence	+0.1913	+0.1656	+0.1817	+0.1185	0.0000

Source : Nos travaux, ablation du moteur de recommandation sur 69 candidats évalués.

L’ajout de Neo4j améliore Precision@5, Recall@10, NDCG@10 et MRR@10. Le Recall@10 passe de 0.4491 à 0.6147, soit un gain de 0.1656. Le NDCG@10 passe de 0.5575 à 0.7392, ce qui montre que le graphe ne se contente pas de retrouver plus d’offres jugées pertinentes : il

les place aussi dans un meilleur ordre. Le HitRate@10 reste stable à 0.9420 ; le pool vectoriel contient donc souvent au moins une bonne offre, mais Neo4j améliore son rang.

Figure 4.2 – Effet de l’enrichissement graphe sur les métriques de retrieval



Source : Nos travaux, ablation du moteur de recommandation sur 69 candidats évalués.

La lecture correcte est séquentielle. pgvector reste le premier étage : il constitue rapidement un vivier d’offres plausibles. Neo4j intervient ensuite pour départager ce vivier à partir des relations `POSSEDE`, `REQUIERT`, `A_NIVEAU` et `REQUIERT_NIVEAU`. L’amélioration mesurée vient donc du reranking relationnel, pas d’un remplacement de la recherche vectorielle.

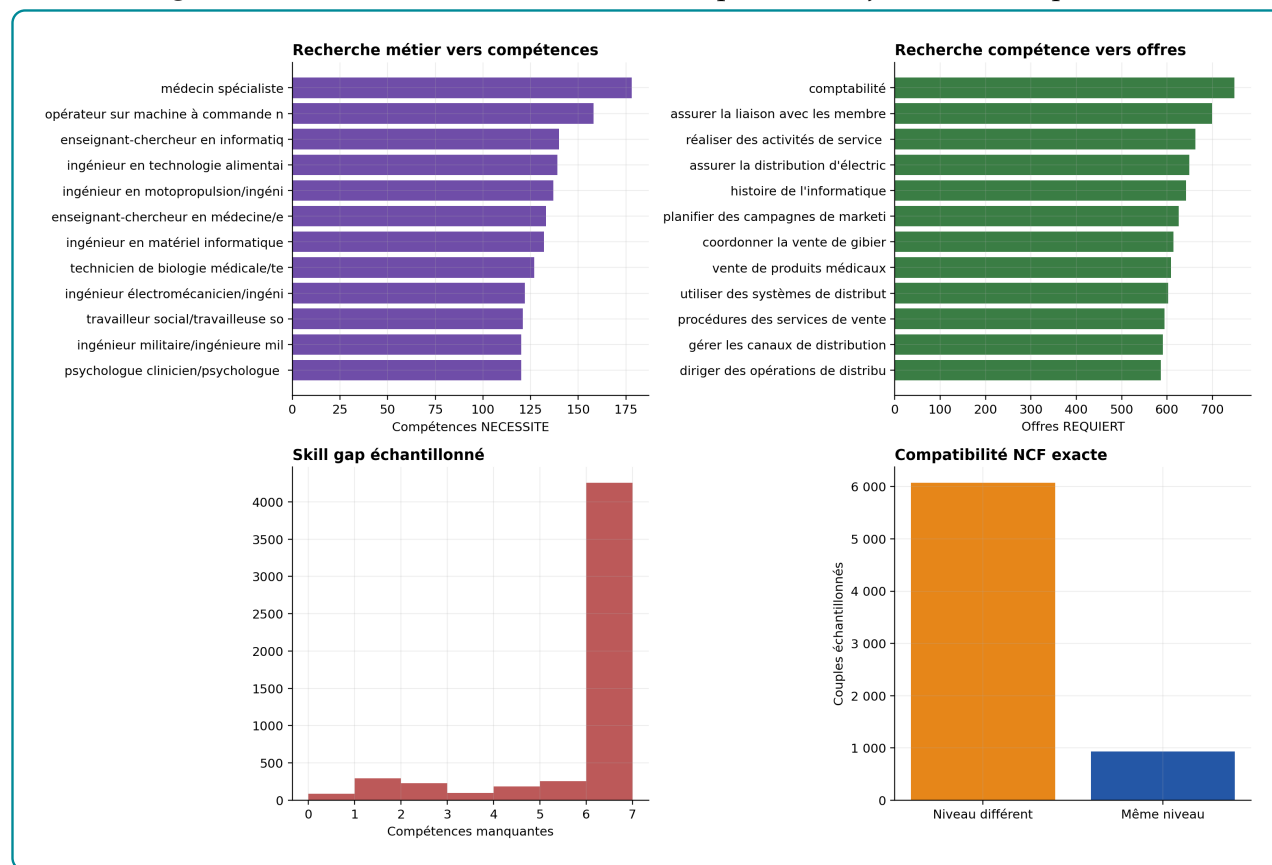
4.3 Validation fonctionnelle du GraphRAG

L’ablation précédente mesure l’effet du graphe sur le classement, mais elle ne suffit pas à vérifier que le GraphRAG fonctionne correctement dans les usages métier. Cette section évalue donc la chaîne fonctionnelle : partir d’une demande utilisateur, activer les chemins Neo4j utiles, récupérer les entités reliées, puis transformer ces relations en contexte exploitable par la génération. La figure 4.3 synthétise ces contrôles selon cinq angles complémentaires : recherche métier vers compétences, recherche compétence vers offres, calcul de *skill gap*, compatibilité NCF et comparaison RAG contre GraphRAG.

Le rôle de cette figure n’est pas seulement illustratif. Elle sert de tableau de bord fonctionnel : chaque barre correspond à une capacité attendue du système. Si une barre est faible, le problème ne vient pas nécessairement du LLM ; il peut venir d’une relation manquante, d’un référentiel incomplet ou d’un chemin Neo4j trop pauvre. La lecture des sous-sections suivantes

reprend donc les mêmes familles de tests que la figure afin de relier les résultats chiffrés au fonctionnement concret du GraphRAG.

Figure 4.3 – Évaluation fonctionnelle des requêtes Neo4j utiles au GraphRAG



Source : Nos travaux, diagnostics Neo4j et GraphRAG sur les scénarios fonctionnels du système.

4.3.1 Requêtes relationnelles métier, compétence et offre

La première partie de la figure 4.3 vérifie la navigation depuis un métier vers les compétences attendues. Le graphe part d'un noeud **Métier** et suit la relation **NECESSITE** pour retrouver les compétences associées. Les métiers les plus riches sont très connectés : « médecin spécialiste » est relié à 178 compétences, « opérateur sur machine à commande numérique » à 158 et « enseignant-chercheur en informatique » à 140. Cette richesse est utile pour l'orientation, car elle donne au GraphRAG un contexte métier large. Elle ne doit cependant pas être lue comme une liste minimale d'exigences pour chaque offre : une offre précise ne mobilise qu'une partie de ce contexte.

La deuxième partie de la même figure contrôle le sens inverse : à partir d'une compétence, le graphe retrouve les offres qui la requièrent. La compétence « comptabilité » est reliée à 748 offres, « réaliser des activités de service après-vente » à 662 offres et « planifier des campagnes

de marketing sur les médias sociaux » à 626 offres. Ce résultat confirme que Neo4j sert bien de moteur de recherche relationnelle pour compléter pgvector. Il révèle aussi une limite importante : certaines compétences fréquentes deviennent des hubs peu discriminants. Dans le score final, elles doivent donc être pondérées avec prudence, sinon le système risque de favoriser des correspondances trop générales.

4.3.2 Skill gap, niveau NCF et comparaison GraphRAG

La troisième lecture de la figure 4.3 porte sur le *skill gap*. Le calcul compare les compétences **REQUIERT** par l'offre aux compétences **POSSEDE** par le candidat. Sur l'échantillon fonctionnel, le déficit moyen est de 5,30 compétences et la médiane vaut 6. Ce résultat est utile parce qu'il montre que le GraphRAG ne se limite pas à dire qu'un candidat correspond ou ne correspond pas. Il identifie les compétences manquantes et peut transformer un mauvais appariement brut en recommandation explicable de montée en compétences.

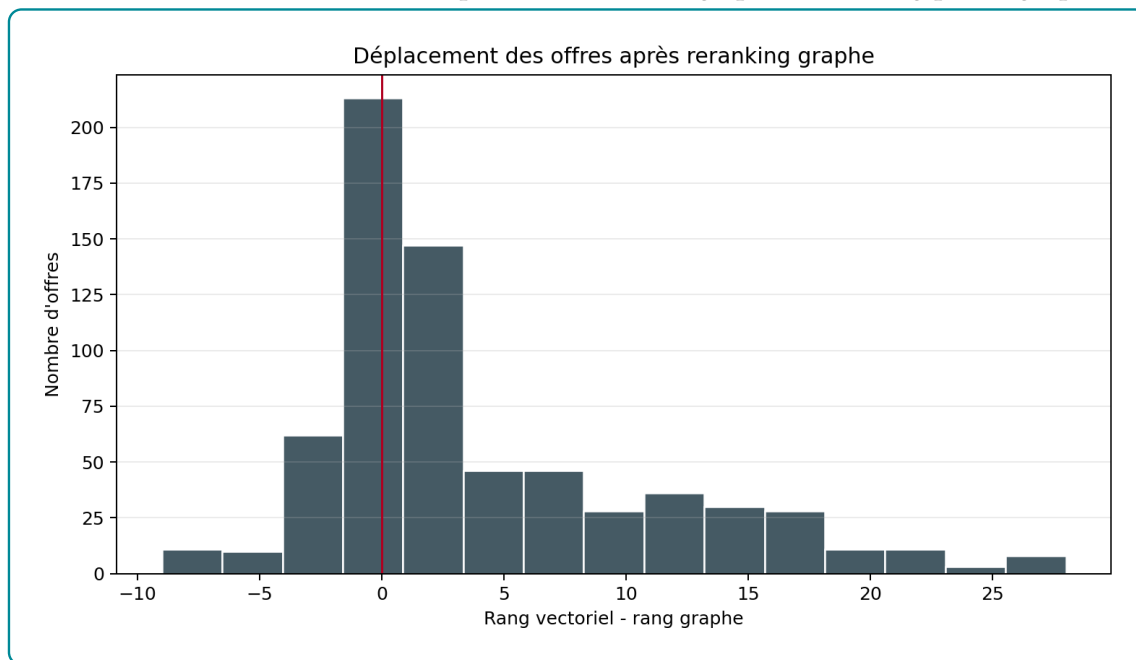
La quatrième lecture concerne la compatibilité NCF. Elle vérifie si le niveau du candidat correspond au niveau requis par l'offre. Dans l'échantillon évalué, 13,29 % des couples partagent exactement le même niveau. Ce taux confirme que le niveau de formation apporte un signal structurant, mais il ne doit pas devenir un couperet automatique. Selon l'expérience et les compétences détenues, un écart de niveau peut rester acceptable ; le graphe doit donc éclairer la décision, pas la remplacer.

Enfin, la dernière partie de la figure 4.3 met en relation ces contrôles fonctionnels avec la comparaison RAG contre GraphRAG. Le RAG vectoriel produit le pool initial, tandis que le GraphRAG ajoute les relations Neo4j pour reranger les offres et fournir des explications. Les résultats sont nets : le NDCG@10 passe de 0.5575 à 0.7392, le Recall@10 de 0.4491 à 0.6147 et le MRR@10 de 0.6908 à 0.8093. Le GraphRAG améliore donc l'ordre des recommandations et pas seulement leur justification textuelle.

4.4 Reranking hybride et calibration du score

4.4.1 Déplacements de rang après reranking par le graphe

Le reranking graphe modifie l'ordre initial produit par pgvector. Une offre peut remonter si elle présente une meilleure couverture de compétences, un meilleur alignement sectoriel ou une meilleure compatibilité de niveau. Inversement, une offre textuellement proche peut être rétrogradée si ses relations métier sont faibles ou mal renseignées.

Figure 4.4 – Distribution des déplacements de rang après reranking par le graphe

Source : Nos travaux, ablation du moteur hybride et analyse des déplacements de rang.

Cette redistribution confirme que Neo4j agit réellement sur le classement. Le graphe ajoute de l'information, mais il ajoute aussi une dépendance à la qualité des relations. Si les compétences sont mal extraites ou si une offre est peu renseignée, le score graphe peut pénaliser ou favoriser des résultats pour de mauvaises raisons. Le résultat doit donc être lu comme une preuve de cohérence interne du système, pas comme une validation métier définitive.

4.4.2 Pondérations optimales avec Optuna

Afin de ne pas conserver des pondérations arbitraires, les poids du score hybride ont été optimisés avec Optuna sur 690 couples candidat-offre issus des 69 candidats évalués. Trois fonctions objectif sont comparées : NDCG@10, moyenne NDCG@10–Recall@10–Precision@10 et moyenne MRR@10–Precision@10. La pertinence utilisée est `proxy_relevance`, un label faible construit à partir des métadonnées normalisées.

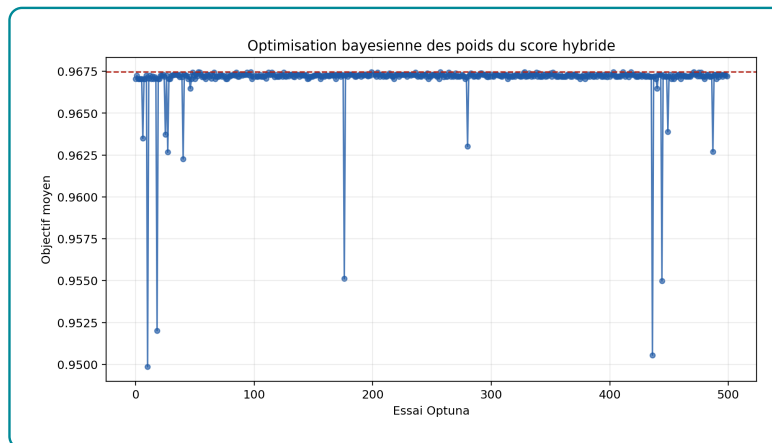
Table 4.3 – Pondérations retenues selon la fonction objectif du score hybride

Fonction objectif	w_{sem}	w_{Neo4j}	NDCG@10	Precision@10	Recall@10	MRR@10
NDCG@10	0.0000	1.0000	0.9697	0.5000	1.0000	0.8109
NDCG@10 + Recall@10 + Precision@10	0.0000	1.0000	0.9697	0.5000	1.0000	0.8109
MRR@10 + Precision@10	0.6615	0.3385	0.9674	0.5000	1.0000	0.8183

Source : Nos travaux, optimisation Optuna des poids du score hybride sur les sorties d'ablation.

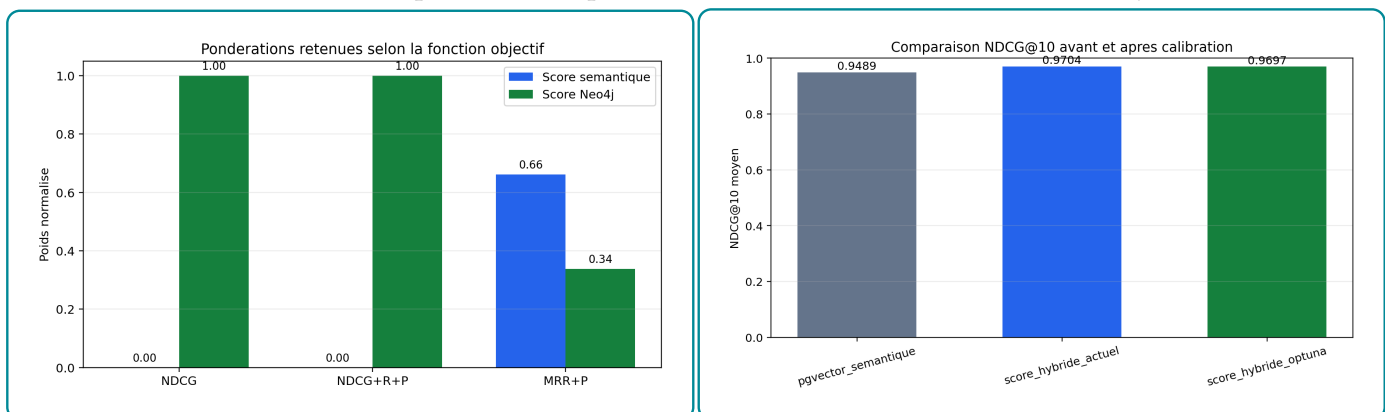
Le résultat est clair mais doit être bien défendu. Lorsque l'objectif privilégie l'ordre global du top 10, Optuna retient un score Neo4j pur : $w_{sem} = 0$ et $w_{Neo4j} = 1$. Cela ne signifie pas que pgvector est inutile ; cela signifie qu'une fois le pool construit par pgvector, le meilleur signal de reclassement dans ce protocole est la couverture relationnelle calculée dans Neo4j. Lorsque l'objectif privilégie davantage la première bonne recommandation, le compromis devient hybride avec $w_{sem} = 0,6615$ et $w_{Neo4j} = 0,3385$.

Figure 4.5 – Évolution de l'objectif NDCG@10 au cours des essais Optuna



Source : Nos travaux, optimisation Optuna des poids du score hybride.

Figure 4.6 – Comparaison des pondérations retenues selon la fonction objectif



Source : Nos travaux, optimisation Optuna des poids du score hybride.

La conclusion opérationnelle est la suivante : pgvector doit être conservé pour le rappel initial, tandis que Neo4j doit peser fortement dans le classement final lorsque les relations **POSSEDE** et **REQUIERT** sont disponibles. La calibration reste cependant dépendante du proxy de pertinence ; une validation humaine est nécessaire avant d'en faire des poids définitifs.

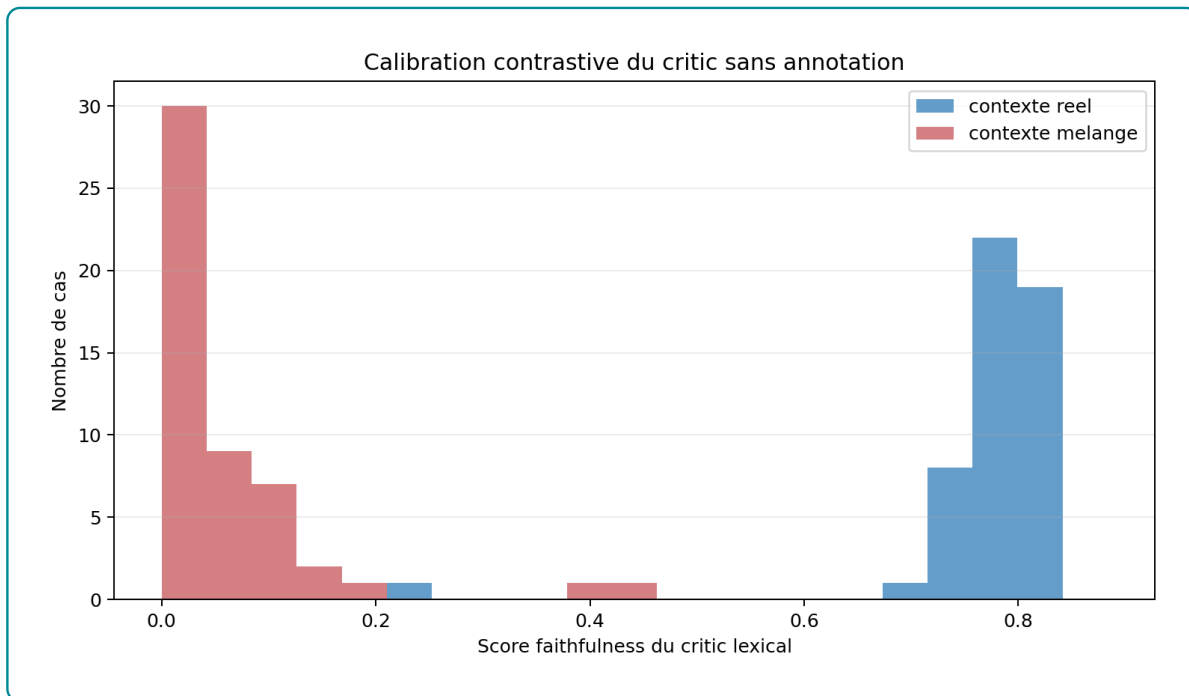
4.5 Contrôle de génération par critic

4.5.1 Calibration contrastive du seuil de fidélité

Après le retrieval et le reranking, le LLM OpenRouter formule la réponse finale. Le critic vérifie ensuite si cette réponse reste suffisamment appuyée par le contexte récupéré. Le score utilisé est un score de fidélité lexicale : il compare les mots significatifs de la réponse aux mots présents dans les preuves. Il ne juge pas la qualité métier complète de la réponse ; il vérifie seulement son ancrage dans le contexte.

Aucun jeu d'annotations humaines n'étant disponible, le seuil est calibré par contraste. Une réponse comparée à son vrai contexte doit obtenir un score plus élevé que la même réponse comparée à un contexte mélangé provenant d'une autre requête. Le protocole contient 51 cas réels évalués deux fois, soit 102 évaluations. Le score moyen vaut 0.7725 avec contexte réel contre 0.0532 avec contexte mélangé ; la médiane passe de 0.7857 à 0.0000.

Figure 4.7 – Distribution des scores du critic avec contexte réel et contexte mélangé



Source : Nos travaux, évaluation du critic à partir des réponses générées et des contextes récupérés.

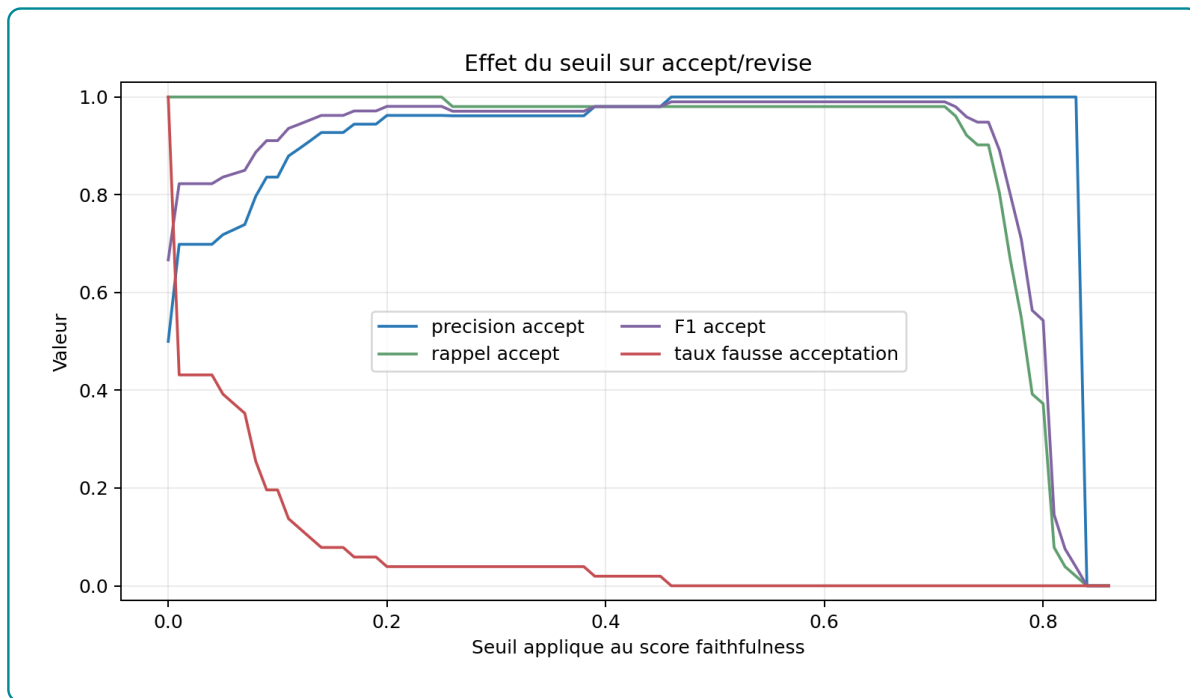
4.5.2 Interprétation des décisions accept/revise

Le seuil opérationnel retenu est 0,46. Il a été obtenu par une calibration empirique sur une grille de seuils appliquée aux 102 évaluations contrastives : 51 réponses comparées à leur vrai contexte et les mêmes 51 réponses comparées à un contexte mélangé. Pour chaque

seuil candidat, le critic transforme le score continu en décision binaire : *accept* si le score est supérieur ou égal au seuil, *revise* sinon. La grille permet ensuite de compter deux erreurs : les fausses acceptations, c'est-à-dire les contextes mélangés acceptés à tort, et les fausses révisions, c'est-à-dire les vrais contextes renvoyés inutilement en révision.

La valeur 0,46 est retenue parce qu'elle maximise le compromis F1 dans ce test tout en respectant une contrainte de prudence : ne laisser passer aucun contexte mélangé. À ce niveau, aucune réponse associée à un contexte mélangé n'est acceptée, tandis que 50 réponses sur 51 restent acceptées avec leur vrai contexte. Le choix n'est donc pas théorique ; il vient du point de la grille où le système bloque les réponses les moins ancrées sans devenir excessivement sévère sur les réponses correctement contextualisées.

Figure 4.8 – Effet du seuil du critic sur les décisions accept/revise



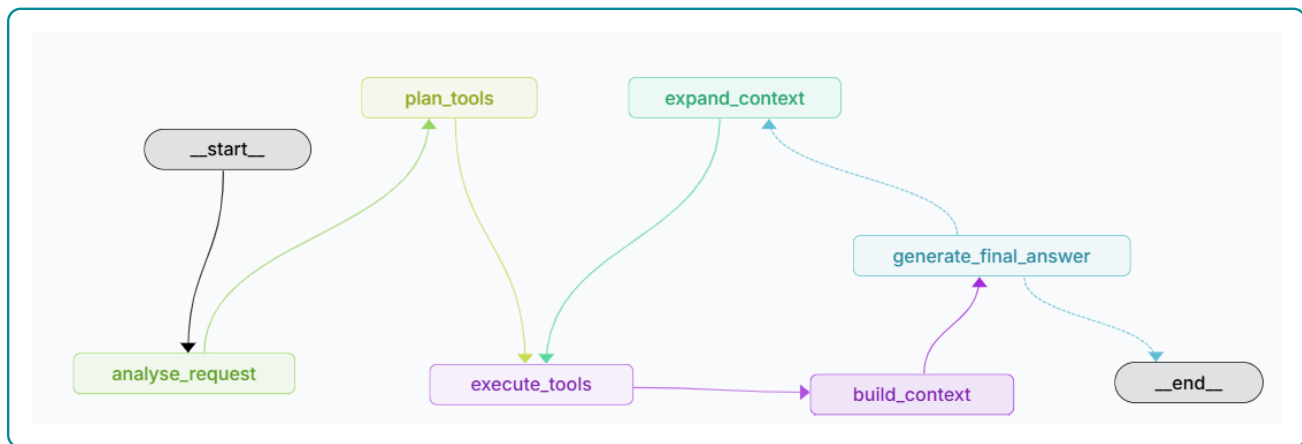
Source : Nos travaux, évaluation du critic à partir des réponses générées et des contextes récupérés.

Cette calibration ne transforme pas le critic en juge parfait. Une réponse acceptée n'est pas automatiquement correcte au sens métier ; elle est seulement suffisamment ancrée lexicalement dans les preuves récupérées. Pour défendre le système devant un jury, la formulation correcte est donc : le seuil 0,46 est un seuil opérationnel calibré empiriquement par contraste, en attendant une validation humaine de la qualité réelle des recommandations générées.

4.6 Orchestration agentique avec LangGraph

Les résultats précédents évaluent séparément le retrieval, le reranking par Neo4j et le critic. Dans l'application, ces briques ne fonctionnent pourtant pas isolément : elles sont coordonnées par un workflow agentique construit avec LangGraph, dans l'écosystème LangChain. Cette orchestration se situe à la fin du workflow global : elle reçoit la question utilisateur, choisit les outils à activer, organise les résultats récupérés, déclenche la génération finale, puis décide si la réponse doit être acceptée ou révisée.

Figure 4.9 – Workflow agentique orchestré dans LangGraph Studio



Source : Nos travaux, visualisation LangGraph Studio du workflow implémenté dans `src/08_agentic_graphrag/graph.py`.

La figure 4.9 montre que le système commence par le nœud `analyse_request`. Ce nœud identifie l'intention de la requête : diagnostic, recommandation, recherche documentaire, question relationnelle Neo4j ou demande générale. Le nœud `plan_tools` choisit ensuite les outils adaptés. Cette étape évite d'appeler toutes les bases inutilement : une question sur un métier peut mobiliser Neo4j, une question documentaire peut mobiliser pgvector sur les chunks, et une recommandation candidat-offre mobilise le moteur hybride.

Le nœud `execute_tools` exécute les outils retenus : recherche sémantique pgvector, requête Neo4j, score hybride, recherche documentaire ou diagnostic de service. Les sorties ne sont pas envoyées directement au LLM. Elles passent d'abord par `build_context`, qui transforme les résultats hétérogènes en contexte lisible : offres candidates, compétences communes, compétences manquantes, relations du graphe, fragments documentaires et métadonnées. Le nœud `generate_final_answer` produit ensuite la réponse finale à partir de ce contexte.

La boucle `generate_final_answer` → `expand_context` est le point de contrôle principal. Si le critic estime que la réponse est insuffisamment ancrée dans les preuves, le workflow ne se

contente pas d’afficher une réponse fragile : il élargit le contexte, relance les outils nécessaires, reconstruit le contexte et régénère la réponse. La décision `accept` signifie donc que le score de fidélité dépasse le seuil opérationnel calibré ; la décision `revise` signifie que le système doit chercher davantage de preuves avant de répondre.

Cette orchestration donne une cohérence applicative aux métriques du chapitre. Le `Recall@10` et le `NDCG@10` évaluent la qualité des résultats récupérés avant génération ; le reranking `Neo4j` évalue la capacité à réordonner le vivier ; le `critic` contrôle l’ancrage de la réponse finale ; `LangGraph` relie ces étapes dans une chaîne exécutable et traçable. La performance du système ne dépend donc pas seulement d’un modèle ou d’une base, mais de la coordination contrôlée entre `pgvector`, `Neo4j`, le score hybride, le `LLM` et le `critic`.

Conclusion

L’évaluation montre trois résultats principaux. Premièrement, le fine-tuning du modèle d’embeddings améliore fortement le retrieval sémantique, avec un `Recall@10` de 0.8520 et un `NDCG@10` de 0.6336. Deuxièmement, l’ajout de `Neo4j` améliore le classement des offres dans le pool `pgvector` : le `NDCG@10` passe de 0.5575 à 0.7392 et le `MRR@10` de 0.6908 à 0.8093. Troisièmement, le `critic` distingue correctement les réponses ancrées dans leur contexte des réponses confrontées à un contexte mélangé, ce qui justifie le seuil opérationnel de 0,46.

La limite principale reste l’absence d’annotation humaine indépendante. Les résultats démontrent la cohérence interne du système et l’intérêt mesurable du graphe dans le protocole actuel. Ils ne prouvent pas encore la satisfaction d’un recruteur, d’un candidat ou d’un conseiller d’orientation. Cette limite doit être assumée : le système est techniquement validé sur ses sorties internes, mais son évaluation métier finale devra passer par un jeu annoté et des tests utilisateurs.

Conclusion générale

Ce mémoire avait pour objectif de concevoir et d'évaluer un système de recommandation hybride pour le rapprochement entre offres d'emploi, profils candidats, compétences et référentiels métier. Le choix retenu n'a pas été de confier toute la décision à un modèle génératif, mais de construire une chaîne contrôlée : nettoyage des données, adaptation des embeddings au domaine emploi-compétences, indexation vectorielle avec pgvector, structuration relationnelle dans Neo4j, reranking hybride, puis génération augmentée par un workflow agentique.

Les résultats obtenus montrent que chaque brique joue un rôle distinct. Le modèle d'embeddings améliore la récupération des éléments proches dans l'espace sémantique. pgvector fournit une mémoire rapide pour retrouver les offres, compétences et fragments documentaires pertinents. Neo4j apporte une lecture relationnelle du matching : il permet d'exploiter les liens entre métiers, compétences, formations et référentiels, puis d'améliorer le classement initial par un score de graphe. Enfin, l'orchestration LangGraph organise ces traitements dans un workflow exploitable par l'application, depuis l'analyse de la requête jusqu'à la génération de la réponse finale.

L'évaluation confirme l'intérêt de cette architecture hybride. Le fine-tuning du modèle d'embeddings améliore le retrieval, l'ajout du graphe renforce le classement des offres et le critic permet de filtrer les réponses insuffisamment ancrées dans les preuves récupérées. Le seuil opérationnel de 0,46 n'est donc pas présenté comme une vérité théorique : il correspond à une calibration empirique issue des sorties contrastives du système. Cette précision est importante, car elle évite de surinterpréter le critic comme un juge métier autonome.

La principale limite du travail reste l'absence d'une validation humaine indépendante à grande échelle. Les métriques internes démontrent la cohérence technique du pipeline, mais elles ne remplacent pas l'appréciation d'un recruteur, d'un conseiller d'orientation ou d'un candidat. Les perspectives prioritaires consistent donc à constituer un jeu annoté par des experts, tester le système auprès d'utilisateurs réels, suivre les erreurs de recommandation par famille de métiers et enrichir progressivement le graphe avec des données institutionnelles mieux normalisées.

Au total, le système proposé constitue une base opérationnelle de recommandation explicable. Sa force ne réside pas seulement dans l'utilisation d'un LLM, mais dans l'articulation entre mémoire vectorielle, mémoire relationnelle, règles de ranking et contrôle de la génération. C'est cette combinaison qui rend le dispositif plus défendable qu'une approche purement textuelle ou purement générative.

Bibliographie

- [1] Francesco RICCI et al., éd. *Recommender Systems Handbook*. Springer, 2011. DOI : [10.1007/978-0-387-85820-3](#).
- [2] PGVECTOR CONTRIBUTORS. *pgvector : Open-source Vector Similarity Search for Postgres*. 2024. URL : <https://github.com/pgvector/pgvector> (visité le 04/05/2026).
- [3] Aidan HOGAN et al. "Knowledge Graphs". In : *ACM Computing Surveys* 54.4 (2021), p. 1-37. DOI : [10.1145/3447772](#).
- [4] Patrick LEWIS et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474. URL : <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [5] Dale T. MORTENSEN et Christopher A. PISSARIDES. "Job Creation and Job Destruction in the Theory of Unemployment". In : *The Review of Economic Studies* 61.3 (1994), p. 397-415. DOI : [10.2307/2297896](#).
- [6] Christopher A. PISSARIDES. *Equilibrium Unemployment Theory*. 2^e éd. Cambridge, MA : MIT Press, 2000.
- [7] George A. AKERLOF. "The Market for "Lemons" : Quality Uncertainty and the Market Mechanism". In : *The Quarterly Journal of Economics* 84.3 (1970), p. 488-500. DOI : [10.2307/1879431](#).
- [8] Michael SPENCE. "Job Market Signaling". In : *The Quarterly Journal of Economics* 87.3 (1973), p. 355-374. DOI : [10.2307/1882010](#).
- [9] Gary S. BECKER. *Human Capital : A Theoretical and Empirical Analysis, with Special Reference to Education*. New York : Columbia University Press, 1964.
- [10] David H. AUTOR. "The "Task Approach" to Labor Markets : An Overview". In : *Journal for Labour Market Research* 46.3 (2013), p. 185-199. DOI : [10.1007/s12651-013-0128-z](#).
- [11] Gediminas ADOMAVICIUS et Alexander TUZHILIN. "Toward the Next Generation of Recommender Systems : A Survey of the State-of-the-Art and Possible Extensions". In : *IEEE Transactions on Knowledge and Data Engineering* 17.6 (2005), p. 734-749. DOI : [10.1109/TKDE.2005.99](#).

- [12] Michael J. PAZZANI et Daniel BILLSUS. "Content-Based Recommendation Systems". In : *The Adaptive Web* (2007), p. 325-341. DOI : [10.1007/978-3-540-72079-9_10](https://doi.org/10.1007/978-3-540-72079-9_10).
- [13] Yehuda KOREN, Robert BELL et Chris VOLINSKY. "Matrix Factorization Techniques for Recommender Systems". In : *Computer* 42.8 (2009), p. 30-37. DOI : [10.1109/MC.2009.263](https://doi.org/10.1109/MC.2009.263).
- [14] Xiangnan HE et al. "Neural Collaborative Filtering". In : *Proceedings of the 26th International Conference on World Wide Web*. 2017, p. 173-182.
- [15] Robin BURKE. "Hybrid Recommender Systems : Survey and Experiments". In : *User Modeling and User-Adapted Interaction* 12 (2002), p. 331-370. DOI : [10.1023/A:1021240730564](https://doi.org/10.1023/A:1021240730564).
- [16] Chen YANG et al. "Modeling Two-way Selection Preference for Person-Job Fit". In : *Proceedings of the 16th ACM Conference on Recommender Systems*. 2022, p. 102-112.
- [17] A. T. WASI. "HRGraph : Leveraging LLMs for HR Data Knowledge Graphs with Information Propagation-based Job Recommendation". In : *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models*. 2024, p. 56-62.
- [18] B. YANG et Z. SHEN. "Knowledge Graph Construction and Talent Competency Prediction for Human Resource Management". In : *Alexandria Engineering Journal* 121 (2025), p. 223-235.
- [19] Ashish VASWANI et al. "Attention Is All You Need". In : *Advances in Neural Information Processing Systems*. 2017, p. 5998-6008.
- [20] Tomas MIKOLOV et al. "Distributed Representations of Words and Phrases and their Compositionality". In : *Advances in Neural Information Processing Systems*. T. 26. 2013. URL : <https://proceedings.neurips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality>.
- [21] Dzmitry BAHDANAU, Kyunghyun CHO et Yoshua BENGIO. "Neural Machine Translation by Jointly Learning to Align and Translate". In : *International Conference on Learning Representations*. 2015. URL : <https://arxiv.org/abs/1409.0473>.
- [22] Jacob DEVLIN et al. "BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding". In : *Proceedings of NAACL-HLT*. 2019, p. 4171-4186. URL : <https://aclanthology.org/N19-1423/>.
- [23] Nils REIMERS et Iryna GUREVYCH. "Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks". In : *Proceedings of EMNLP-IJCNLP*. 2019, p. 3982-3992. URL : <https://aclanthology.org/D19-1410/>.

- [24] Tom B. BROWN et al. "Language Models are Few-Shot Learners". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 1877-1901. URL : <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [25] Colin RAFFEL et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In : *Journal of Machine Learning Research* 21.140 (2020), p. 1-67. URL : <https://www.jmlr.org/papers/v21/20-074.html>.
- [26] Long OUYANG et al. "Training Language Models to Follow Instructions with Human Feedback". In : *Advances in Neural Information Processing Systems* 35 (2022), p. 27730-27744. URL : https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- [27] Nandan THAKUR et al. "BEIR : A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models". In : *Advances in Neural Information Processing Systems*. T. 34. 2021, p. 28974-28989. URL : <https://arxiv.org/abs/2104.08663>.
- [28] Niklas MUENNIGHOFF et al. "MTEB : Massive Text Embedding Benchmark". In : *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, p. 2014-2037. URL : <https://aclanthology.org/2023.eacl-main.148/>.
- [29] Yu A. MALKOV et D. A. YASHUNIN. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), p. 824-836. DOI : [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [30] Jeff JOHNSON, Matthijs DOUZE et Hervé JÉGOU. "Billion-scale Similarity Search with GPUs". In : *IEEE Transactions on Big Data* 7.3 (2019), p. 535-547. DOI : [10.1109/TBDATA.2019.2921572](https://doi.org/10.1109/TBDATA.2019.2921572).
- [31] Thomas N. KIPF et Max WELLING. "Semi-Supervised Classification with Graph Convolutional Networks". In : *International Conference on Learning Representations*. 2017.
- [32] Petar VELIČKOVIĆ et al. "Graph Attention Networks". In : *International Conference on Learning Representations*. 2018.
- [33] Xiangnan HE et al. "LightGCN : Simplifying and Powering Graph Convolution Network for Recommendation". In : *Proceedings of SIGIR*. 2020, p. 639-648.
- [34] Yunfan GAO et al. *Retrieval-Augmented Generation for Large Language Models : A Survey*. 2023. arXiv : [2312.10997](https://arxiv.org/abs/2312.10997) [cs.CL]. URL : <https://arxiv.org/abs/2312.10997>.

- [35] Shunyu YAO et al. "ReAct : Synergizing Reasoning and Acting in Language Models". In : *International Conference on Learning Representations*. 2023. URL : https://openreview.net/forum?id=WE_vluYUL-X.
- [36] Akari ASAI et al. "Self-RAG : Learning to Retrieve, Generate, and Critique through Self-Reflection". In : *International Conference on Learning Representations*. 2024. URL : <https://openreview.net/forum?id=hSyW5go0v8>.
- [37] Shahul Es et al. "RAGAS : Automated Evaluation of Retrieval Augmented Generation". In : *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics : System Demonstrations*. 2024, p. 150-158. URL : <https://aclanthology.org/2024.eacl-demo.16/>.
- [38] Yang LIU et al. "G-Eval : NLG Evaluation using GPT-4 with Better Human Alignment". In : *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, p. 2511-2522. URL : <https://aclanthology.org/2023.emnlp-main.153/>.
- [39] LANGCHAIN. *LangGraph Documentation*. Documentation officielle du framework LangGraph. 2026. URL : <https://langchain-ai.github.io/langgraph/> (visit  le 06/06/2026).
- [40] Takuya AKIBA et al. "Optuna : A Next-generation Hyperparameter Optimization Framework". In : *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, p. 2623-2631. DOI : [10.1145/3292500.3330701](https://doi.org/10.1145/3292500.3330701).
- [41] William L. HAMILTON. *Graph Representation Learning*. Morgan & Claypool Publishers, 2020. DOI : [10.2200/S01045ED1V01Y202009AIM046](https://doi.org/10.2200/S01045ED1V01Y202009AIM046).

Annexes

Table A.1 – Labels de nœuds et relations structurantes du graphe

Bloc	Labels de nœuds	Relations principales et interprétation
Données d'emploi	OffreEmploi, Candidat, Employeur, Secteur, Localisation	PUBLIEE_PAR relie une offre à son employeur; DANS_SECTEUR la rattache à un secteur; LOCALISEE_A indique la ville ou zone d'exercice; A_NIVEAU relie le candidat à son niveau de formation.
Compétences et métiers	Compétence, Métier, GroupeCompétences, GroupeISCO	NECESSITE relie un métier aux compétences attendues; PARTIE_DE rattache une compétence à son groupe; CLASSIFIE_DANS relie un métier à un groupe ISCO; PLUS_LARGE_QUE représente les relations hiérarchiques entre compétences.
Référentiel MEPC	GrandGroupeMEPC, SousGroupeMEPC, GroupeBaseMEPC	CONTIENT représente la hiérarchie interne de la nomenclature; ALIGNE_AVEC rapproche les groupes MEPC des groupes ISCO; CORRESPOND_MEPC rattache un métier ESCO à un groupe de base MEPC.
Référentiel NCF	NiveauFormationNCF, GrandDomaineNCF, DomaineSpécialiséNCF, DomaineDétailléNCF	CONTIENT représente la hiérarchie formation; REQUIERT_NIVEAU_NCF relie une offre au niveau demandé; A_FORMATION rattache un candidat à son domaine de formation; PREPARE_POUR relie un domaine de formation aux métiers visés.
Recommandation	OffreEmploi, Candidat, Compétence, Métier, DomaineDétailléNCF	Les chemins candidat-compétence-offre permettent d'identifier les compétences acquises et manquantes; les chemins offre-niveau-formation servent à contrôler la compatibilité de qualification; les chemins formation-métier soutiennent la proposition de montée en compétences.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.2 – Volume d’entités indexées dans la base vectorielle pgvector

Type d’entité	Nombre	Source principale
OFFRE_EMPLOI	15 722	Scraping KmerAI
COMPETENCE	13 939	Référentiel ESCO v1.1
METIER	3 039	Référentiel ESCO v1.1
CANDIDAT	1 105	Base locale de profils demandeurs
GROUPE_BASE_MEPC	209	INS Cameroun - MEPC 2013
DOMAINE_DETAILLE_NCF	201	INS Cameroun - NCF 2017
Total	34 215	

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.3 – Pipeline général implémenté : étapes, entrées et sorties

Ordre	Étape	Entrées principales	Sorties concrètes
1	ETL	Données brutes d’offres, candidats et référentiels	Fichiers normalisés, champs <code>text_to_embed</code> , paires <code>pairs_train/val/test.jsonl</code> .
2	Fine-tuning	Paires offre–description et modèle <code>all-MiniLM-L6-v2</code>	Modèle sauvegardé dans <code>models/st_finetuned/final</code> et métriques d’entraînement.
3	Indexation vectorielle	Entités nettoyées et encodeur fine-tuné	Table <code>embeddings</code> , table <code>doc_chunks</code> , index HNSW et index textuels PostgreSQL.
4	Graphe de connaissances	Offres, candidats, compétences, métiers, niveaux et référentiels	Noeuds Neo4j, relations métier-compétence-offre-candidat, contraintes et chemins explicatifs.
5	GraphRAG hybride	Résultats pgvector, chemins Neo4j et règles métier	Contexte candidat–offre, score hybride, analyse de <i>skill gap</i> , critic et Text2Cypher.
6	Orchestration LangGraph	Question utilisateur et registre d’outils	État partagé, routage, exécution des outils, génération finale et traces du workflow.
7	Exposition applicative	Workflow agentique et services locaux	API FastAPI, CLI de test et interface Streamlit.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.4 – Correspondance entre objectifs spécifiques et choix méthodologiques

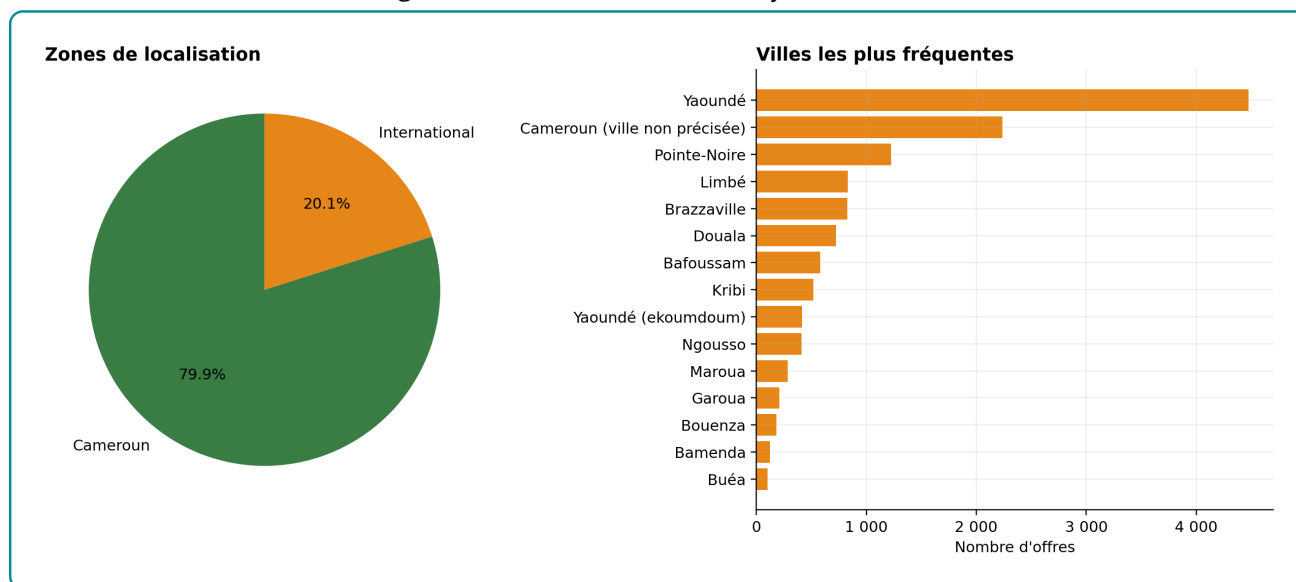
Objectif spécifique	Choix méthodologique retenu
Normaliser les offres, candidats et référentiels	Construction d'une couche de données standardisée : identifiants stables, champs nettoyés, niveaux de formation, secteurs, compétences et textes d'embedding.
Adapter la recherche sémantique au domaine emploi-compétences	Fine-tuning contrastif d'un modèle SentenceTransformer, puis indexation des embeddings dans PostgreSQL avec pgvector pour le retrieval top-K [23, 2].
Représenter les relations entre métiers, compétences et profils	Construction d'un graphe de connaissances Neo4j reliant candidats, offres, compétences, métiers, secteurs, localisations et référentiels [3, 41].
Produire une recommandation explicable	Score hybride combinant similarité vectorielle, couverture de compétences, compatibilité de niveau et alignement sectoriel, complété par une analyse de <i>skill gap</i> .
Orchestrer la réponse utilisateur	Workflow LangGraph routant la requête vers les outils adaptés : pgvector, Neo4j, recherche documentaire, Text2Cypher, critic et génération finale.
Evaluer la qualité du système	Protocole séparant l'évaluation du retrieval, du graphe, du classement et de la réponse générée.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.5 – Volumes réellement exploités par les modules de recommandation

Objet exploité	Volume	Utilité pour le système
Offres normalisées	13 957	Base principale à recommander, indexée dans pgvector et reliée au graphe.
Candidats normalisés	41 298	Vivier à partir duquel sont calculés les rapprochements offre–profil.
Compétences indexées	13 939	Signal de matching, de filtrage et de justification du <i>skill gap</i> .
Secteurs nettoyés	1 039	Signal de contexte pour limiter les rapprochements absurdes entre domaines trop éloignés.
Métiers indexés	3 039	Niveau d’agrégation utile pour stabiliser les recommandations au-delà des intitulés d’offres.
Groupes métiers MEPC	209	Référentiel externe pour structurer les proximités métier.
Domaines détaillés NCF	201	Référentiel de formation utilisé pour contextualiser les niveaux et domaines.
Localisations représentées dans Neo4j	312	Signal géographique pour filtrage souple et explication des résultats.

Source : Nos travaux, diagnostics PostgreSQL

Figure A.1 – Localisation nettoyée des offres

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.6 – Synthèse des traitements de nettoyage utiles au moteur de recommandation

Traitement	Décision implémentée	Effet attendu
Casse et formats	Harmonisation des libellés pour réduire les variantes d'écriture.	Moins de fragmentation dans les compétences, secteurs, villes et sources.
Doublons	Validation uniquement si toute la ligne est identique sur toutes les variables.	Suppression prudente des répétitions exactes, sans effacer des offres proches mais distinctes.
Localisation	Séparation Cameroun, international et non précisée; conservation des localités fines disponibles.	Filtrage géographique possible sans perte des offres incomplètes.
Champs vides	Modalités explicites lorsque le champ reste exploitable comme absence d'information.	Le système peut pénaliser ou contextualiser le manque d'information au lieu d'ignorer l'offre.
Descriptions faibles	Enrichissement de la représentation par métadonnées et compétences.	Robustesse accrue pour les sources où le détail de l'offre est absent ou court.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.7 – Comparaison directe entre le baseline et le modèle fine-tuné

Métrique test	Baseline	Fine-tuné	Gain
NDCG@10	0,1681	0,6232	+0,4552
MRR@10	0,1410	0,5874	+0,4463
Recall@1	0,0986	0,5268	+0,4281
Recall@5	0,1983	0,6632	+0,4648
Recall@10	0,2560	0,7398	+0,4837
Precision@1	0,0986	0,5268	+0,4281

Source : Nos travaux, traitements Python et SentenceTransformer

Table A.8 – Diagnostic statistique du chunking documentaire

Indicateur	Valeur observée
Documents sources indexés	3
Chunks documentaires indexés	142
Stratégie réellement utilisée	100 % structurelle
Taille moyenne	187,1 tokens
Taille médiane	116,0 tokens
Écart-type	169,6 tokens
Minimum / maximum	6 / 492 tokens
Coefficient de variation	0,91
Chunks avec embedding	142 / 142
Chunks synchronisés avec Neo4j	142 / 142

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.9 – Couverture documentaire et duplication des chunks

Source	Couverture approximative	Chunks
NCF 2017	25,9 %	38
MEPC 2013	23,6 %	61
Référentiel des diplômes	79,3 %	43
Taux de duplication cosinus > 0,95	0,0 %	142
Similarité maximale hors diagonale	0,921	–

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.10 – Cohérence sémantique intra-chunk

Source	Moyenne	Médiane	Intervalle observé
MEPC 2013	0,307	0,288	0,191–0,502
NCF 2017	0,336	0,321	0,259–0,474
Référentiel des diplômes	0,268	0,260	0,066–0,457

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.11 – Indicateurs techniques pgvector

Indicateur	Valeur observée
Embeddings d'entités structurées	72 643
Chunks documentaires indexés	142
Dimension des vecteurs	384
Modèle d'encodage	all-MiniLM-L6-v2-ft-offres-cm
Index PostgreSQL sur embeddings et doc_chunks	19
Taille totale de la table embeddings	259,05 Mo
Taille de l'index emb_hnsw	81,30 Mo
Taille totale de doc_chunks	1,51 Mo
Latence moyenne top-10 HNSW sur 25 requêtes	34,31 ms
Latence médiane top-10 HNSW sur 25 requêtes	30,89 ms
Latence p90 top-10 HNSW sur 25 requêtes	64,41 ms

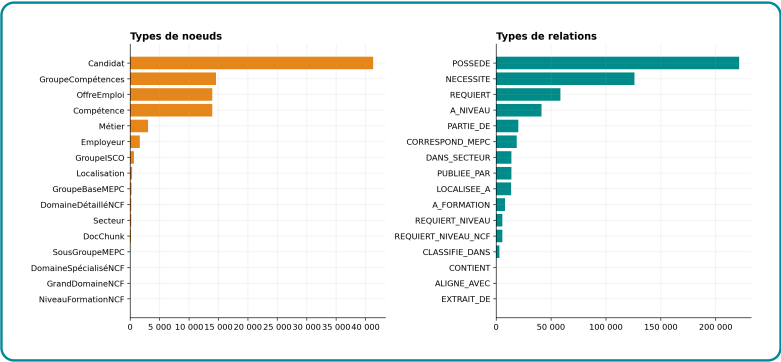
Source : Nos travaux, diagnostics PostgreSQL

Table A.12 – Index pgvector et index complémentaires

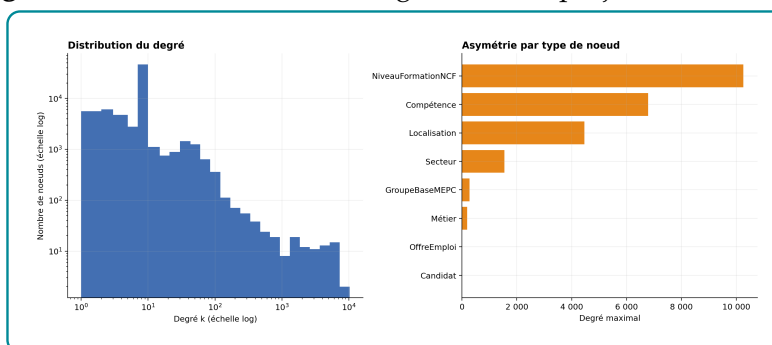
Famille d'index	Index observés	Usage dans le retrieval
Vectoriel HNSW	emb_hnsw, doc_chunks_hnsw vector_cosine_ops.	Recherche approximative des plus proches voisins avec en similarité cosinus.
B-tree métier	emb_offre_idx, emb_candidat_idx, emb_competence_idx, emb_metier_idx.	Filtrage par type d'entité et accès rapide aux objets métiers.
Synchronisation graphe	emb_neo4j_idx.	Passage d'un résultat vectoriel vers le noeud Neo4j correspondant.
Modèle et unicité	emb_model_idx, contrainte unique entity_kind, entity_id, model_id.	Éviter les doublons d'embeddings et gérer les futures versions de modèles.
Recherche lexicale	emb_fulltext_fr_idx, doc_chunks_fulltext_fr_idx	Complément lexical français pour requêtes par mots exacts, acronymes ou intitulés rares.
Référentiels	doc_chunks_ncf_idx, doc_chunks_mepc_idx, doc_chunks_source_idx.	Recherche ciblée dans NCF, MEPC et référentiel des diplômes.

Source : Nos travaux, diagnostics PostgreSQL

Figure A.2 – Composition du graphe par types de noeuds et relations



Source : Nos travaux, diagnostics Neo4j.

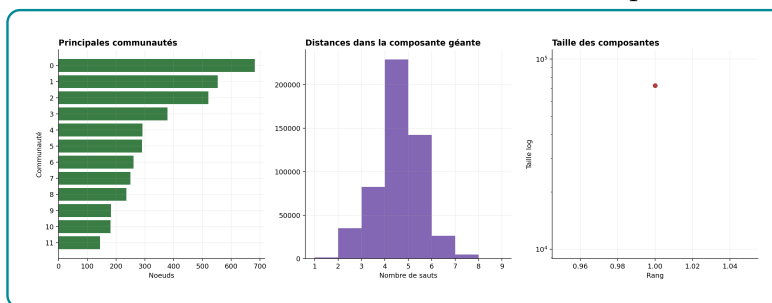
Figure A.3 – Distribution des degrés dans la projection matching

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

Table A.13 – Hétérogénéité des degrés par famille de noeuds

Label	Moyenne	Médiane	p90	Max
NiveauFormationNCF	5 812,56	7 073	10 672,4	11 050
GroupeBaseMEPC	90,22	73	201	276
Secteur	72,69	5,5	151,6	1 548
Métier	48,61	44	77	186
Localisation	43,96	1	8	4 467
Compétence	30,57	5	28	6 789
OffreEmploi	7,97	9	11	11
Candidat	6,56	7	8	8

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

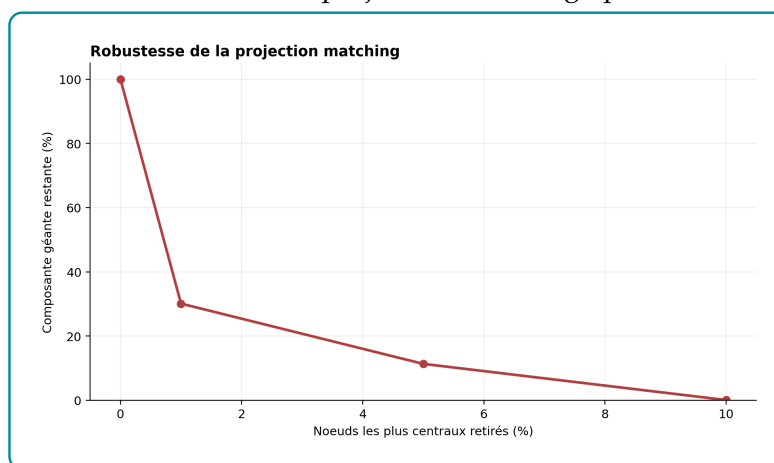
Figure A.4 – Communautés Louvain, distances et composantes connexes

Source : Nos travaux, diagnostics Neo4j

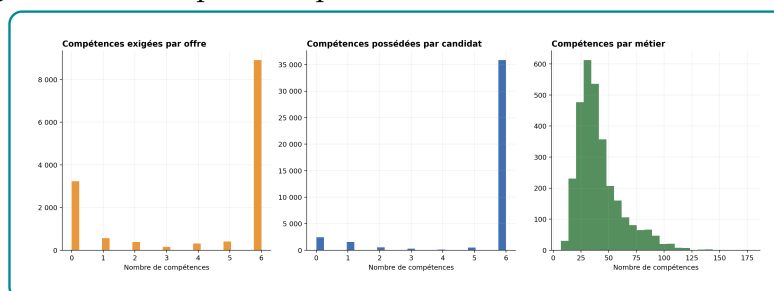
Table A.14 – Exemples de compétences centrales selon trois métriques

Degré	Valeur PageRank	Valeur Intermédiarité	Valeur
Coordonner le transport	6787	Coordonner le transport	0,00672
Utiliser différents moyens de communication			0,02919
Respecter le planning de travail dans les transports	6726	Respecter le planning de travail dans les transports	0,00664
Gérer le transport de fonds	6718	Gérer le transport de fonds	0,00664
Former le personnel			0,02297
Comptabilité	5822	Comptabilité	0,00571
Normes de qualité			0,02193

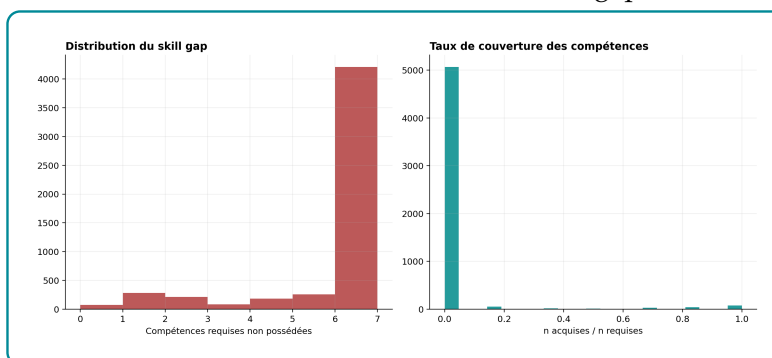
Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

Figure A.5 – Robustesse de la projection matching après retrait des hubs

Source : Nos travaux, diagnostics Neo4j

Figure A.6 – Compétences par offre, candidat et métier dans Neo4j

Source : Nos travaux, diagnostics Neo4j.

Figure A.7 – Distribution échantillonnée du skill gap candidat–offre

Source : Nos travaux, diagnostics Neo4j

Table A.15 – Résumé du déplacement de rang après reranking Neo4j

Indicateur	Valeur
Couples candidat–offre évalués	690
Candidats distincts	69
Rang moyen avant reranking pgvector	9,51
Rang moyen après reranking Neo4j/Optuna	5,50
Déplacement moyen de rang	+4,01
Déplacement médian	+1,00
Offres remontées	59,86 %
Offres rétrogradées	13,77 %
Offres inchangées	26,38 %

Source : Nos travaux, traitements Python de nettoyage et de diagnostic

Table des matières

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	v
Liste des tableaux	vi
Liste des figures	viii
Avant-propos	ix
Résumé	x
Abstract	xi
Introduction générale	1
 I Cadre théorique et conceptuel	 6
1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation	7
1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG	7
1.2 Fondements économiques et problématique d'appariement	9
1.2.1 Marché du travail, recherche d'emploi et frictions	9
1.2.2 Capital humain, tâches et inadéquation des compétences	10
1.3 Systèmes de recommandation et person-job fit	10
1.3.1 Approches par contenu, collaboratives et hybrides	10
1.3.2 Spécificité du person-job fit	11
1.4 LLMs, embeddings et graphes de connaissances	12
1.4.1 Grands modèles de langage et génération contrôlée	12
1.4.2 Embeddings, recherche sémantique et bases vectorielles	13
1.4.3 Graphes de connaissances et recommandation relationnelle	13
1.5 RAG, GraphRAG et orchestration agentique	14
1.5.1 De la RAG au GraphRAG	14
1.5.2 Agentic RAG, critic et traçabilité	15

1.5.3	Évaluer la performance et la factualité	16
2	Source de données et Approche méthodologique	19
2.1	Cadre méthodologique et sources de données	19
2.1.1	Démarche d'ingénierie appliquée retenue	19
2.1.2	Sources de données et leur rôle dans le système	22
2.1.3	Préparation et normalisation des données	23
2.1.4	Alignement des référentiels métiers et formations	24
2.2	Modélisation sémantique et structuration des connaissances	25
2.2.1	Choix du modèle d'embeddings et fine-tuning	25
2.2.2	Indexation vectorielle dans PostgreSQL et pgvector	25
2.2.3	Construction du graphe de connaissances sous Neo4j	26
2.2.4	Score hybride et logique de skill gap	26
2.3	Orchestration agentique et génération contrôlée	28
2.3.1	Architecture Agentic RAG et orchestration LangGraph	28
2.3.2	Text2Cypher et rôle du LLM	29
2.3.3	Modes d'accès et traçabilité des décisions	29
2.3.4	Protocole d'évaluation prévu du système	30
II	Présentation des résultats	31
3	Construction opérationnelle du système hybride emploi-compétences	32
3.1	Données exploitables et contraintes de qualité du système	32
3.1.1	Volumes disponibles après nettoyage	32
3.1.2	Qualité des variables utiles au matching emploi-compétences	33
3.1.2.1	Localisation exploitable pour filtrer sans exclure	33
3.1.2.2	Compétences et descriptions : signal utile mais hétérogène	34
3.1.3	Nettoyage réalisé et limites conservées	35
3.2	Adaptation du modèle d'embeddings au domaine emploi-compétences	35
3.2.1	Encodeur retenu et paires contrastives	35
3.2.2	Fine-tuning : contraintes de calcul et preuve de gain	37
3.2.2.1	Paramétrage contraint et dynamique d'apprentissage	37
3.2.2.2	Preuve de gain sur le ranking sémantique	39
3.2.3	Structure de l'espace vectoriel après fine-tuning	40
3.3	Mémoire vectorielle pgvector pour le retrieval	41
3.3.1	Schéma physique, tables et volumes indexés	42
3.3.2	Référentiels découpés en chunks interrogeables	43
3.3.3	Index et modes de recherche mobilisés	46

3.4	Neo4j : mémoire relationnelle du matching	46
3.4.1	Schéma, noeuds et relations mobilisés par le matching	46
3.4.2	Structure du graphe et risques de hubs	48
3.4.2.1	Taille réelle du graphe et couverture de la projection matching	48
3.4.2.2	Noeuds très connectés : signal utile ou bruit de popularité	49
3.4.2.3	Compétences pivots selon degré, PageRank et intermédiarité	50
3.4.2.4	Communautés Louvain et robustesse aux hubs	50
3.4.2.5	Skill gap observé entre profils et offres	51
3.5	Score hybride et reranking des recommandations	52
3.5.1	Formalisation du score hybride	52
3.5.2	Effet du reranking hybride	54
4	Évaluation de la performance du système	57
4.1	Protocole expérimental et métriques retenues	57
4.2	Retrieval sémantique et apport mesuré du graphe	58
4.2.1	Modèle d’embeddings spécialisé	58
4.2.2	Ablation pgvector seul contre pgvector enrichi par Neo4j	59
4.3	Validation fonctionnelle du GraphRAG	60
4.3.1	Requêtes relationnelles métier, compétence et offre	61
4.3.2	Skill gap, niveau NCF et comparaison GraphRAG	62
4.4	Reranking hybride et calibration du score	62
4.4.1	Déplacements de rang après reranking par le graphe	62
4.4.2	Pondérations optimales avec Optuna	63
4.5	Contrôle de génération par critic	65
4.5.1	Calibration contrastive du seuil de fidélité	65
4.5.2	Interprétation des décisions accept/revise	65
4.6	Orchestration agentique avec LangGraph	67
	Conclusion générale	I
	Bibliographie	II
A	Annexes	VI